AlumniLink — Complete Project Documentation

AI-Powered Alumni Mentorship Network & Modular LMS

Team Status / Code:	G239
Team Members:	Rashi Agarwal and Pranshi Soni
Track & Duration:	Application Development • 12 Weeks (Semester 3 Scope)
Target Domain:	Higher Education, Alumni Relations & Outcome-Driven Mentorship

Table of Contents

1. Project Overview
2. Business Requirements (BRD)
3. Product Requirements (PRD)
4. UX Requirements
5. Technical Requirements (TRD)
6. High-Level Design (HLD)
7. Database Design
8. Repository API Specification
9. Low-Level Design (LLD)
10. Frontend Architecture
11. Security Design
12. Testing Strategy
13. CI/CD Pipeline
14. Observability
15. Deployment Architecture
16. Cost Analysis
17. Project Roadmap
18. Team Responsibilities
19. Github Repository Structure
20. README
21. Architecture Decision Records (ADRs)

1. Project Overview

Mentorship Network with AI Matching

Team Status:	G239
Team Members:	Rashi Agarwal and Pranshi Soni
Project:	AlumniLink — AI-Powered Alumni Mentorship Network
Track & Duration:	Application Development • 20 Weeks
Target Domain:	Alumni Relations
Technology Stack:	Next.js 14, TypeScript, PostgreSQL, Prisma, pgvector, Gemini API, Gemini Embeddings, NextAuth, Zod, Nodemailer, Recharts

Problem Summary

Colleges often have thousands of alumni, but their professional knowledge and experience remain underutilized. Alumni information is commonly stored in spreadsheets, LinkedIn groups, or disconnected databases that students cannot effectively use to find the right mentor.

Students face three core problems:

- **Difficulty finding the right mentor:** Students may know that alumni are available but do not know which person is relevant to their specific career goals, skills, or desired industry.
- **Lack of structured mentorship:** Many mentorship interactions begin with a message or meeting but gradually become inactive because there are no defined goals, timelines, or follow-ups.
- **Hesitation in approaching professionals:** Students may not know how to write an introduction message or start a meaningful conversation with an alumnus.

Alumni also face an important problem:

- **Mentor overload:** A small number of popular or highly visible alumni may receive too many requests, while other relevant alumni receive none. Over time, overloaded mentors may stop participating completely.

For colleges and alumni relations teams, the problem is difficult to manage at scale:

- Alumni mentorship is often coordinated manually.
- Mentor capacity and availability are difficult to track.
- There is limited visibility into whether mentorship relationships remain active.
- Colleges cannot easily measure match quality, engagement, or programme completion.

The result is that colleges may have a large alumni network but no effective system for converting it into a sustainable mentorship programme.

Proposed Solution

A full-stack web application that connects students with relevant alumni mentors while managing mentor workload and providing structure throughout the mentorship journey.

AlumniLink combines:

- AI-assisted semantic matching based on career goals, skills, expertise, and career trajectory.
- Capacity-aware mentor recommendations so alumni are not assigned more mentees than they can realistically manage.
- Availability and eligibility constraints to remove unsuitable matches.
- Structured mentorship cycles with defined goals, sessions, check-ins, and completion tracking.
- AI-assisted communication tools that help students approach alumni and prepare for mentorship sessions.
- Engagement analytics that help colleges understand whether mentorship programmes are actually working.

The platform is designed as a college-focused mentorship infrastructure system, rather than a generic global mentor marketplace. AI supports recommendations and productivity, but it does not automatically assign mentors. Students and alumni remain in control of the final mentorship decision.

Core Capabilities

1. **Role-based authentication:** Student, Alumni, and Admin roles; secure access to role-specific dashboards and features.
2. **Student career profiles:** Skills and interests, career goals, target industry or role, experience and projects, and mentorship requirements.
3. **Alumni professional profiles:** Current role and industry, company and professional experience, areas of expertise, mentorship topics, and availability.
4. **Mentor capacity management:** Alumni define the maximum number of concurrent mentees; system tracks active mentees; backend prevents mentor capacity from being exceeded.
5. **AI-assisted mentor matching:** Profile information is converted into semantic embeddings; pgvector similarity search identifies relevant alumni; matching considers career trajectory and goal fit, not only department.
6. **Deterministic constraint enforcement:** Capacity validation, availability validation, mentor eligibility checks, and conflict-of-interest checks where applicable.
7. **Mentorship request and approval workflow:** Students browse recommended alumni and send mentorship requests; alumni can accept or reject requests; active mentorship is created after approval.
8. **Structured mentorship cycles:** Defined mentorship duration, clear goals, session planning, regular check-ins, and progress/completion tracking.
9. **AI-generated introduction messages:** Personalized draft messages based on student goals and alumni expertise; students can edit and approve messages before sending.
10. **AI-generated first-session agendas:** Personalized discussion topics, career goal alignment, and suggested questions and action points.
11. **AI session summaries and action items:** Convert session notes into concise summaries, extract important recommendations, and generate actionable next steps.
12. **Student and Alumni dashboards:** Active mentorships, pending requests, upcoming sessions, goals and action items, and mentorship progress.
13. **Admin analytics dashboard:** Total students and alumni, active mentorships, available mentor capacity, match acceptance rate, session engagement, and mentorship cycle completion.
14. **Feedback and match evaluation:** Students evaluate mentor relevance; alumni evaluate workload and mentorship experience; feedback is used to measure product effectiveness.

Business Value

Stakeholder	Value
Students	Find relevant alumni mentors based on career goals instead of manually searching through large alumni networks.
Alumni	Give back to students while controlling availability, expertise areas, and maximum mentoring workload.
Colleges & Universities	Transform underused alumni databases into an active and measurable mentorship programme.
Alumni Relations Teams	Reduce manual coordination and gain visibility into mentorship engagement and outcomes.
Placement & Career Teams	Help students receive career guidance from professionals with relevant industry experience.
Students (Portfolio/OJT)	Demonstrates full-stack application development, AI integration, semantic search, database design, RBAC, analytics, and real-user validation.

Why Would Alumni Join?

AlumniLink is designed to solve the biggest barrier for potential mentors: giving back should never become an unmanageable burden or a source of spam.

Alumni can:

- **Cap their workload:** Set a strict limit on active mentees (e.g., maximum 2 per term) that auto-locks when full.

- **Control their availability:** Choose their own schedule and specify exact mentoring domains (e.g., backend systems, off-campus hiring, resume reviews).
- **Eliminate aimless calls:** Receive pre-screened requests with student blockers, resumes, and an AI-generated 3-point agenda ready before the call.
- **Earn verified career capital:** Gain institutional recognition, mentor badges, and trackable leadership credentials to showcase on LinkedIn.
- **Build a trusted referral pipeline:** Vet high-potential juniors directly through structured problem-solving sessions for internal company referral bonuses.
- **Engage in bounded cycles:** Commit only to fixed-term, milestone-driven cycles (e.g., 2–4 weeks) instead of indefinite, open-ended chats.

> Instead of facing an endless flood of unorganized LinkedIn DMs and unvetted referral asks, mentors get a protected, high-impact channel completely on their own terms.
> **Core Value Proposition for Alumni:** High-impact mentorship and talent scouting with zero burnout and total time control.

Why Would Students Join?

Students join AlumniLink because finding the right mentor is often harder than simply finding a mentor.

The platform helps students:

- Discover alumni relevant to their actual career goals.
- Find mentors based on career trajectory, not only department.
- Understand why a mentor was recommended.
- Approach professionals with AI-assisted introduction drafts.
- Enter structured mentorship cycles instead of having one-time conversations.
- Track advice and action items after mentorship sessions.

Example: A student interested in moving into Product Management may benefit more from an alumnus who transitioned from Software Engineering to Product Management than from a randomly selected alumnus from the same department.

> **Core Value Proposition for Students:** Find someone who has already travelled a career path similar to the one you want to pursue.

What Makes AlumniLink Different?

The mentorship market already contains mentor marketplaces and mentoring platforms. AlumniLink does not attempt to compete by simply providing another directory of mentors. Its differentiation comes from combining four ideas for the higher education ecosystem:

1. **College-Specific Alumni Context:** Focuses on a college's existing alumni network. Alumni understand the educational environment and context of current students, allowing institutions to verify and manage their own community.
2. **Career Trajectory Matching:** Instead of only asking *"Which alumni studied in the same department?"*, AlumniLink considers where the student is currently, what skills they have, where they want to go, and which alumni have followed a similar or relevant career path.
3. **Mentor Load Management:** The platform does not only identify the most relevant mentor; it also checks if that mentor can realistically take another mentee based on expertise, current count, capacity limits, and availability.
4. **Structured Mentorship Lifecycle:** The product supports the complete journey: `Discover → Match → Request → Accept → Set Goals → Meet → Track Action Items → Check-in → Complete → Feedback`.

Business Perspective

AlumniLink can be positioned as a **B2B2C mentorship platform**.

- **Potential Customers:** Colleges, universities, alumni relations departments, and career/placement offices.
- **End Users:** Students and Alumni.
- **Delivery Path:** `College → AlumniLink → Students and Alumni`.

The long-term business opportunity is to provide mentorship infrastructure that institutions can use to activate and manage their existing alumni network. Future SaaS offerings may include institution-specific branding, advanced analytics, department dashboards, custom programmes, and advanced administrative governance.

Core Product Hypothesis & Success Criteria

> **Core Hypothesis:** Students will find mentorship more useful when they are recommended alumni based on career goals and trajectory, while alumni will be more willing to participate when they can control their mentoring capacity and time commitment. Evaluated through a real-user pilot.

The MVP will aim to demonstrate:

- At least 70% of evaluated users find one of their top 3 mentor recommendations useful.

- Zero mentor capacity violations during functional and stress testing.
- Successful completion of the complete mentorship lifecycle.
- Students find AI-assisted introductions and session agendas useful.
- Alumni report that their mentoring workload is manageable.
- Mentorship cycle completion and engagement are tracked.
- The application is deployed and publicly accessible with responsive layouts across mobile and desktop.
- Loading, error, and empty states are properly handled, and AI failure fallback is verified.
- API keys remain securely stored on the server.

What Makes This Different from a Typical College Project

- Solves a real two-sided platform problem involving students and alumni.
- Goes beyond simple CRUD by implementing semantic profile matching.
- Uses vector embeddings and pgvector for application-level recommendations.
- Combines AI recommendations with deterministic business constraints.
- Implements mentor capacity management to prevent workload overload.
- Models a complete mentorship lifecycle rather than only a mentor directory.
- Uses AI for practical productivity features such as introductions, agendas, and summaries.
- Implements role-based access control for multiple user types.
- Includes real-user pilot testing and measurable product validation.
- Tracks engagement and mentorship outcomes through analytics.
- Includes responsible AI boundaries where AI assists decisions but does not automatically make them.
- Demonstrates production-oriented application architecture suitable for a full-stack development portfolio.

2. Business Requirements (BRD)

AI-Powered Alumni Mentorship Network

Project Name:	AlumniLink – AI-Powered Alumni Mentorship Network
Team Status:	G239
Team Members:	Rashi Agarwal and Pranshi Soni
Track:	Application Development
Academic Context:	Semester 3
Duration:	20 Weeks
Domain:	Alumni Relations

1. Executive Summary

AlumniLink is a full-stack web application designed to help colleges transform their underutilized alumni networks into structured and sustainable mentorship programmes. Many colleges have large databases of alumni, but students often struggle to identify which alumni can help them with specific career goals. At the same time, alumni who are willing to mentor may receive too many requests, while many other relevant alumni remain underutilized.

AlumniLink addresses this problem through:

- Career and expertise-based alumni discovery.
- AI-assisted semantic matching between students and alumni.
- Mentor capacity and availability management.
- Structured mentorship cycles.
- AI-assisted communication and session preparation.
- Session summaries and action item tracking.
- Engagement and programme analytics.

The platform serves three primary user groups:

- Students seeking career guidance.
- Alumni willing to mentor students.
- College administrators managing alumni engagement and mentorship programmes.

> **Central Product Principle:** Students should be able to find relevant mentors, while alumni should be able to contribute without becoming overloaded.
> AlumniLink is positioned as a college-focused mentorship infrastructure platform, rather than a generic global mentor marketplace.

2. Business Problem Statement

2.1 Current Situation

Colleges often maintain alumni information through spreadsheets, alumni databases, LinkedIn groups, WhatsApp groups, alumni associations, and informal networks. Although valuable professional knowledge exists within these networks, students often cannot efficiently access it.

A typical mentorship programme currently works as follows:

```
Student needs guidance
        ↓
Searches manually for alumni
        ↓
Finds a few popular professionals
        ↓
Sends messages
        ↓
Some alumni receive too many requests
        ↓
Mentorship becomes unstructured
        ↓
Communication gradually stops
```

This process is difficult to scale.

2.2 Problems Faced by Students

- **Difficulty identifying the right mentor:** Students may know that alumni work in different industries but do not know who works in their target industry, who has experience in their desired role, who has followed a similar career path, or who is currently available to mentor.
- **Information overload:** A large alumni network can be overwhelming. Students may have access to hundreds or thousands of alumni profiles but still struggle to answer: *"Who should I approach?"*
- **Hesitation in communication:** Students may hesitate to contact professionals because they do not know how to introduce themselves, how to explain their goals, what questions to ask, or how to start a professional conversation.
- **Lack of continuity:** Many mentorship interactions end after one or two conversations because there are no defined goals, no clear duration, no follow-up mechanism, and no structured action plan.

2.3 Problems Faced by Alumni

- **Mentor overload:** Highly visible or popular alumni may receive a large number of requests, resulting in too many messages, excessive time commitments, difficulty managing mentees, and reduced willingness to continue mentoring.
- **Irrelevant requests:** Alumni may receive requests from students whose goals do not match their expertise (e.g., Alumni Expertise: Software Engineering vs. Student Requirement: Higher Studies in Psychology). Such requests are not useful for either person.
- **Unclear commitment:** Alumni may hesitate to volunteer because they do not know how many students they will need to mentor, how long the mentorship will continue, or how much time will be required.

2.4 Problems Faced by Colleges

Colleges and alumni relations teams face challenges including manual mentorship coordination, difficulty tracking mentor capacity, difficulty identifying inactive mentorship relationships, limited visibility into engagement, limited measurement of programme outcomes, and underutilized alumni networks.

3. Business Opportunity

AlumniLink creates an opportunity to convert an existing alumni database into an active mentorship ecosystem:

```
Existing Alumni Network → Structured Profiles → Relevant Discovery and Matching → Managed Mentorship Requests
        → Structured Mentorship Cycles → Engagement Tracking → Measurable Outcomes
```

The platform does not require a college to build a new community from zero; instead, it helps institutions activate a community they already have.

4. Product Vision

To create a sustainable mentorship ecosystem where students can easily connect with relevant alumni, alumni can contribute within manageable limits, and colleges can measure the impact of their mentorship programmes.

5. Business Objectives

- **BO-01 (Improve mentor discovery):** Enable students to discover relevant alumni based on career goals, skills, industry interests, expertise, and career trajectory.
- **BO-02 (Prevent mentor overload):** Enable alumni to control availability, areas of expertise, and maximum number of concurrent mentees. The system must ensure that mentor capacity is not exceeded.
- **BO-03 (Improve mentorship engagement):** Provide a structured mentorship lifecycle with defined goals, defined duration, sessions, check-ins, action items, and completion tracking.
- **BO-04 (Reduce communication barriers):** Help students approach alumni through AI-assisted introduction drafts, AI-generated session agendas, and structured conversation prompts.
- **BO-05 (Improve programme visibility):** Provide administrators with information about active mentorships, available mentor capacity, engagement levels, match quality, and programme completion.

6. Target Users

6.1 Primary User: Student

- **Profile:** Students exploring career opportunities, preparing for industry roles, considering higher studies, building specific technical skills, interested in entrepreneurship, or looking for career guidance.
- **Primary Goal:** Find a relevant person who can provide useful guidance for my career goals.
- **Key Problems:** Does not know whom to approach, does not know how to initiate contact, receives generic career advice, and lacks structured guidance.
- **Value Provided:** Relevant alumni recommendations, clear mentor profiles, structured mentorship, AI-assisted communication, and actionable career guidance.

6.2 Primary User: Alumni

- **Profile:** Former students who have professional experience, industry knowledge, career experience, and a willingness to guide students.
- **Primary Goal:** Help students without creating an unmanageable time commitment.
- **Key Problems:** Receives too many requests, receives irrelevant requests, has limited availability, and does not want an indefinite mentorship commitment.
- **Value Provided:** Control over mentee capacity, control over availability, more relevant student requests, defined mentorship cycles, and a clear mentorship structure.

6.3 Primary User: Administrator

- **Profile:** Users from alumni relations teams, career development teams, placement cells, or department mentorship programmes.
- **Primary Goal:** Manage and understand the effectiveness of the mentorship programme.
- **Key Problems:** Manual coordination, lack of engagement visibility, and difficulty measuring programme outcomes.
- **Value Provided:** Centralized mentorship management, programme analytics, mentor capacity visibility, and engagement tracking.

7. Stakeholders

Stakeholder	Role	Interest
Students	End Users	Finding relevant mentors and career guidance
Alumni	End Users / Mentors	Giving back while managing workload
College Administration	Potential Customer	Improving student and alumni engagement
Alumni Relations Team	Programme Manager	Managing alumni participation
Placement & Career Team	Programme Stakeholder	Supporting student career development
Development Team	Product Builders	Building and validating the MVP

8. Business Model Perspective

AlumniLink can be positioned as a **B2B2C platform**.

- **Potential Business Customers:** Colleges, universities, and educational institutions.
- **End Users:** Students and alumni.



In the future, institutions could potentially subscribe to the platform as a Software-as-a-Service solution. The MVP will not include payments or subscription management.

9. Business Value Proposition

- **For Students:** *"Find someone who has already travelled a career path similar to the one you want to pursue."* Easier mentor discovery, more relevant recommendations, career-contextual guidance, structured mentorship, and help initiating professional conversations.
- **For Alumni:** *"Help students without becoming overwhelmed."* Workload control, relevant requests, clear time commitment, structured mentorship cycles, and the ability to decide whom to mentor.
- **For Colleges:** *"Transform an underused alumni database into an active and measurable mentorship programme."* Centralized mentorship management, reduced manual coordination, better alumni engagement, programme analytics, and measurable outcomes.

10. Competitive Differentiation

AlumniLink differentiates itself through four interconnected pillars rather than merely providing mentor profiles or AI matching:

- **10.1 College-Specific Context:** Focuses on an institution's existing alumni community, leveraging shared educational context, greater institutional relevance, easier community verification, and stronger alumni-student connections.

- **10.2 Career Trajectory Matching:** Traditional matching maps `Student Department → Same Department Alumni`. AlumniLink integrates `Current Skills + Career Goals + Industry Interest + Career Trajectory + Alumni Expertise` to surface non-obvious, high-value matches.
- **10.3 Mentor Load Management:** Evaluates both relevance and realistic availability, factoring in expertise relevance, semantic similarity, current mentee count, maximum capacity limits, and availability constraints.
- **10.4 Structured Mentorship Lifecycle:** Manages the complete journey: `Discover → Match → Request → Accept → Set Goals → Sessions → Action Items → Check-ins → Completion → Feedback`.

11. Business Requirements

Req ID	Requirement Name	System Capability	Underlying Business Need
`BR-01`	User Role Management	Support Student, Alumni, and Admin user personas.	Different users require distinct capabilities and access levels.
`BR-02`	Student Profile Setup	Capture career goals, skills, target industry, role, and experience.	Requires sufficient data to identify relevant mentors.
`BR-03`	Alumni Profile Setup	Capture professional role, company, expertise, availability, and capacity.	Students need sufficient information to evaluate potential mentors.
`BR-04`	Capacity Management	Define max mentees, track active count, and prevent overbooking.	Mentor overload reduces long-term alumni participation.
`BR-05`	Discovery & Matching	Recommend alumni using goals, skills, expertise, and trajectory fit.	Manual discovery does not scale across large alumni networks.
`BR-06`	Eligibility Constraints	Enforce deterministic capacity, availability, and conflict rules.	AI alone may recommend unavailable mentors.
`BR-07`	Mentorship Requests	Support browse, request dispatch, and alumni accept/reject workflow.	Both parties must maintain control of relationships.
`BR-08`	Structured Cycles	Defined start/end dates, goals, sessions, check-ins, and completion status.	Open-ended mentorship frequently loses momentum.
`BR-09`	AI Introduction Drafts	Generate editable draft intros for student review and approval.	Students hesitate to initiate professional outreach.
`BR-10`	AI Session Agendas	Suggest discussion topics based on goals and mentor expertise.	Users may not know how to structure first conversations.
`BR-11`	AI Session Summaries	Synthesize meeting notes into key points and action items.	Important advice can be forgotten between sessions.
`BR-12`	Action Item Tracking	Allow users to assign, track, and monitor action item progress.	Mentorship should lead to actionable progress.
`BR-13`	Milestone Check-ins	Provide structured check-ins during active cycles.	Identifies stalled or inactive mentorship relationships.
`BR-14`	System Notifications	Notify users of new requests, decisions, sessions, and check-ins.	Prevents users from forgetting to return to platform.
`BR-15`	Programme Analytics	Display user counts, active cycles, capacity usage, and completion rates.	Admins need evidence of whether programme is working.
`BR-16`	Feedback Collection	Collect ratings on match relevance, usefulness, and workload.	Provides measurable feedback for iteration.

12. Functional Requirements

- **12.1 Authentication:** User registration, login, secure session management, role-based access control, and protected routes.
- **12.2 Student Features:** Profile CRUD, goal definition, view recommendations/profiles, dispatch requests, track cycles, manage sessions, complete action items, and submit feedback.
- **12.3 Alumni Features:** Profile CRUD, expertise definition, availability/capacity toggles, review/accept/reject requests, manage active cycles, log sessions, and submit feedback.
- **12.4 Administrator Features:** View aggregate stats, monitor active mentorships, track capacity utilization, review engagement metrics, and analyze feedback trends.

13. Non-Functional Requirements

- **NFR-01 (Security):** API keys kept server-side, accounts protected via authentication, RBAC enforced, and sensitive data secured.
- **NFR-02 (Performance):** Pages load efficiently, matching returns within acceptable interactive timeframes, and database queries are optimized.
- **NFR-03 (Reliability):** API failures handled gracefully, AI errors do not break core workflows, and actionable error messages are displayed.
- **NFR-04 (Usability):** Intuitive interface for students and alumni, visible CTAs, and responsive desktop/mobile layouts.
- **NFR-05 (Maintainability):** Modular code, separation of UI and business logic, consistent ORM access, and structured input validation.
- **NFR-06 (Privacy):** Strict role-based information access, no unnecessary personal data exposure, and minimal data passed to AI APIs.

14. AI Requirements and Boundaries

AI Use Cases: Semantic matching embeddings, personalized introduction drafts, first-session agenda generation, and note summarization/action-item extraction.

> **Responsible AI Boundary:** AI must never automatically assign mentors, force acceptance, or override capacity and availability constraints. AI assists; deterministic business rules enforce boundaries; humans make final decisions.

15. Business Rules

- **Rule 1 (Mentor Capacity):** `Current Mentees < Maximum Mentees`. No mentorship created if mentor is full.
- **Rule 2 (Alumni Approval):** Mentorship only activates upon explicit alumni acceptance.
- **Rule 3 (AI Recommendations):** Recommendations are suggestions, never auto-assignments.
- **Rule 4 (User Control):** Students choose whom to request; alumni choose whom to accept.
- **Rule 5 (Structured Duration):** Every cycle has a defined duration, goals, and measurable status.

16. Success Metrics & Pilot Plan

Success Targets: ≥ 70% of participants find top-3 recommendations useful; zero capacity violations; successful complete lifecycle completion; validated workload manageability.

Pilot Cohort: ~25 students and ~10 alumni. Evaluates match relevance, request relevance, workload balance, and cycle completion.

17. Scope & Constraints

- **MVP Scope:** User auth, profile CRUD, semantic vector search, deterministic filtering, request workflow, structured cycles, AI assistance (intros, agendas, summaries), dashboards, and feedback.
- **Out of Scope:** Payments, native mobile apps, global directories, and unassisted auto-assignments.
- **Constraints:** 2-member team, 8-to-20 weeks delivery timeline, free-tier resources, and server-side key management.

18. Business Requirement Traceability Summary

Business Challenge	Platform Feature Solution	Primary User
Students cannot find relevant mentors	AI-assisted mentor matching	Students
Popular mentors become overloaded	Capacity management	Alumni
Requests may be irrelevant	Expertise & trajectory matching	Students & Alumni
Students hesitate to reach out	AI introduction drafts	Students
Mentorship becomes inactive	Structured mentorship cycles	Students & Alumni
Sessions lack direction	AI-generated agendas	Students & Alumni
Advice is forgotten	AI summaries & action items	Students
Colleges lack programme visibility	Analytics dashboard	Admin
Outcomes are difficult to measure	Feedback & engagement tracking	Admin

19. Final Business Statement

AlumniLink is designed around a simple business principle: a successful mentorship programme requires more than connecting two people. It must help students find relevant guidance, protect alumni from overload, provide structure to the relationship, and give institutions visibility into whether the programme is creating real engagement. The MVP validates whether career-aware matching, mentor capacity management, and structured mentorship cycles make college alumni mentorship sustainable and impactful.

3. Product Requirements (PRD)

Project Title:	**AlumniLink** – AI-Powered Mentorship Network & Learning Management System
Team Status / Code:	G239
Team Members:	Rashi Agarwal, Pranshi Soni
Track & Duration:	Application Development • 12 Weeks
Target Domain:	Higher Education, Alumni Relations & Outcome-Driven Mentorship
Technology Stack:	Next.js 14 (TypeScript), FastAPI (Python), WebRTC (P2P), PostgreSQL 16, pgvector, Hugging Face Sentence-Transformers (all-MiniLM-L6-v2), Prisma ORM, Zod, Cloudflare R2, Nodemailer, Recharts

1. Product Vision

Deliver a self-contained, 12-week mentorship platform and Learning Management System (LMS) where college alumni conduct structured video training, track student progress through custom modules, evaluate assignments and tests, and utilize self-hosted AI matching–all built with zero third-party video subscriptions or paid infrastructure dependencies.

2. Product Goals & Core Tenets

Goal / Tenet	Target Metric	Architectural Constraint
Zero Mentor Overload	100% enforcement of concurrent student limits (N ≤ 3) via atomic database locks.	Alumni must explicitly accept mentees; automated assignment is strictly prohibited.
In-House Real-Time Video	Sub-250ms audio/video latency over 1-on-1 peer connections.	Zero reliance on external meeting providers (Zoom, Google Meet); peer-to-peer WebRTC via FastAPI signaling.
Self-Hosted Semantic Matching	Sub-100ms vector generation and similarity search (< 384 dimensions).	CPU-based Sentence-Transformers run locally in Python/ FastAPI; embeddings stored directly in pgvector.
Verifiable Student Evaluation	100% of completed cycles feature verified attendance, assignment grades, and final exam scoring.	Mentors control modular curriculum authoring, grading points, and track completion sign-offs.
$0 Technical Infrastructure	Platform operates entirely within verifiable free tiers (Vercel, Render/Railway, Neon, Cloudflare R2).	All operational budgets are allocated strictly toward human incentives (alumni rewards), not servers.

3. Features & MoSCoW Prioritization

Must Have (P0 — MVP Launch Blockers)

ID	Feature	Description	Technical Boundary
`F-001`	**Role-Based Auth**	Role segmentation for `STUDENT`, `ALUMNI`, and `ADMIN` with route guards.	NextAuth.js (JWT in HTTP-only cookies) + FastAPI header token verification.
`F-002`	**Self-Hosted Embeddings**	Generate dense 384-dim trajectory and goal embeddings in-house without paid APIs.	Hugging Face `sentence-transformers` (`all-MiniLM-L6-v2`) run on CPU via FastAPI.
`F-003`	**Capacity Vector Match**	Semantic search ranking available alumni while filtering out mentors at full capacity.	PostgreSQL `pgvector` HNSW index with composite filter (`activeMentees < maxMentees`).
`F-004`	**In-House Video Calling**	Browser-to-browser 1-on-1 video rooms without Zoom or Google Meet.	Native WebRTC, FastAPI WebSocket signaling, Google STUN + Metered.ca TURN fallback.
`F-005`	**Mentor Dashboard & LMS**	Dedicated mentor console to view mentees, author course modules, and upload notes.	Next.js 14 App Router, Cloudflare R2 for notes/PDFs, PostgreSQL relational schema.
`F-006`	**Assignments & Submissions**	Mentors post homework tasks; students upload code/PDF files for mentor review.	FastAPI file upload pipeline streaming to S3-compatible Cloudflare R2 storage.
`F-007`	**Final Test & Point Grading**	End-of-course test evaluation where mentor grades performance, assignments, and test points.	Relational scoring schema; composite rating aggregation stored on cycle completion.
`F-008`	**Attendance & Inactivity Alert**	Detect missed sessions or abandoned pairs and trigger transactional nudges.	FastAPI WebSocket heartbeat listener + Nodemailer/Resend transactional alert queue.

Should Have (P1 — Core Experience Enhancements)

ID	Feature	Description	Technical Boundary
`F-009`	**AI Outreach Assistant**	Assists students in drafting structured outreach pitches based on mentor expertise.	Gemini 1.5 Flash structured prompt with client-side editable review box.
`F-010`	**Pre-Session Agendas**	Auto-generates a 3-point targeted agenda 24 hours prior to scheduled video calls.	Scheduled cron endpoint invoking LLM brief generation.
`F-011`	**Admin Analytics Hub**	Cohort-level oversight tracking cycle completion rates, test scores, and attendance rates.	Recharts data visualizer querying aggregated PostgreSQL materialized views.

Could Have (P2)

ID	Feature	Description / Exclusion Justification
`F-012`	**Impact Portfolio PDF**	Downloadable dossier documenting mentor teaching hours and mentee scores for company reviews.
`F-013`	**Curriculum Templates**	Pre-loaded 4-week module blueprints (Backend, AI/ML) mentors can import in one click.

4. User Stories & Acceptance Criteria

EPIC 1: MATCHING, CAPACITY & IN-HOUSE EMBEDDINGS

US-001 (Self-Hosted Semantic Matching): As a Student, I want the platform to match my goals against mentor profiles using self-hosted embeddings, so that I find relevant mentors quickly without external API dependencies.

- FastAPI generates 384-dimensional vector embeddings on CPU using `all-MiniLM-L6-v2`.
- PostgreSQL executes cosine similarity queries (`<=>`) via `pgvector` under 100ms.
- SQL query filters out mentors where `activeMentees >= maxMentees` prior to vector distance sorting.

US-002 (Capacity Protection): As an Alumni Mentor, I want incoming requests locked when my slots are filled, so that I am never overcommitted.

- Request creation executes inside a Prisma interactive transaction (`prisma.$transaction`).
- System verifies that active cycles plus pending requests do not exceed `maxMentees` (1 ≤ N ≤ 3).
- If capacity is reached, subsequent requests are rejected gracefully with informative UI messaging.

EPIC 2: IN-HOUSE VIDEO SYSTEM & ATTENDANCE TRACKING

US-003 (Zero-Third-Party Video Calls): As a Student and Mentor, I want to conduct 1-on-1 video sessions directly inside our platform, so that we don't have to manage external Zoom or Google Meet links.

- WebRTC establishes a direct browser-to-browser peer connection using SDP offer/answer handshakes.
- FastAPI WebSockets manage real-time signaling and ICE candidate exchange.
- Google Public STUN handles IP discovery, with Metered.ca TURN server configured for firewall fallback.

US-004 (Attendance & Inactivity Tracking): As an Administrator and Mentor, I want the system to record whether calls occur and alert absent parties, so that mentorship pairs stay accountable.

- Session start and disconnect events are logged to the database via WebSocket connection lifecycle hooks.
- If zero participant pings are detected 10 minutes past the scheduled time, a background task triggers.
- Automated transactional reminder emails are dispatched to both parties via Nodemailer.

EPIC 3: LEARNING MANAGEMENT SYSTEM (LMS) & STUDENT DASHBOARD

US-005 (Modular LMS Authoring): As an Alumni Mentor, I want a dedicated dashboard to manage my students, add modules, and attach notes, so that I can deliver a structured 12-week curriculum.

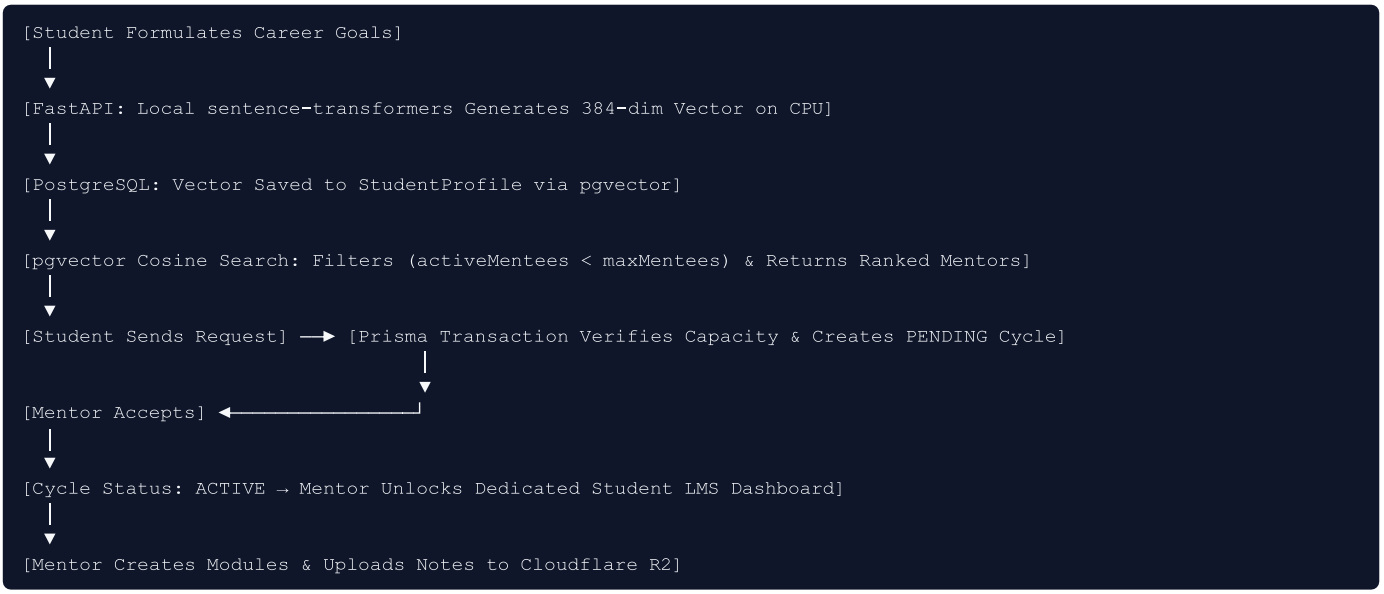
- Clicking a student card opens a dedicated Student Management Workspace.
- Mentor can create, update, and order course modules with markdown text and reference links.
- PDF lecture notes and learning resources are uploaded to Cloudflare R2 via secure presigned URLs.

US-006 (Assignment Submission & Final Exam Grading): As an Alumni Mentor, I want to review submitted student assignments and grade their final test, so that I can provide an objective evaluation score.

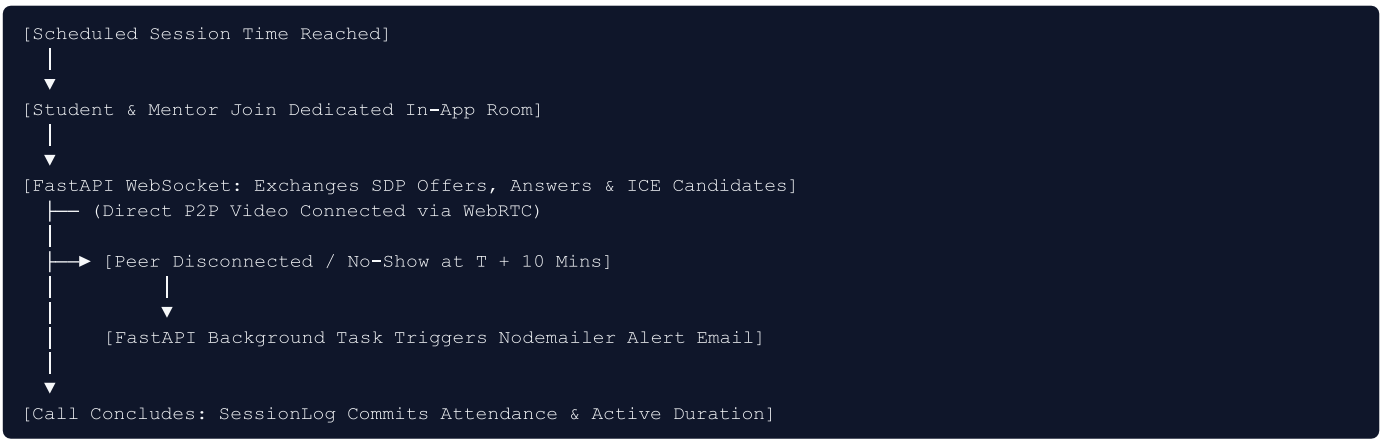
- Students submit assignment deliverables (GitHub links, PDF reports) for each module.
- Mentor assigns points and qualitative feedback per submission.
- Mentor submits a final evaluation covering Activeness, Assignments, and Exam Score, calculating an aggregate grade.

5. End-to-End User & System Flows

FLOW 1: MATCH DISCOVERY TO ACTIVE COURSE SETUP



FLOW 2: WEBRTC VIDEO CALL & INACTIVITY MONITOR



6. Technical Architecture & Database Schema

Prisma Schema Specification (`schema.prisma`)

```
datasource db {
  provider   = "postgresql"
  url        = env("DATABASE_URL")
  extensions = [pgvector(map: "vector")]
}

generator client {
  provider        = "prisma-client-js"
  previewFeatures = ["postgresqlExtensions"]
}

enum Role {
  STUDENT
  ALUMNI
  ADMIN
}

enum CycleStatus {
  PENDING
  ACTIVE
  COMPLETED
  EXPIRED_CAPACITY
  CANCELLED
}

model User {
  id             String          @id @default(cuid())
  email          String          @unique
  name           String
  role           Role            @default(STUDENT)
  createdAt      DateTime        @default(now())
  updatedAt      DateTime        @updatedAt
  studentProfile StudentProfile?
  mentorProfile  MentorProfile?
}

model StudentProfile {
  id            String                      @id @default(cuid())
  userId        String                      @unique
  user          User                        @relation(fields: [userId], references: [id], onDelete: Cascade)
  branch        String
  semester      Int
  careerGoals   String                      @db.Text
  skills        String[]
  goalEmbedding Unsupported("vector(384)")?
  cycles        MentorshipCycle[]
}

model MentorProfile {
  id                  String                      @id @default(cuid())
  userId              String                      @unique
  user                User                        @relation(fields: [userId], references: [id], onDelete: Cascade)
  currentRole         String
  currentCompany      String
  yearsExperience     Int
  expertise           String[]
  maxMentees          Int                         @default(2)
  activeMentees       Int                         @default(0)
  isAvailable         Boolean                     @default(true)
  trajectoryEmbedding Unsupported("vector(384)")?
  cycles              MentorshipCycle[]
}

model MentorshipCycle {
  id             String        @id @default(cuid())
  studentId      String
  student        StudentProfile @relation(fields: [studentId], references: [id])
  mentorId       String
  mentor         MentorProfile  @relation(fields: [mentorId], references: [id])
  status         CycleStatus    @default(PENDING)
  goalSummary    String        @db.Text
  startDate      DateTime?
  endDate        DateTime?
  createdAt      DateTime      @default(now())

  // LMS Modules, Sessions & Evaluation
  modules        CourseModule[]
  sessions       SessionLog[]
  evaluation     FinalEvaluation?
```

```
}

model CourseModule {
  id          String        @id @default(cuid())
  cycleId     String
  cycle       MentorshipCycle @relation(fields: [cycleId], references: [id], onDelete: Cascade)
  title       String
  description String        @db.Text
  notesUrl    String?       // Cloudflare R2 URL
  assignments Assignment[]
}

model Assignment {
  id          String       @id @default(cuid())
  moduleId    String
  module      CourseModule @relation(fields: [moduleId], references: [id], onDelete: Cascade)
  title       String
  instructions String      @db.Text
  maxPoints   Int          @default(100)
  submissions Submission[]
}

model Submission {
  id           String     @id @default(cuid())
  assignmentId String
  assignment   Assignment @relation(fields: [assignmentId], references: [id], onDelete: Cascade)
  fileUrl      String     // Cloudflare R2 URL
  studentNotes String?    @db.Text
  awardedScore Int?
  mentorReview String?    @db.Text
  submittedAt  DateTime   @default(now())
}

model FinalEvaluation {
  id              String          @id @default(cuid())
  cycleId         String          @unique
  cycle           MentorshipCycle @relation(fields: [cycleId], references: [id], onDelete: Cascade)
  activenessScore Int             // 1-100
  assignmentScore Int             // 1-100
  finalExamScore  Int             // 1-100
  overallFeedback String          @db.Text
  evaluatedAt     DateTime        @default(now())
}

model SessionLog {
  id          String          @id @default(cuid())
  cycleId     String
  cycle       MentorshipCycle @relation(fields: [cycleId], references: [id], onDelete: Cascade)
  sessionDate DateTime        @default(now())
  durationMin Int             @default(0)
  attended    Boolean         @default(false)
  notes       String?         @db.Text
}
```

7. Quality Assurance & Release Gates

Verification Gate	Target Standard	Status / Verification Mode
In-House Video Traversal	WebRTC connections connect successfully across campus Wi-Fi networks using STUN/TURN.	Tested across isolated LAN and NAT-restricted mobile hotspots.
Capacity Lock Atomicity	Concurrent request surges cannot exceed mentor `maxMentees` limit.	Prisma interactive transaction stress tests with 10 concurrent requests.
Local Embedding Latency	In-house `all-MiniLM-L6-v2` completes 384-dim vector generation in < 50ms on CPU.	FastAPI benchmark endpoint under 100 sequential requests.
LMS File Boundary	Assignment PDFs/code archives upload securely to Cloudflare R2 with signed verification.	Direct presigned URL upload tests; zero payloads stored directly in PostgreSQL.
$0 Cloud Budget Audit	All infrastructure operates completely within free quotas (Vercel, Neon, Cloudflare R2).	Production dashboard billing verification confirming zero cloud expenses.

4. UX Requirements

User Experience, Information Architecture & Interaction Design

Track:	Application Development
Team:	G239
Team Members:	Rashi Agarwal and Pranshi Soni
Duration:	20 Weeks

1. Information Architecture

```
AlumniLink Platform
├── Authentication
│   ├── Login
│   ├── Sign Up
│   └── Role Selection
├── Student Portal
│   ├── Dashboard
│   ├── My Profile
│   ├── Find Mentors
│   ├── My Requests
│   └── My Mentorships
├── Alumni Portal
│   ├── Dashboard
│   ├── My Profile
│   ├── Mentorship Requests
│   └── My Mentorships
├── Admin Portal
│   ├── Dashboard
│   ├── Users
│   ├── Mentorship Cycles
│   └── Analytics
└── Shared
    ├── Notifications
    ├── AI Introduction Draft
    ├── AI Session Agenda
    ├── AI Session Summary
    └── Settings
```

2. User Flows

Student Onboarding → Mentor Discovery

```
Sign Up
→ Select Student
→ Complete Profile
→ Add Education, Skills and Goals
→ Submit Profile
→ Generate/Update Embedding
→ Student Dashboard
→ Find Mentors
→ View AI Recommendations
→ View Mentor Profile
→ View Match Explanation
→ Request Mentorship
```

Student → Mentorship Request

```
Find Mentor
→ View Profile
→ Check Availability and Capacity
→ Request Mentorship
→ Write Introduction
→ Optional AI Draft
→ Review/Edit
→ Send Request
→ PENDING
```

Alumni → Accept Request

```
Alumni Dashboard
→ Pending Requests
→ View Student Profile
→ Check Capacity
→ Accept
→ Transactional Capacity Recheck
→ Create Mentorship Cycle
→ Increment Active Mentee Count
→ Notify Student
```

Mentorship Session

```
Mentorship Cycle
→ Upcoming Session
→ Generate Optional AI Agenda
→ Review/Edit Agenda
→ Conduct Session
→ Add Notes
→ Generate Optional Summary
→ Review Suggested Action Items
→ Save Confirmed Action Items
```

Cycle Completion

```
Active Cycle
→ Complete Goals/Sessions
→ Final Check-in
→ Complete Cycle
→ Release Alumni Capacity
→ Update Engagement Analytics
```

3. Screen Requirements

Authentication

- **Login / Sign Up:** Email and password inputs, clear validation, loading state during authentication, friendly error messages, and secure session handling.
- **Role Selection:** Student and Alumni role cards, short explanation of each role, continue action, and role-driven onboarding and portal navigation routing.

Student Screens

- **Student Onboarding:** Multi-step form capturing education and academic background, skills and experience, target industry and role, career goals, and a multi-stage progress indicator.
- **Student Dashboard:** Profile completion tracker, recommended mentors, pending requests, active mentorship details, upcoming sessions, and assigned action items.
- **Find Mentors:** AI-recommended alumni feed, search and dynamic filter controls, mentor profile cards, availability and remaining capacity indicators, and an understandable *"Why this match?"* explanation.
- **Mentor Profile:** Career journey overview, current role and industry, expertise domains, availability schedule, capacity status, and a primary "Request Mentorship" call-to-action.
- **Mentorship Request:** Introduction message editor, optional AI-generated draft button, editable AI output textarea, and request confirmation with status indicators.
- **Mentorship Cycle:** Cycle overview, milestone goals, scheduled sessions, action items list, progress tracking indicator, and structured check-in forms.

Alumni Screens

- **Alumni Onboarding:** Professional background inputs, core expertise domains, availability preferences, and maximum concurrent mentee capacity configuration.
- **Alumni Dashboard:** Active mentees gauge, maximum capacity ceiling, available open slots, incoming pending requests queue, and upcoming mentorship sessions.
- **Mentorship Requests:** Student profile summary, career goals, relevant skills list, Accept / Decline action buttons, and clear preview of capacity impact before acceptance.

Admin Screens

- **Admin Dashboard:** Total students, total alumni, available active mentors, active mentorship cycles, total incoming requests, match acceptance rate, cycle completion rate, and mentor capacity utilization metrics.

4. Accessibility Requirements

Requirement	UX Implementation
Screen Readers	Semantic HTML elements and meaningful, descriptive accessible labels (ARIA).
Keyboard Navigation	All interactive controls and inputs are fully accessible via keyboard tabs and focus.
Color Independence	System status conveys meaning using text and icon visual cues in addition to color alone.
Focus States	Prominent and clearly visible keyboard focus outlines across all interactive elements.
Form Labels	Every form input control possesses an explicitly associated label element.
Error Messages	Descriptive, actionable error messages placed directly near the affected input field.
Touch Targets	Adequate target size (≥ 44x44px) across all mobile touch controls.
Contrast	Strict adherence to readable contrast ratios for all text, borders, and interactive controls.
Font Scaling	Application layouts remain flexible and fully usable with enlarged system font sizes.

5. UX Principles

Principle	AlumniLink Application Standard
Simplicity	Keep discovery, requesting, and structured mentoring flows strictly focused and minimal.
Trust	Clearly explain the underlying reasons why a specific mentor was recommended.
User Control	Alumni retain complete authority over their availability and maximum concurrent mentee capacity.
Transparency	Display request statuses, remaining capacity slots, and cycle progression clearly at all times.
Feedback	Deliver immediate, clear loading, success, and error feedback states for every action.
Error Prevention	Perform atomic transactional checks on mentor capacity before permitting request acceptance.
Consistency	Reuse standardized components, design tokens, and recurring interaction patterns across all portals.
AI Assistance	AI provides suggestions and drafts; users always review, edit, and control final sent content.
Progress Visibility	Milestone goals, completed sessions, and assigned action items visually communicate cycle progression.
Low Cognitive Load	Employ progressive disclosure during onboarding and maintain clutter-free, structured dashboards.

6. Design System

- **Typography:** Clean modern sans-serif font family (Inter). Clear hierarchy separating headings, body copy, and metadata text. Readable line heights and standardized font weight pairings.
- **Spacing:** Base 4px grid unit. Consistent layout scales: 4, 8, 12, 16, 24, 32, and 48px.
- **Component Primitives:** Button, Input, Textarea, Select, Card, Badge, Modal/Dialog, Tabs, Progress Indicator, Toast, Skeleton, Empty State, and Alert.
- **Status Patterns:**
  - *Success:* Completed goals, submitted assignments, and finished action items.
  - *Warning:* Limited remaining mentor capacity and pending request states.
  - *Error:* Failed action submissions or client-side form validation issues.
  - *Neutral:* Informational notifications and inactive profile states.

7. Responsive Design

- **Desktop:** Persistent sidebar navigation, multi-column workspace dashboards, mentor discovery cards presented in structured grids, and analytics charts.
- **Tablet:** Collapsible sidebar menu and adaptive two-column layouts configured to screen real estate.
- **Mobile:** Fluid single-column layouts, mobile-friendly sheet/drawer navigation, enlarged touch targets, vertically stacked mentor and request cards, and optimized mobile dialogs.

> The application must remain fully usable and interactive on mobile devices rather than merely scaling down desktop views.

8. Loading, Empty and Error States

- **Loading States:** Skeleton view loaders for dashboards and profile cards, loading spinner feedback on button submissions, clear *"Generating…"* indicator states during AI tasks, and submission buttons disabled to block duplicate requests.
- **Empty States:**

> ***No Matches:*** *"No mentors are currently available. Try updating your goals or check back later."*

> ***No Requests:*** *"You don't have any mentorship requests yet. Explore recommended mentors to get started."*

- **Error States:** Actionable, human-readable notifications (e.g., *"We couldn't load this information right now. Please try again."*). Never expose raw stack traces, SQL errors, or technical HTTP 500 exceptions to the end user.

9. AI UX Requirements

AI-assisted features encompass introduction drafts, first-session agendas, session note summaries, suggested action items, and profile embeddings for semantic matching.

For every generative AI feature:

- Clearly label all AI-generated suggestions with distinct visual tags.
- Allow students and alumni to edit generated drafts in place.
- Provide a simple "Regenerate" option where appropriate.
- Provide full manual input fallback if AI services fail or timeout.
- Never allow AI outputs to override capacity, availability, or authorization rules.

10. Animation Requirements

All interface animations must remain subtle, lightweight, and functional without obstructing user workflows:

- Smooth page and view route transitions.
- Modal and sheet drawer opening and closing eases.
- Mentor card hover states and active keyboard focus feedback.
- Skeleton pulse animations during asynchronous loading.
- Progress bar value transitions and floating toast notifications.
- Subtle pulsing indicators for active AI generation operations.

11. UX Acceptance Criteria

- Students can complete onboarding without external assistance.
- Students can discover relevant, available alumni based on goals and trajectory.
- Students can clearly understand the reasoning behind mentor recommendations.
- Alumni can view and modify their active mentoring capacity limits at any time.
- Mentors who are unavailable or at full capacity cannot receive new mentorship requests.
- All AI-generated outputs are optional and fully editable before submission.
- Mentorship cycle progress, upcoming sessions, and action items are clearly visible.
- Graceful loading, empty, and error states exist for every major asynchronous view.
- Role-specific navigation and authorization boundaries are strictly enforced.
- The entire interface is responsive across mobile, tablet, and desktop viewports.
- All specified accessibility requirements (keyboard nav, labels, contrast) are fulfilled.

5. Technical Requirements (TRD)

Architecture Specifications, In-House Video, Local AI & Modular LMS Infrastructure

Project Name:	AlumniLink – AI-Powered Alumni Mentorship Network & Modular LMS
Team Status / Code:	G239
Team Members:	Rashi Agarwal and Pranshi Soni
Track & Duration:	Application Development • 12 Weeks (Semester 3 Scope)
Target Domain:	Higher Education, Alumni Relations & Outcome-Driven Mentorship
Technology Stack:	Next.js 14, TypeScript, FastAPI (Python), PostgreSQL 16, pgvector, Hugging Face Sentence-Transformers (all-MiniLM-L6-v2), WebRTC, Prisma ORM, NextAuth, Zod, Cloudflare R2, Nodemailer, Recharts

1. Technical Goals

- Build a decoupled, production-oriented full-stack web application leveraging Next.js 14 for UI and FastAPI for asynchronous backend workloads.
- Deploy in-house, zero-cost 1-on-1 audio/video calling using native WebRTC and WebSocket signaling without third-party subscriptions.
- Execute CPU-optimized semantic trajectory matching locally via `all-MiniLM-L6-v2` and PostgreSQL `pgvector` at zero API cost.
- Deliver a structured 12-week Learning Management System (LMS) with module authoring, assignment submissions to Cloudflare R2, and composite final test grading.
- Enforce strict concurrency-safe mentor capacity locking and automated attendance heartbeat monitoring.
- Maintain absolute type safety across TypeScript and Python via Zod, Prisma, and Pydantic v2 schemas.

2. Technology Stack

Layer	Technology	Purpose & Architectural Scope
Frontend Framework	Next.js 14 (App Router) + TypeScript	Server-rendered dashboards, client-side WebRTC video streaming hooks, and LMS classroom UI.
Backend API & Signaling	FastAPI (Python 3.11+), Uvicorn	Asynchronous REST endpoints, WebSocket signaling server for WebRTC, and in-memory ML inference.
Real-Time Video Mesh	Native WebRTC, STUN/TURN	Browser-to-browser 1-on-1 P2P media connections; Google STUN + Metered.ca TURN fallback.
Database & Vectors	PostgreSQL 16 + `pgvector`	Relational cycle/module records and 384-dimensional cosine similarity search via HNSW index.
Local Embedding Engine	Hugging Face Sentence-Transformers	`all-MiniLM-L6-v2` running locally on CPU inside FastAPI; sub-30ms embedding latency with $0 bill.
ORM & Schemas	Prisma ORM (Node) & Pydantic v2 (Py)	Declarative database modeling, atomic transactions, and automated migration management.
Binary Storage	Cloudflare R2 (S3-compatible)	Stores lecture notes, module materials, and student assignment PDFs with zero egress download fees.
Authentication	NextAuth / Auth.js (JWT)	Secure HTTP-only session cookies and role-based access control (`STUDENT` , `ALUMNI` , `ADMIN`).
Validation Layer	Zod	Runtime boundary checking for client forms, server actions, and API request schemas.
Alert Notifications	Nodemailer SMTP	Transactional emails for requests, agendas, milestone releases, and inactivity warnings.
Analytics & Metrics	Recharts	Visualizations for administrative dashboards, cohort completion rates, and skill demand.

3. Project Constraints

- **Engineering Capacity:** Two-person student engineering team (Rashi Agarwal & Pranshi Soni).
- **Development Timeline:** Strict 12-week development and delivery schedule.

- **Infrastructure Budget:** Absolute $0 cloud spend constraint; all services must operate within free-tier quotas (Vercel, Render, Neon, Cloudflare R2).
- **Video Call Policy:** Complete ban on third-party video subscriptions (Zoom, Google Meet); calls must be conducted natively in-app.
- **AI Deployment Strategy:** Zero ongoing token costs for vector generation via local CPU embeddings; external LLM calls restricted to lightweight auxiliary tasks with manual fallbacks.

4. Architecture Requirements

The system leverages a decoupled frontend/backend topology with isolated media and vector pipelines:



> **Server-Side Isolation:** Local embedding inference (`all-MiniLM-L6-v2`) runs in FastAPI memory. API tokens, database connection strings, and presigned upload signing secrets remain strictly server-side.

5. Functional Technical Requirements

- **Decoupled Authentication:** NextAuth manages encrypted JWT tokens in HTTP-only cookies; FastAPI validates incoming bearer/cookie tokens for backend access control.
- **Role-Based Authorization (RBAC):** Route-level and action-level guards restricting endpoints to `STUDENT` , `ALUMNI` , or `ADMIN` scopes.
- **Local Trajectory Embeddings:** FastAPI embeds student career blockers and mentor milestones into 384-dimensional dense vectors on CPU.
- **Hybrid Matching Query:** Single-query PostgreSQL execution combining cosine similarity (`<=>`) with hard capacity constraints (`activeMentees < maxMentees`).
- **Atomic Capacity Locking:** Concurrency-safe request acceptance handled via database interactive transactions to eliminate overbooking race conditions.
- **Native WebRTC Video Calling:** Complete SDP offer/answer handshakes and ICE candidate forwarding over FastAPI WebSockets.
- **Modular LMS Workspace:** Mentors create 12-week modules, attach markdown syllabi, and link lecture notes uploaded to Cloudflare R2.
- **Assignment Uploads & Submissions:** Secure file upload handling streaming student PDFs/archives directly to Cloudflare R2 with signed access URLs.
- **Comprehensive Final Grading:** Mentors score student Activeness, Assignments, and Final Exam performance (1-100 scale), committing verified evaluation records.
- **Attendance & Inactivity Monitor:** WebSocket connection listener logs call durations in `SessionLog` and dispatches Nodemailer alerts if scheduled sessions are missed.
- **Alumni Incentive Ledger:** Tracks completed weeks, computes prorated honorariums, and manages sponsored developer tool voucher redemptions.

6. Non-Functional Requirements (NFRs)

Metric / Boundary	Target SLA & Production Standard
Page Load Performance	Initial interactive render under 2.5 seconds on standard 4G/broadband connections.
Local Embedding Latency	Vector generation completes in under 50 ms on CPU via `all-MiniLM-L6-v2` .
Matching Pipeline SLA	HNSW cosine similarity retrieval executes in under 150 ms across 5,000+ vector records.
WebRTC Media Latency	Sub-250 ms round-trip audio/video latency over established peer connections.
Capacity Violation Rate	Zero violations; mentors can never exceed configured capacity limits under concurrent load.
Cloud Infrastructure Cost	Strictly $0.00 billed per month across all development and production services.
File Upload SLA	Direct presigned streaming to Cloudflare R2 supports files up to 25 MB with zero load on database compute.
Device Responsiveness	Complete functional parity across desktop (1440px), tablet (768px), and mobile (375px) viewports.

7. Validation & Schema Integrity

Validation operates on both ends of the wire via coordinated schema engines:

- **Frontend Layer:** Zod schemas parse form inputs (profiles, capacity boundaries, module forms) before triggering Server Actions or fetch dispatches.
- **Backend Layer:** Pydantic v2 models validate incoming JSON payloads and WebSocket frames inside FastAPI before processing.
- **Storage Layer:** Prisma schema enforces relational foreign keys, column constraints, and vector dimension sizes at the database level.

8. Security Architecture

- **Token Protection:** NextAuth session tokens reside in `HttpOnly`, `SameSite=Strict`, `Secure` cookies to prevent XSS exfiltration.
- **Backend Token Verification:** FastAPI extracts and verifies user session signatures on every authenticated request and WebSocket connection.
- **Private Bucket Security:** Cloudflare R2 buckets reject public reads; all file downloads and uploads require short-lived HMAC-signed presigned URLs.
- **Sanitized Logging:** Database exceptions and API errors return generic error IDs; raw SQL syntax, internal IPs, and stack traces are suppressed in production.

9. Resilience & AI Failure Strategy

> **Zero-Dependency Core:** Because embeddings run locally on CPU, vector matching functions without any external AI API availability. Auxiliary LLM features (drafts, agendas) degrade gracefully to manual user input.

- If external AI drafting times out, students and alumni use rich-text manual message boxes without blocking workflow execution.
- If WebSockets disconnect during video sessions, peer connections attempt ICE restarts before reverting to reconnection prompts.
- Database disconnects automatically retry with exponential backoff before surfacing friendly error banners.

10. Testing Strategy

Testing Phase	Scope & Verification Procedures
Unit Testing	Zod/Pydantic validation, capacity math, fee prorating, and vector normalization logic.
Concurrency Stress	Simultaneous simulated requests against a mentor with 1 open slot to verify transaction isolation.
WebRTC Integration	SDP offer/answer exchange, ICE candidate collection, and STUN/TURN fallback under restricted network conditions.
LMS Pipeline Tests	Module creation, Cloudflare R2 presigned file uploads, submission state transitions, and exam grade calculations.
End-to-End Pilot QA	Full user journeys across desktop and mobile devices for the pilot cohort (25 students, 10 mentors).

11. Deployment Specifications

- **Frontend Service:** Deployed to **Vercel (Hobby Tier)** with global edge caching and automated HTTPS.
- **Backend API & Signaling:** Deployed to **Render / Railway** running FastAPI via Uvicorn with native WebSocket support.
- **Database Engine:** Managed **PostgreSQL 16** on Neon Serverless with pre-compiled `pgvector`.
- **Object Store: Cloudflare R2** providing 10 GB free S3-compatible storage with zero egress bandwidth fees.
- **Relay Infrastructure:** Google Public STUN + Metered.ca TURN (50 GB free monthly relay quota).

12. Technical Acceptance Criteria

The system is accepted for pilot deployment when the following gates pass:

- All role flows (Student, Alumni, Admin) function correctly with role-gated navigation and permissions.
- In-house WebRTC 1-on-1 video connects successfully with audio, video, and screen-sharing controls.
- Local `sentence-transformers` embeddings compute on CPU in under 50 ms and populate `pgvector` columns.
- Capacity management strictly prevents mentor overbooking during concurrent request surges.
- LMS modules can be authored, assignment PDFs uploaded to Cloudflare R2, and final grades recorded.
- All production services operate stably within verified $0 free-tier cloud quotas.

6. High-Level Design (HLD)

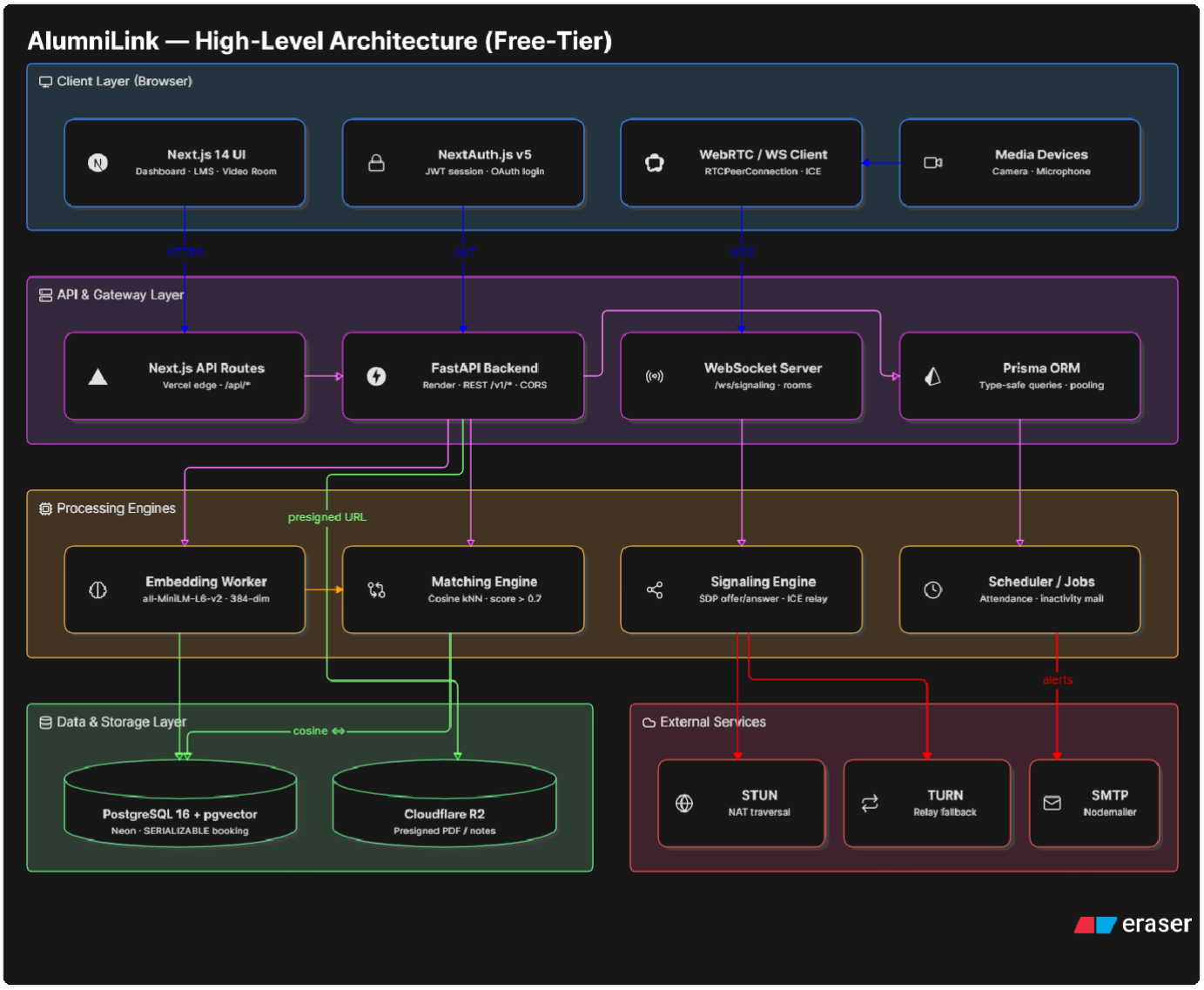
AI-Powered Alumni Mentorship Network & Learning Management System

Project Name:	AlumniLink – AI-Powered Alumni Mentorship Network
Team Status / Code:	G239
Team Members:	Rashi Agarwal and Pranshi Soni
Track & Duration:	Application Development • 12 Weeks (Semester 3 Scope)
Domain:	Alumni Relations & Career Mentorship
Technology Stack:	Next.js 14, TypeScript, FastAPI (Python), PostgreSQL 16, pgvector, Hugging Face Sentence-Transformers (all-MiniLM-L6-v2), WebRTC, Prisma ORM, NextAuth, Zod, Cloudflare R2, Nodemailer, Recharts

1. System Overview

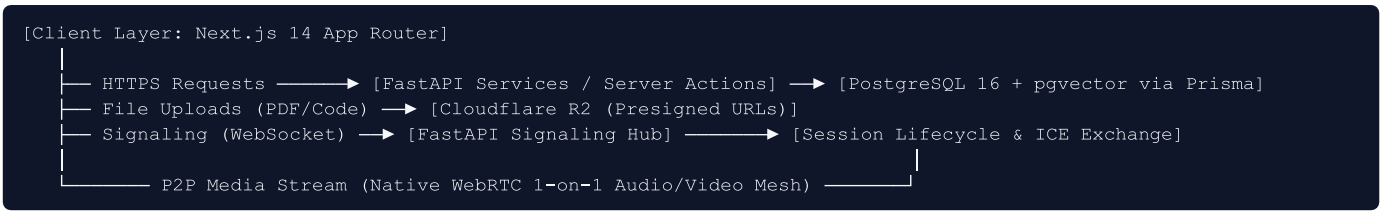
AlumniLink is an institutional, three-role full-stack application designed specifically for students, alumni mentors, and institutional administrators. The platform bridges the gap between academic preparation and industry hiring through vector-based career trajectory matching, atomic capacity management, structured 12-week mentorship tracks, native WebRTC 1-on-1 video calling, and an integrated modular Learning Management System (LMS).

2. High-Level Architecture



AlumniLink — High-Level Architecture (Free-Tier): Client Layer, API & Gateway Layer, Processing Engines, Data & Storage Layer, and External Services.

The system is organized into a modular, multi-tier topology ensuring clean boundary separation, responsive client rendering, and zero-cost cloud operations:



3. Major Architectural Components

Layer / Subsystem	Component Responsibilities	Underlying Technologies
Presentation Layer	Role-gated dashboards, student goal profiles, mentor discovery feeds, LMS classroom workspaces, live WebRTC video views, and administrative analytics.	Next.js 14 (App Router), React Server Components, Tailwind CSS, Radix UI, Recharts.
Authentication Layer	Manages user login, encrypted session cookies, account identity verification, and role-based access control (RBAC).	NextAuth / Auth.js (JWT in HTTP-only cookies) with backend token verification.
Application Layer	Coordinates incoming requests, dispatches background tasks, handles file uploads, and directs WebSocket signaling traffic.	Next.js Server Actions, Route Handlers, and FastAPI Asynchronous Routers.
Business Layer	Encapsulates core business rules: atomic capacity locking, request state machine, hybrid similarity ranking, and LMS grading workflows.	Business Services Layer, Prisma Transactions, and Pydantic Domain Models.
Data Layer	Relational record storage, foreign key enforcement, transactional commits, and vector indexing.	PostgreSQL 16, `pgvector` extension (HNSW index), Prisma ORM.
AI & Media Layer	Local embedding generation (384-dim), WebRTC peer connection establishment, and optional LLM assistants (intros, agendas, summaries).	Hugging Face `all-MiniLM-L6-v2` (FastAPI CPU), Native WebRTC, STUN/TURN, Gemini 1.5 Flash.

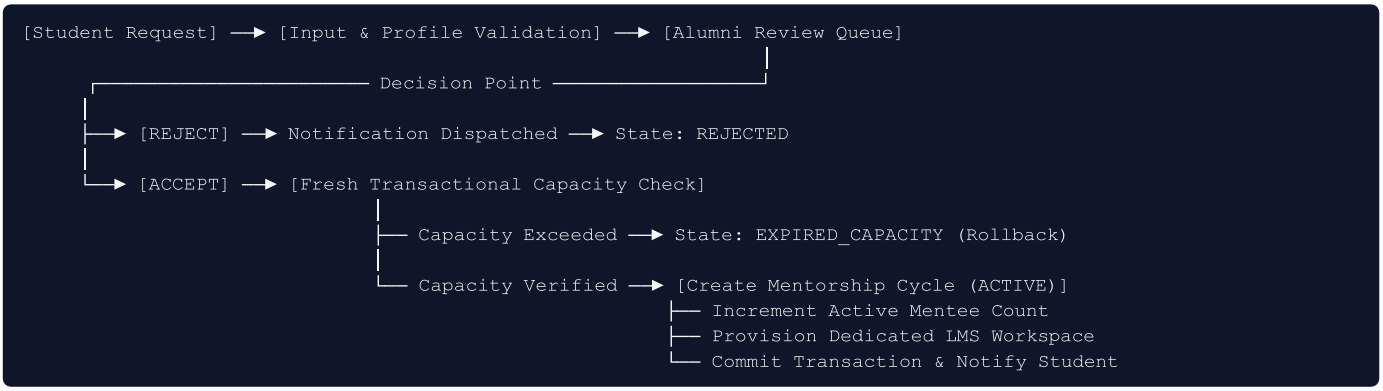
4. Semantic Matching Pipeline

The candidate matching pipeline combines dense vector similarity with deterministic capacity filters to produce actionable, relevant recommendations:

1. **Profile Update:** A student updates their profile with skills, current blockers, and target career trajectories.
2. **Structured Representation:** The system formats career goals and blockers into a structured text representation.
3. **Local Embedding Generation:** The FastAPI ML service computes a 384-dimensional vector using `all-MiniLM-L6-v2` on CPU and stores it in PostgreSQL via `pgvector`.
4. **Vector Retrieval:** A cosine distance query (`<=>`) retrieves the nearest alumni vectors based on career milestones.
5. **Deterministic Filtering:** Business logic immediately eliminates candidates who are inactive, unavailable, or have reached their capacity cap (`activeMentees >= maxMentees`).
6. **Ranked Delivery:** Top-ranked candidate profiles are delivered to the student alongside a transparent *"Why this match?"* breakdown.
7. **Student Agency:** The student reviews profiles and decides whether to dispatch a formal mentorship request.

5. Request-to-Mentorship State Machine

The lifecycle transition from discovery to an active classroom cycle follows a strict sequential pipeline:



6. Capacity Safety & Concurrency Control

To eliminate mentor burnout and overbooking race conditions, capacity is validated at two distinct checkpoints:

- **Checkpoint 1 (Discovery Phase):** Mentors whose active mentee count meets or exceeds their defined limit are filtered out of public recommendation queries.
- **Checkpoint 2 (Acceptance Phase):** When an alumnus accepts a request, capacity is re-checked inside an **atomic database transaction** (`prisma.$transaction`) with row-level locking. This guarantees that simultaneous student requests can never violate the configured limit ($N \leq 3$).

7. Deployment & Infrastructure View

- **Frontend Web Client:** Hosted on **Vercel (Free Hobby Tier)** with automated HTTPS, global CDN edge caching, and serverless route execution.
- **Backend API & WebSockets:** Deployed on **Render / Railway** running FastAPI via Uvicorn with native WebSocket signaling capabilities.
- **Database Infrastructure:** Managed **PostgreSQL 16 on Neon Serverless** pre-configured with `pgvector`.
- **Binary Storage: Cloudflare R2** providing S3-compatible file storage for syllabus notes and assignment PDFs with zero egress charges.
- **External Integrations:** Auxiliary Gemini LLM API calls and Nodemailer transactional notifications run exclusively from server environments.

8. Core Architectural Design Principles

Guiding Principles:

- **AI Does Not Make Final Decisions:** The AI layer provides candidate suggestions and draft scaffolding; humans retain 100% control over request submissions and approvals.
- **Business Rules Are Deterministic:** Capacity limits, eligibility conditions, and role permissions are strictly enforced by database rules, never probabilistic AI outputs.
- **Sensitive Operations Run Server-Side:** Authentication checks, secret keys, file signing, and transaction commits are isolated from the client bundle.
- **Core Functionality Survives AI Failures:** If generative AI services time out or fail, manual text entry and deterministic attribute matching remain fully operational.
- **Modules Remain Independently Testable:** Presentation, business logic, ML inference, and persistence layers are decoupled to allow isolated unit and integration testing.

7. Database Design

Relational Schemas, pgvector Extension, LMS Entities & Transactional Constraints

Project Name:	AlumniLink – AI-Powered Alumni Mentorship Network & Modular LMS
Team Status / Code:	G239
Team Members:	Rashi Agarwal and Pranshi Soni
Track & Duration:	Application Development • 12 Weeks (Semester 3 Scope)
Target Domain:	Higher Education, Alumni Relations & Outcome-Driven Mentorship
Technology Stack:	Next.js 14, TypeScript, FastAPI (Python), PostgreSQL 16, pgvector, Hugging Face Sentence-Transformers (all-MiniLM-L6-v2), WebRTC, Prisma ORM, NextAuth, Zod, Cloudflare R2, Nodemailer, Recharts

1. Database Choice

PostgreSQL 16 serves as the unified database engine for AlumniLink. The platform relies on strongly typed relational structures, multi-table foreign keys, and strict ACID transaction semantics to prevent overbooking race conditions. Through the native **pgvector** extension, PostgreSQL handles high-dimensional vector embeddings and similarity searches directly within SQL queries, eliminating the need to deploy and pay for a separate third-party vector database service.

2. Entity-Relationship (ER) Overview

```
User 1 ── 1 StudentProfile
User 1 ── 1 AlumniProfile

StudentProfile 1 ── * MentorshipRequest
AlumniProfile  1 ── * MentorshipRequest

StudentProfile 1 ── * MentorshipCycle
AlumniProfile  1 ── * MentorshipCycle

MentorshipCycle  1 ── * CourseModule (LMS Tracks & Syllabus Notes)
CourseModule     1 ── * Assignment (Tasks & Homework Prompts)
Assignment       1 ── * Submission (Student Files on Cloudflare R2 & Scores)

MentorshipCycle  1 ── 1 FinalEvaluation (Activeness, Tasks, Exam Scores out of 100)
MentorshipCycle  1 ── * MentorshipGoal
MentorshipCycle  1 ── * Session (WebRTC Video Room & Call Duration Logs)
Session          1 ── * ActionItem
MentorshipCycle  1 ── * CheckIn
MentorshipCycle  1 ── * Feedback

AlumniProfile    1 ── * MentorPerkGrant (Sponsored Tool Licenses)
MentorshipCycle  1 ── * MentorshipPayout (Prorated Weekly Stipends)
```

3. Core Relational Tables

Table Name	Field Definitions, Foreign Keys & Column Types
`users`	`id` (UUID/CUID, PK), `name` (VARCHAR), `email` (VARCHAR, Unique), `role` (ENUM: STUDENT, ALUMNI, ADMIN), `created_at` (TIMESTAMP), `updated_at` (TIMESTAMP).
`student_profiles`	`id` (UUID/CUID, PK), `user_id` (FK → users.id, Unique), `branch` (VARCHAR), `semester` (INT), `bio` (TEXT), `target_industry` (VARCHAR), `target_role` (VARCHAR), `career_goal_text` (TEXT), `experience_summary` (TEXT), `embedding` (VECTOR(384)), `timestamps` .
`alumni_profiles`	`id` (UUID/CUID, PK), `user_id` (FK → users.id, Unique), `current_role` (VARCHAR), `company` (VARCHAR), `industry` (VARCHAR), `years_experience` (INT), `experience_summary` (TEXT), `availability_status` (BOOLEAN), `max_concurrent_mentees` (INT, Default 2), `active_mentee_count` (INT, Default 0), `embedding` (VECTOR(384)), `timestamps` .
`skills` & `student_skills`	`skills` (`id` , `name`); `student_skills` (`student_profile_id` FK, `skill_id` FK). Many-to-many link table.
`alumni_expertise`	`id` (UUID/CUID, PK), `alumni_profile_id` (FK → alumni_profiles.id), `expertise_name` (VARCHAR).
`mentorship_requests`	`id` (UUID/CUID, PK), `student_profile_id` (FK), `alumni_profile_id` (FK), `message` (TEXT), `status` (ENUM: PENDING, ACCEPTED, REJECTED, CANCELLED), `created_at` , `updated_at` .
`mentorship_cycles`	`id` (UUID/CUID, PK), `student_profile_id` (FK), `alumni_profile_id` (FK), `goal_summary` (TEXT), `start_date` (TIMESTAMP), `end_date` (TIMESTAMP), `status` (ENUM: PENDING, ACTIVE, COMPLETED, CANCELLED), `timestamps` .
`course_modules` (LMS)	`id` (UUID/CUID, PK), `cycle_id` (FK → mentorship_cycles.id), `title` (VARCHAR), `description` (TEXT), `notes_url` (VARCHAR, Cloudflare R2 presigned link), `order_index` (INT).
`assignments` (LMS)	`id` (UUID/CUID, PK), `module_id` (FK → course_modules.id), `title` (VARCHAR), `instructions` (TEXT), `max_points` (INT, Default 100).
`submissions` (LMS)	`id` (UUID/CUID, PK), `assignment_id` (FK → assignments.id), `file_url` (VARCHAR, Cloudflare R2 link), `student_notes` (TEXT), `awarded_score` (INT), `mentor_review` (TEXT), `submitted_at` (TIMESTAMP).
`final_evaluations` (LMS)	`id` (UUID/CUID, PK), `cycle_id` (FK → mentorship_cycles.id, Unique), `activeness_score` (INT), `assignment_score` (INT), `final_exam_score` (INT), `overall_feedback` (TEXT), `evaluated_at` (TIMESTAMP).
`sessions`	`id` (UUID/CUID, PK), `cycle_id` (FK → mentorship_cycles.id), `scheduled_at` (TIMESTAMP), `duration_min` (INT), `attended` (BOOLEAN), `notes` (TEXT), `ai_summary` (TEXT), `status` (VARCHAR).
`action_items`	`id` (UUID/CUID, PK), `session_id` (FK → sessions.id), `description` (TEXT), `assigned_to_user_id` (FK), `status` (ENUM: PENDING, COMPLETED), `due_date` (TIMESTAMP).
`check_ins`	`id` (UUID/CUID, PK), `cycle_id` (FK → mentorship_cycles.id), `submitted_by_user_id` (FK), `engagement_status` (VARCHAR), `comment` (TEXT), `created_at` (TIMESTAMP).
`feedback`	`id` (UUID/CUID, PK), `cycle_id` (FK → mentorship_cycles.id), `user_id` (FK), `match_rating` (INT), `usefulness_rating` (INT), `workload_rating` (INT), `comment` (TEXT).
`mentor_perk_grants`	`id` (UUID/CUID, PK), `mentor_id` (FK → alumni_profiles.id), `tool_type` (ENUM: GITHUB_COPILOT, CHATGPT_PLUS, CLAUDE_PRO, AWS_SANDBOX), `duration_months` (INT), `voucher_code` (VARCHAR), `granted_at` .
`mentorship_payouts`	`id` (UUID/CUID, PK), `cycle_id` (FK), `mentor_id` (FK), `weeks_completed` (INT), `calculated_amount` (FLOAT), `currency` (VARCHAR), `status` (ENUM: PENDING, APPROVED, PAID), `payment_reference` (VARCHAR), `disbursed_at` .
`notifications`	`id` (UUID/CUID, PK), `user_id` (FK), `type` (VARCHAR), `title` (VARCHAR), `body` (TEXT), `read_at` (TIMESTAMP), `created_at` (TIMESTAMP).

4. Integrity Constraints & Validation Rules

- **Email Uniqueness:** `users.email` enforces a strict database-level unique index.
- **Single Role Profile:** One user record can link to at most one `student_profiles` or `alumni_profiles` record (enforced via `user_id UNIQUE`).
- **Capacity Floor & Ceiling:** `max_concurrent_mentees >= 0` . In addition, business validation constrains mentor capacity to $N \in [1, 3]$ to prevent mentor burnout.

- **Overbooking Guard:** Database check constraint ensures `active_mentee_count <= max_concurrent_mentees`.
- **Duplicate Request Prevention:** A composite unique constraint on `(student_profile_id, alumni_profile_id)` where `status = 'PENDING'` stops duplicate concurrent applications to the same mentor.
- **LMS Grade Bounds:** Scores on `final_evaluations` (activeness, assignments, exam) must be integers between `0` and `100`.

5. Indexing Strategy

```
-- Unique identifier & lookup indexes
CREATE UNIQUE INDEX idx_users_email ON users(email);
CREATE INDEX idx_requests_alumni_status ON mentorship_requests(alumni_profile_id, status);
CREATE INDEX idx_requests_student_status ON mentorship_requests(student_profile_id, status);
CREATE INDEX idx_cycles_student_status ON mentorship_cycles(student_profile_id, status);
CREATE INDEX idx_cycles_alumni_status ON mentorship_cycles(alumni_profile_id, status);

-- LMS relational foreign key lookups
CREATE INDEX idx_modules_cycle ON course_modules(cycle_id);
CREATE INDEX idx_assignments_module ON assignments(module_id);
CREATE INDEX idx_submissions_assignment ON submissions(assignment_id);

-- Vector Similarity Index (HNSW for Sub-100ms Cosine Distance Queries)
CREATE INDEX idx_alumni_trajectory_embedding ON alumni_profiles
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

6. Concurrency & Transaction Requirements

> **Atomic Request Acceptance Pipeline:** To eliminate race conditions where multiple students attempt to claim a mentor's final available slot simultaneously, acceptance must execute within an isolated interactive database transaction.

When an alumni mentor accepts an incoming request, the backend performs the following steps inside a single `SERIALIZABLE` / `FOR UPDATE` transaction block:

1. **Acquire Row Lock:** Query and lock the `alumni_profiles` record for the accepting mentor (`SELECT * FROM alumni_profiles WHERE id = $1 FOR UPDATE;`).
2. **Verify Available Capacity:** Evaluate `active_mentee_count < max_concurrent_mentees`. If capacity is exhausted, immediately abort the transaction and raise a `CAPACITY_EXCEEDED` exception.
3. **Transition Request Status:** Update the target record in `mentorship_requests` from `PENDING` to `ACCEPTED`.
4. **Instantiate Mentorship Cycle:** Insert a new record into `mentorship_cycles` with status `ACTIVE` and link it to both student and mentor profiles.
5. **Increment Mentee Counter:** Atomically increment `active_mentee_count = active_mentee_count + 1` on the mentor record.
6. **Commit Transaction:** Commit all operations atomically. If any constraint or query fails, rollback the entire transaction cleanly to maintain state integrity.

```
-- Transaction Logic Preview (SQL / Prisma $transaction)
BEGIN;
  SELECT * FROM "alumni_profiles" WHERE "id" = $alumni_id FOR UPDATE;
  -- If active_mentee_count >= max_concurrent_mentees: ROLLBACK;
  UPDATE "mentorship_requests" SET "status" = 'ACCEPTED', "updated_at" = NOW() WHERE "id" = $request_id;
  INSERT INTO "mentorship_cycles" ("id", "student_profile_id", "alumni_profile_id", "status", "created_at")
    VALUES (gen_random_uuid(), $student_id, $alumni_id, 'ACTIVE', NOW());
  UPDATE "alumni_profiles" SET "active_mentee_count" = "active_mentee_count" + 1 WHERE "id" = $alumni_id;
COMMIT;
```

8. Repository API Specification

RESTful Contracts, WebSocket Signaling Protocol, LMS Endpoints & Status Codes

Project Name:	AlumniLink – AI-Powered Alumni Mentorship Network & Modular LMS
Team Status / Code:	G239
Team Members:	Rashi Agarwal and Pranshi Soni
Track & Duration:	Application Development • 12 Weeks (Semester 3 Scope)
Target Domain:	Higher Education, Alumni Relations & Outcome-Driven Mentorship
Technology Stack:	Next.js 14, TypeScript, FastAPI (Python), PostgreSQL 16, pgvector, Hugging Face Sentence-Transformers (all-MiniLM-L6-v2), WebRTC, Prisma ORM, NextAuth, Zod, Cloudflare R2, Nodemailer, Recharts

1. API Architectural Principles

- **Standardized JSON Payloads:** All REST request bodies and responses communicate via structured UTF-8 JSON payloads.
- **Authentication by Default:** Every API endpoint requires an authenticated session token unless explicitly marked public.
- **Server-Side Authorization (RBAC):** Role-based access control (`STUDENT`, `ALUMNI`, `ADMIN`) is strictly verified server-side prior to controller execution.
- **Runtime Schema Validation:** Inputs are parsed and validated via Zod (Next.js) and Pydantic v2 (FastAPI) before database reads or mutations.
- **Deterministic Response Envelope:** Uniform success and error envelopes ensure consistent client handling across all endpoints.

2. Response Envelope Formats

Success Envelope (HTTP 200 / 201)

```
{
  "success": true,
  "data": {
    /* Response resource payload */
  }
}
```

Error Envelope (HTTP 4xx / 5xx)

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request payload or schema constraints failed",
    "details": []
  }
}
```

3. Authentication & Identity Routes

- `/api/auth/*`: Standard Auth.js / NextAuth route handlers managing credential validation, OAuth flows, and session callbacks.
- Issues encrypted JWTs stored exclusively in `HttpOnly`, `SameSite=Strict`, `Secure` cookies.

4. Profile Management APIs

Student Profile

- `GET /api/students/profile`
  Auth Scope: `STUDENT`
  Description: Returns the authenticated student's profile, academic metadata, target industry, skills, and current goal statement.
- `PATCH /api/students/profile`
  Auth Scope: `STUDENT`
  Payload: `{ "branch": string, "semester": number, "careerGoals": string, "skills": string[], "targetIndustry": string, "targetRole": string }`
  Description: Updates student attributes and automatically invokes the embedding service to update the 384-dimensional vector in PostgreSQL.

Alumni Profile

- `GET /api/alumni/profile`
  Auth Scope: `ALUMNI`
  Description: Returns the authenticated alumnus's current role, company, verified expertise tags, availability status, and mentee limits.
- `PATCH /api/alumni/profile`
  Auth Scope: `ALUMNI`
  Payload: `{ "currentRole": string, "currentCompany": string, "yearsExperience": number, "expertise": string[], "maxMentees": number, "isAvailable": boolean }`
  Description: Updates professional attributes, sets availability toggles, and configures the concurrent mentee ceiling.

5. Candidate Discovery & Matching APIs

- `GET /api/matches`
  Auth Scope: `STUDENT`
  Description: Retrieves top alumni candidates ranked by cosine distance (`<=>`) via `pgvector`. Enforces deterministic pre-filtering so only mentors with `activeMentees < maxMentees` and `isAvailable = true` are returned.
- `POST /api/matches/refresh`
  Auth Scope: `STUDENT`
  Description: Forces re-computation of the student's trajectory embedding and flushes cached candidate rankings.

6. Mentorship Request Pipeline APIs

- `POST /api/mentorship-requests`
  Auth Scope: `STUDENT`
  Payload: `{ "alumniProfileId": "uuid", "message": "string" }`
  Description: Submits an application to a mentor. Checks for existing pending requests between the pair to prevent duplicates.
- `GET /api/mentorship-requests`
  Auth Scope: `STUDENT` | `ALUMNI`
  Description: Returns all pending, accepted, and rejected requests associated with the authenticated user.
- `PATCH /api/mentorship-requests/:id`
  Auth Scope: `ALUMNI`
  Payload: `{ "action": "ACCEPT" | "REJECT" }`
  Description: Accepting executes an atomic interactive transaction: locks the mentor row, re-verifies `activeMentees < maxMentees`, transitions request to `ACCEPTED`, increments active count, and creates an `ACTIVE` cycle.

7. Structured Mentorship Cycle APIs

- `GET /api/mentorships`
  Auth Scope: `STUDENT` | `ALUMNI`
  Description: Lists all active and historical mentorship cycles for the user.
- `GET /api/mentorships/:id`
  Auth Scope: Participant or `ADMIN`
  Description: Returns complete cycle details including goals, modules, sessions, assignments, and evaluation summaries.
- `POST /api/mentorships/:id/goals`
  Auth Scope: Participant
  Payload: `{ "title": string, "description": string }`
  Description: Creates a trackable milestone goal inside the cycle workspace.
- `POST /api/mentorships/:id/sessions`
  Auth Scope: Participant
  Payload: `{ "scheduledAt": "ISO8601", "durationMin": number, "notes": string }`
  Description: Schedules an upcoming meeting or logs completed session attendance.
- `POST /api/mentorships/:id/check-ins`
  Auth Scope: Participant
  Payload: `{ "engagementStatus": string, "comment": string }`
  Description: Submits structured progress check-ins to monitor engagement and prevent inactive dropouts.

8. Modular LMS & Evaluation APIs

- `POST /api/lms/modules`
  Auth Scope: `ALUMNI`
  Payload: `{ "cycleId": string, "title": string, "description": string, "notesUrl": string, "orderIndex": number }`
  Description: Creates a course module with markdown syllabus text and links to notes stored on Cloudflare R2.

- `POST /api/lms/assignments`
  Auth Scope: `ALUMNI`
  Payload: `{ "moduleId": string, "title": string, "instructions": string, "maxPoints": number }`
  Description: Adds a homework deliverable or practical coding task to a specific module.
- `POST /api/lms/assignments/submit`
  Auth Scope: `STUDENT`
  Payload: `{ "assignmentId": string, "fileUrl": string, "studentNotes": string }`
  Description: Submits student work via Cloudflare R2 presigned object URLs.
- `PATCH /api/lms/submissions/:id/grade`
  Auth Scope: `ALUMNI`
  Payload: `{ "awardedScore": number, "mentorReview": string }`
  Description: Mentor awards points (0–100) and provides qualitative code review feedback.
- `POST /api/lms/evaluations/final`
  Auth Scope: `ALUMNI`
  Payload: `{ "cycleId": string, "activenessScore": number, "assignmentScore": number, "finalExamScore": number, "overallFeedback": string }`
  Description: Submits the final composite evaluation, transitions cycle to `COMPLETED`, and releases the mentor's active capacity slot.

9. Real-Time WebRTC Video & Signaling Protocol

Browser-to-browser media connections are negotiated over a dedicated WebSocket path: `/ws/signaling/{roomId}?userId={userId}`

Signaling Message Event	Payload Schema	Signaling Operation
`join-room`	`{ "roomId": string, "userId": string }`	Registers peer into the room registry and alerts existing peer.
`offer`	`{ "sdp": string, "type": "offer" }`	Transmits WebRTC session description offer from caller to callee.
`answer`	`{ "sdp": string, "type": "answer" }`	Returns WebRTC session description answer from callee to caller.
`ice-candidate`	`{ "candidate": object, "sdpMid": string, "sdpMLineIndex": number }`	Relays network candidates between peers for NAT and firewall traversal.
`leave-room`	`{ "roomId": string, "userId": string }`	Notifies remaining peer of disconnect; commits duration to `SessionLog`.

10. AI-Assisted Auxiliary APIs

> All generative AI calls run strictly server-side with verified session tokens. If external AI services fail, the endpoints return appropriate status codes and client forms fall back to manual entry.

- `POST /api/ai/introduction`
  Auth Scope: `STUDENT`
  Payload: `{ "alumniProfileId": string, "focusGoal": string }`
  Description: Generates a structured 3-paragraph draft introduction for student review and editing.
- `POST /api/ai/agenda`
  Auth Scope: Participant
  Payload: `{ "cycleId": string, "sessionNumber": number }`
  Description: Ingests student blockers and past notes to synthesize a 3-point pre-call discussion brief.
- `POST /api/ai/summary`
  Auth Scope: Participant
  Payload: `{ "rawNotes": string }`
  Description: Synthesizes unstructured meeting notes into bulleted takeaways and actionable to-do items.

11. Feedback & Institutional Admin APIs

- `POST /api/feedback`
  Auth Scope: Participant
  Payload: `{ "cycleId": string, "matchRating": number, "usefulnessRating": number, "workloadRating": number, "comment": string }`
  Description: Stores match relevance and mentorship quality ratings for programmatic evaluation.

- `GET /api/admin/analytics`
  Auth Scope: `ADMIN`
  Description: Returns aggregate metrics: total users, active cycles, capacity utilization, acceptance ratios, and completion rates.
- `GET /api/admin/payouts/pending`
  Auth Scope: `ADMIN`
  Description: Lists verified completed weekly milestones awaiting stipend disbursement approval.
- `PATCH /api/admin/payouts/:id/disburse`
  Auth Scope: `ADMIN`
  Payload: `{ "paymentReference": string, "disbursedAt": "ISO8601" }`
  Description: Confirms disbursement of milestone honorarium and stores banking/UTR reference number.

12. HTTP Status Codes & Error Semantics

Status Code	Semantic Definition	Standard Usage Context in AlumniLink
`200 OK`	Successful read/update	Profile retrieved, matches calculated, session updated successfully.
`201 Created`	Resource created	Request submitted, cycle instantiated, module published, goal added.
`400 Bad Request`	Invalid request	Zod/Pydantic validation failure; malformed JSON body or invalid data types.
`401 Unauthorized`	Not authenticated	Missing, expired, or invalid session cookie/bearer token.
`403 Forbidden`	Not authorized	User does not possess the required role scope (e.g. Student accessing Admin API).
`404 Not Found`	Resource not found	Requested profile ID, cycle ID, or assignment ID does not exist in database.
`409 Conflict`	Business rule conflict	Capacity exceeded (`CAPACITY_EXCEEDED`), duplicate request, or invalid cycle transition.
`429 Too Many Requests`	Rate limit exceeded	Excessive calls to AI drafting or embedding inference endpoints.
`500 Internal Server Error`	Unexpected server error	Uncaught server exception; internal error details sanitized before client delivery.
`503 Service Unavailable`	External AI service unavailable	Generative AI API network timeout or provider outage; client reverts to manual fallback.

Low-Level Design (LLD) — AlumniLink

Object-Oriented Design, Modular Code Architecture, State Lifecycles & API Logic

Project Name:	AlumniLink — AI-Powered Alumni Mentorship Network & Modular LMS
Team Status / Code:	G239
Team Members:	Rashi Agarwal and Pranshi Soni
Track & Duration:	Application Development • 12 Weeks (Semester 3 Scope)
Target Domain:	Higher Education, Alumni Relations & Outcome-Driven Mentorship
Technology Stack:	Next.js 14, TypeScript, FastAPI (Python), PostgreSQL 16, pgvector, Hugging Face Sentence-Transformers (all-MiniLM-L6-v2), WebRTC, Prisma ORM, NextAuth, Zod, Cloudflare R2, Nodemailer, Recharts

1. Module & Directory Architecture

The system uses a decoupled full-stack architecture dividing client rendering, API contracts, local ML operations, and data access layers:

```
src/
├── app/
│   ├── (auth)/
│   │   ├── login/
│   │   ├── register/
│   │   └── onboarding/
│   ├── student/
│   │   ├── dashboard/
│   │   ├── mentors/
│   │   ├── requests/
│   │   └── mentorships/
│   ├── alumni/
│   │   ├── dashboard/
│   │   ├── requests/
│   │   └── mentorships/
│   ├── admin/
│   │   ├── dashboard/
│   │   ├── users/
│   │   ├── payouts/
│   │   └── analytics/
│   └── api/
│       ├── auth/
│       ├── matches/
│       ├── mentorship-requests/
│       ├── lms/
│       └── webhooks/
│
├── components/
│   ├── ui/                # Radix UI primitives & Tailwind components
│   ├── auth/              # Login & role-guard wrappers
│   ├── student/           # Goal input, mentor directory grid
│   ├── alumni/            # Capacity toggles, candidate review
│   ├── classroom/         # Modular LMS, assignment viewer
│   ├── video/             # Native WebRTC peer stream canvas
│   ├── ai/                # Editable AI draft & agenda modals
│   └── admin/             # Cohort analytics & stipend approval cards
│
├── services/
│   ├── ProfileService.ts
│   ├── MatchingService.ts
│   ├── MentorshipRequestService.ts
│   ├── MentorshipCycleService.ts
│   ├── LMSService.ts
│   ├── SessionService.ts
│   ├── AIService.ts
│   ├── PayoutService.ts
│   └── NotificationService.ts
│
├── repositories/
│   ├── UserRepository.ts
│   ├── ProfileRepository.ts
│   ├── MatchRepository.ts
│   ├── MentorshipRepository.ts
│   ├── LMSRepository.ts
│   └── SessionRepository.ts
│
├── lib/
│   ├── auth.ts            # NextAuth configuration & callbacks
│   ├── prisma.ts          # Prisma client singleton instance
│   ├── r2.ts              # Cloudflare R2 presigned S3 client
│   └── email.ts           # Nodemailer SMTP transport setup
│
├── validators/
│   ├── profile.ts         # Zod schemas for student & alumni
│   ├── mentorship.ts      # Request & cycle schemas
│   ├── lms.ts             # Module, task & grading bounds
│   └── session.ts         # Call log & attendance validation
│
├── types/
│   ├── models.ts          # Shared TypeScript entity models
│   ├── api.ts             # Request/Response standard envelopes
│   └── webrtc.ts          # SDP & ICE candidate packet types
│
└── utils/
    ├── matching.ts        # Distance calculation & scoring
    ├── capacity.ts        # Concurrency threshold checks
    └── validation.ts      # Formatters & sanitizers
```

1.1 LLD Architecture Diagram

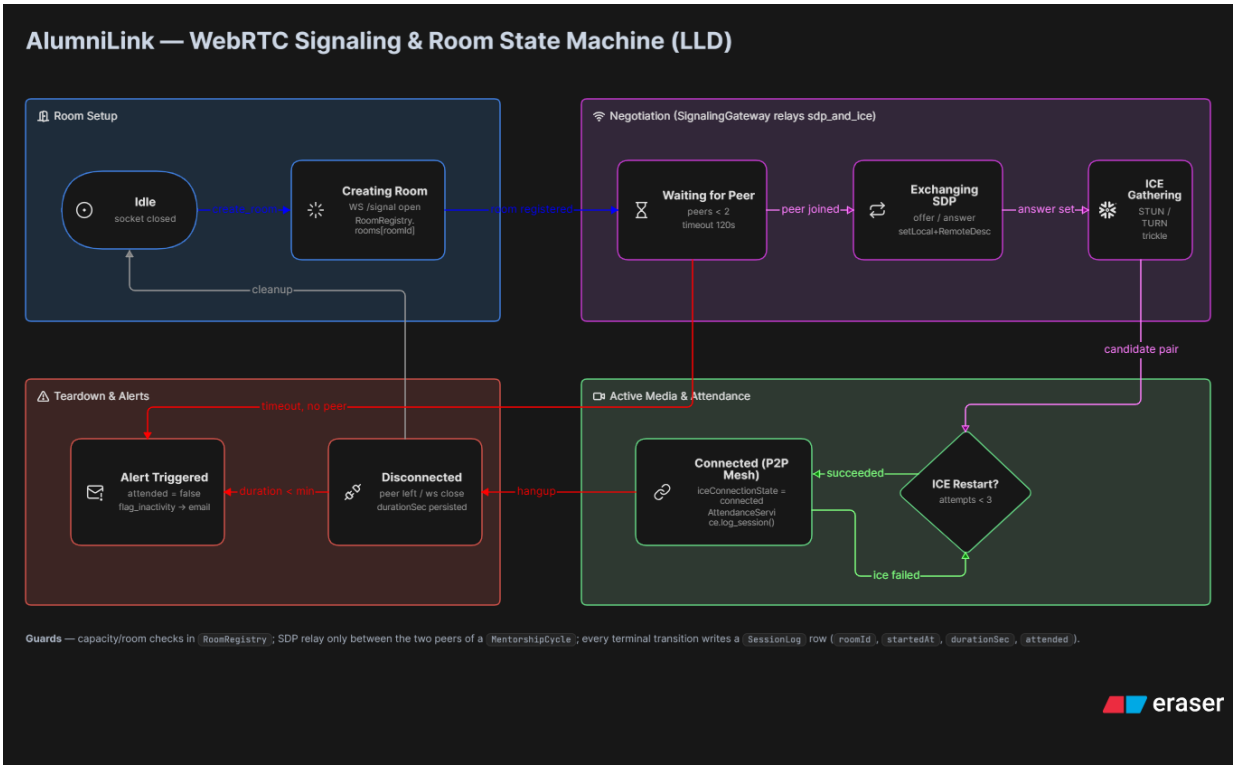


Figure 1.1 — AlumniLink WebRTC Signaling & Room State Machine (LLD)

2. Key Classes, Interfaces & Domain Contracts

2.1 Data Interfaces

```
export interface StudentProfileDTO {
  id: string;
  userId: string;
  branch: string;
  semester: number;
  careerGoals: string[];
  targetIndustries: string[];
  skills: string[];
  bio?: string;
  embedding?: number[]; // 384-dimensional dense vector
}

export interface AlumniProfileDTO {
  id: string;
  userId: string;
  currentCompany: string;
  currentRole: string;
  industry: string;
  expertise: string[];
  yearsExperience: number;
  isAvailable: boolean;
  maxConcurrentMentees: number;
  activeMentees: number;
  embedding?: number[]; // 384-dimensional dense vector
}

export interface MatchResultDTO {
  mentorId: string;
  mentorName: string;
  currentRole: string;
  company: string;
  similarityScore: number;
  availableSlots: number;
  alignmentReason: string;
}

export interface FinalEvaluationDTO {
  cycleId: string;
  activenessScore: number;  // 1-100
  assignmentScore: number;  // 1-100
  finalExamScore: number;   // 1-100
  overallFeedback: string;
}
```

2.2 Service Layer Class Definitions

```
export class MatchingService {
  /**
   * Invokes the local FastAPI ML endpoint to compute 384-dim vector on CPU
   */
  async generateEmbedding(goalText: string): Promise<number[]> {
    const response = await fetch("http://127.0.0.1:8000/api/v1/embeddings/compute", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ text: goalText })
    });
    const data = await response.json();
    return data.vector;
  }

  /**
   * Executes HNSW cosine similarity search combined with hard capacity bounds
   */
  async getTopMatches(studentId: string, limit: number = 5): Promise<MatchResultDTO[]> {
    const student = await prisma.studentProfile.findUnique({ where: { id: studentId } });
    if (!student || !student.goalEmbedding) {
      throw new Error("Student goal embedding missing");
    }

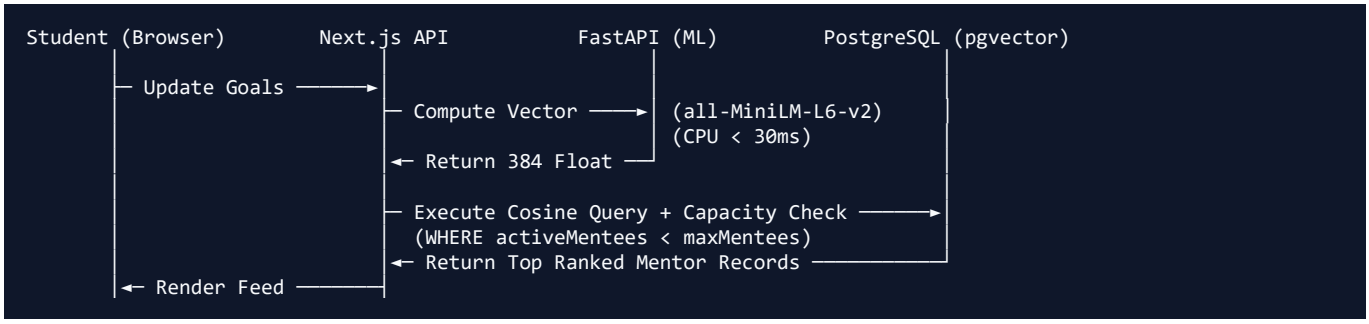
    return await prisma.$queryRaw`
      SELECT
        m.id AS "mentorId",
        u.name AS "mentorName",
        m."currentRole",
        m."currentCompany" AS "company",
        (1 - (m."trajectoryEmbedding" <=> ${student.goalEmbedding}::vector)) AS "similarityScore",
        (m."maxMentees" - m."activeMentees") AS "availableSlots",
        'Direct trajectory alignment in ' || m.industry AS "alignmentReason"
      FROM "MentorProfile" m
      JOIN "User" u ON u.id = m."userId"
      WHERE m."isAvailable" = true
        AND m."activeMentees" < m."maxMentees"
      ORDER BY m."trajectoryEmbedding" <=> ${student.goalEmbedding}::vector ASC
      LIMIT ${limit};
    `;
  }
}

export class LMSService {
  async submitAssignment(assignmentId: string, fileUrl: string, notes?: string) {
    return await prisma.submission.create({
      data: { assignmentId, fileUrl, studentNotes: notes }
    });
  }

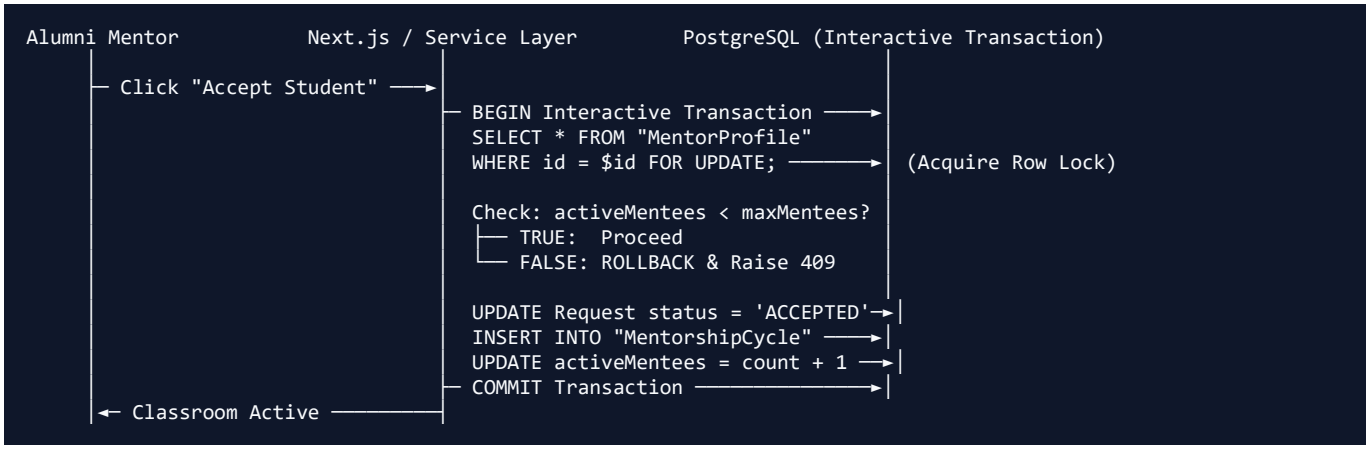
  async gradeSubmission(submissionId: string, score: number, review: string) {
    if (score < 0 || score > 100) throw new Error("Score out of bounds (0-100)");
    return await prisma.submission.update({
      where: { id: submissionId },
      data: { awardedScore: score, mentorReview: review }
    });
  }
}
```

3. End-to-End Sequence Diagrams

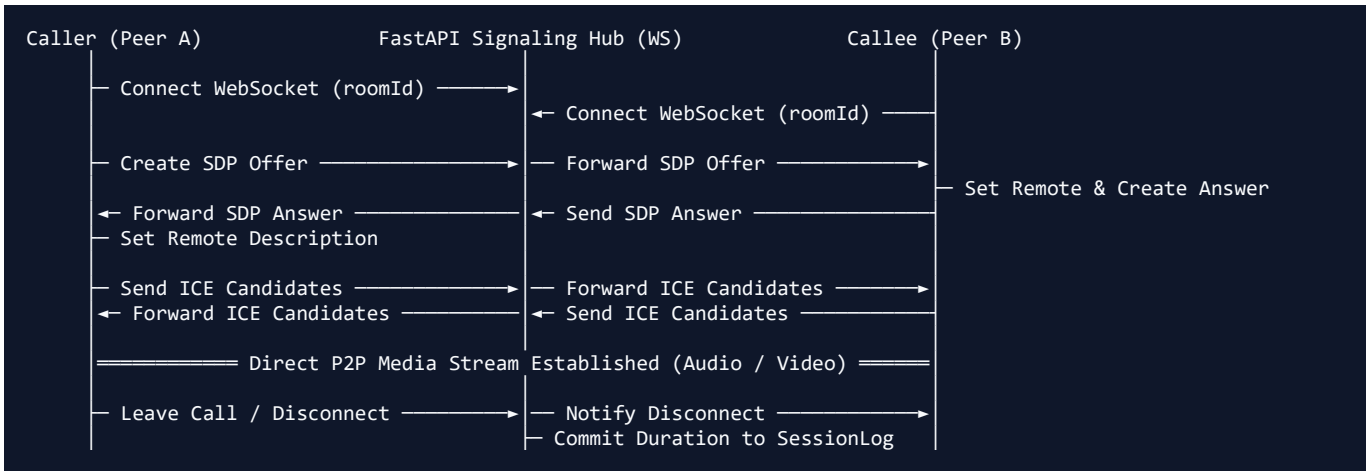
Sequence 1: Student Trajectory Match Discovery



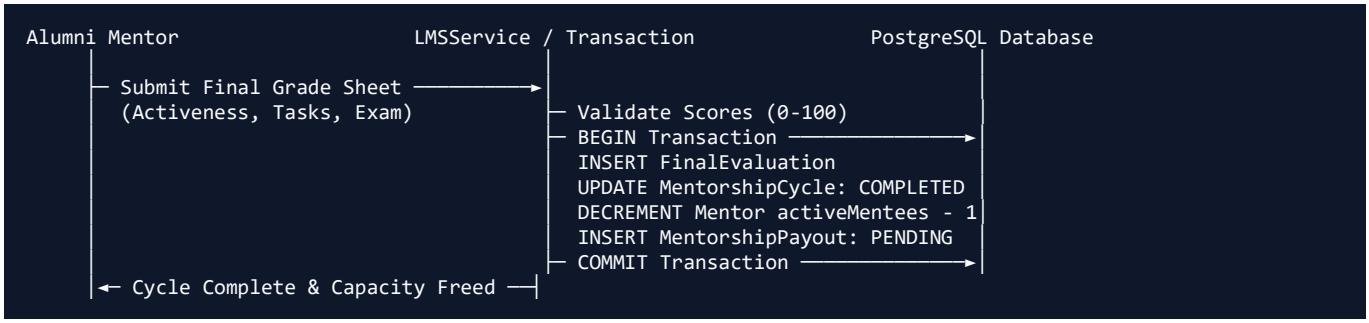
Sequence 2: Concurrency-Safe Request Acceptance & Capacity Lock



Sequence 3: Native WebRTC 1-on-1 Video Session Negotiation



Sequence 4: 12-Week LMS Final Evaluation & Mentor Capacity Release


Alumni Mentor
LMSService / Transaction
PostgreSQL Database
Submit Final Grade Sheet
(Activeness, Tasks, Exam)
Validate Scores (0-100)
BEGIN Transaction
INSERT FinalEvaluation
UPDATE MentorshipCycle: COMPLETED
DECREMENT Mentor activeMentees - 1
INSERT MentorshipPayout: PENDING
COMMIT Transaction
Cycle Complete & Capacity Freed

4. State Management Strategy

State is partitioned to avoid anti-patterns, separating transient UI interactions, form buffers, and persistent database records:

State Scope	Technology Used	Implementation Scope & Lifecycle
Authentication & RBAC	NextAuth / Auth.js	JWT token stored in HTTP-only cookie; tracks session identity, user ID, and active role scope (`STUDENT`, `ALUMNI`, `ADMIN`).
Server Data & Queries	React Server Components / SWR	Server-rendered data fetches for mentor directories, incoming request lists, and classroom syllabus modules.
Form Data Buffer	React Hook Form + Zod	Manages client inputs, validation states, character boundaries, and error display before submission.
Transient UI States	`useState`, `useReducer`	Sidebar collapse toggles, modal open/close states, active tab selections, and filter drawers.
Real-Time Video Media	Custom Hook (`useWebRTC`)	Maintains local `MediaStream`, remote `MediaStream`, peer connection state (`connecting`, `connected`, `failed`), and audio/video track mutes.
Persistent Domain State	PostgreSQL 16	Definitive state of record for user profiles, mentorship requests, course progress, grades, and payouts.

5. Error Handling & Fault Tolerance Matrix

Failure Scenario	System Layer	Resolution & Graceful Fallback Strategy
Schema Validation Failure	Zod / API Boundary	Rejects payload immediately; returns HTTP 400 with field-level error messages.
Unauthenticated Access	Auth Middleware	HTTP 401 Unauthorized; redirects browser to `/login`.
Unauthorized Scope (RBAC)	Server Action Guard	HTTP 403 Forbidden; logs security attempt and halts controller execution.
Mentor Capacity Exceeded	CapacityService	HTTP 409 Conflict (`CAPACITY_EXCEEDED`); alerts student that slots filled.
Duplicate Pending Request	MentorshipService	HTTP 409 Conflict; blocks creation of redundant applications between same pair.
WebRTC ICE Failure	Signaling / Client	Initiates ICE restart; switches connection path from STUN to Metered.ca TURN relay.
External LLM Unavailable	AIService	HTTP 503; client form falls back to manual textarea for intros and agendas.
Local ML Service Timeout	MatchingService	Falls back to deterministic attribute matching (matching by department/industry tags).
Database Transaction Abort	Prisma Repository	Automated rollback; records failure and presents safe retry alert to user.

6. Schema Validation & Business Rules

6.1 Zod Runtime Schemas (TypeScript)

```
import { z } from "zod";

// Student Onboarding Schema
export const studentProfileSchema = z.object({
  branch: z.string().min(2, "Branch is required"),
  semester: z.number().int().min(1).max(8),
  careerGoals: z.array(z.string()).min(1, "At least one career goal is required"),
  targetIndustries: z.array(z.string()).min(1, "Select at least one industry"),
  skills: z.array(z.string()).min(1, "Add at least one technical skill"),
  bio: z.string().max(1000, "Bio cannot exceed 1000 characters").optional()
});

// Alumni Capacity & Boundary Schema
export const alumniProfileSchema = z.object({
  currentRole: z.string().min(2, "Current role is required"),
  currentCompany: z.string().min(2, "Company is required"),
  yearsExperience: z.number().int().min(0),
  expertise: z.array(z.string()).min(1, "Specify at least one domain of expertise"),
  maxMentees: z.number().int().min(1).max(3, "Maximum capacity cannot exceed 3 mentees"),
  isAvailable: z.boolean().default(true)
});

// Modular LMS Final Evaluation Schema
export const finalEvaluationSchema = z.object({
  cycleId: z.string().cuid(),
  activenessScore: z.number().int().min(0).max(100, "Score must be 0-100"),
  assignmentScore: z.number().int().min(0).max(100, "Score must be 0-100"),
  finalExamScore: z.number().int().min(0).max(100, "Score must be 0-100"),
  overallFeedback: z.string().min(20, "Please provide comprehensive written feedback")
});
```

6.2 Capacity Enforcement Logic

```
export function canAcceptMentorship(activeMentees: number, maxMentees: number): boolean {
  // Hard barrier: active mentees must never equal or surpass the cap
  return activeMentees < maxMentees && maxMentees <= 3;
}
```

7. Design Patterns Applied

Design Pattern	Applied Module	Architectural Rationale
Repository Pattern	`repositories/*Repository.ts`	Decouples raw Prisma queries from business logic; facilitates unit test mocking.
Service Layer	`services/*Service.ts`	Centralizes business rules (matching, grading, capacity checks) away from API endpoints.
Strategy Pattern	`MatchingService.ts`	Allows runtime switching between vector cosine search and fallback tag matching.
Adapter Pattern	`AIService.ts`, `r2.ts`	Wraps external APIs (Gemini LLM, S3/Cloudflare R2 SDK) to isolate provider changes.
State Machine	`MentorshipCycleService.ts`	Enforces valid status transitions (`PENDING → ACTIVE → COMPLETED`).
Unit of Work / Tx	`MentorshipTransaction.ts`	Executes seat verification, cycle creation, and counter increment in atomic blocks.

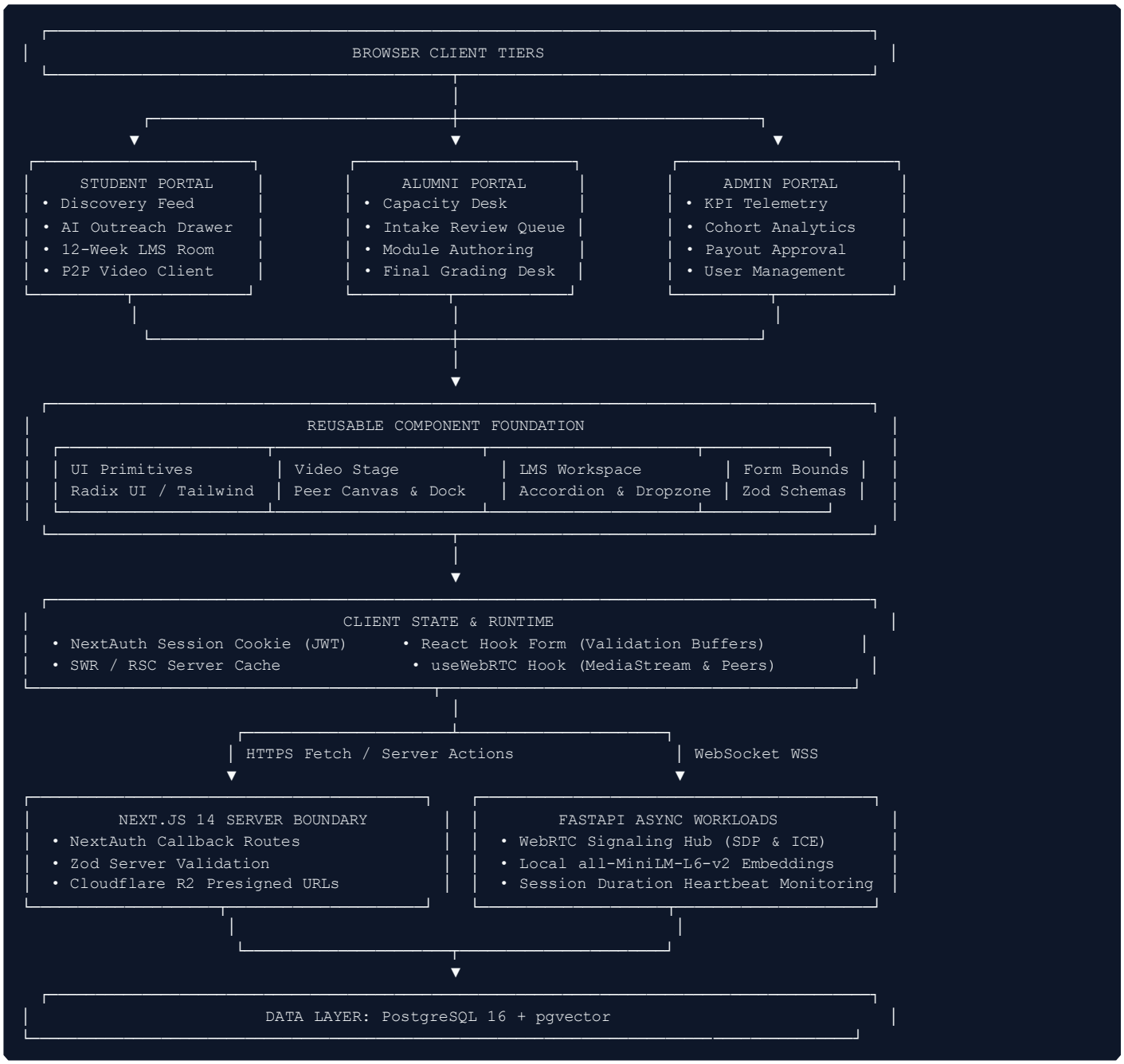
Frontend Architecture — AlumniLink

1. Frontend Architectural Philosophy

The frontend architecture of AlumniLink is engineered using **Next.js 14 (App Router)** and **TypeScript** to establish a resilient, strictly typed presentation tier. The design adopts a vertical slice pattern: React Server Components (RSC) handle data fetching and initial page delivery at the edge, while interactive client components manage browser WebRTC media streams, modular LMS authoring trees, dynamic capacity counters, and editable AI drafting drawers.

2. Complete Frontend Architecture Diagram

The structural hierarchy organizes page layouts, client state containers, data flow adapters, and backend communication boundaries:


BROWSER CLIENT TIERS
STUDENT PORTAL
• Discovery Feed
• AI Outreach Drawer
• 12-Week LMS Room
• P2P Video Client
ALUMNI PORTAL
• Capacity Desk
• Intake Review Queue
• Module Authoring
• Final Grading Desk
ADMIN PORTAL
• KPI Telemetry
• Cohort Analytics
• Payout Approval
• User Management
REUSABLE COMPONENT FOUNDATION
UI Primitives
Radix UI / Tailwind
Video Stage
Peer Canvas & Dock
LMS Workspace
Accordion & Dropzone
Form Bounds
Zod Schemas
CLIENT STATE & RUNTIME
• NextAuth Session Cookie (JWT)
• SWR / RSC Server Cache
• React Hook Form (Validation Buffers)
• useWebRTC Hook (MediaStream & Peers)
HTTPS Fetch / Server Actions
WebSocket WSS
NEXT.JS 14 SERVER BOUNDARY
• NextAuth Callback Routes
• Zod Server Validation
• Cloudflare R2 Presigned URLs
FASTAPI ASYNC WORKLOADS
• WebRTC Signaling Hub (SDP & ICE)
• Local all-MiniLM-L6-v2 Embeddings
• Session Duration Heartbeat Monitoring
DATA LAYER: PostgreSQL 16 + pgvector

3. Interactive Prototype & Live Experience Access

LIVE INTERACTIVE PROTOTYPE

AlumniLink UI System Sandbox

The complete frontend experience, responsive artboards, student discovery flows, modular LMS workspaces, and interactive components are deployed and testable via the dedicated interactive sandbox:

https://lovable.dev/preview/czurLKWEiT7VI62lYMkoK0M1Hnhx2sAG

Verification Scope: Real-time role switching (`STUDENT` , `ALUMNI` , `ADMIN`), responsive viewport layout adaptation (desktop 1440px down to mobile 375px), trajectory fit pill animations, and 12-week module accordion interaction.

4. Role-Based Page Directory Structure (`src/app/`)

The Next.js 14 App Router organizes routes strictly by authenticated functional role, protected via route middleware:

Directory Path	Role Scope	Page Capabilities & Interactive Scope
`app/(auth)/`	Public	Credential login, student/alumni registration, and multi-step profile onboarding wizards.
`app/student/mentors/`	`STUDENT`	Trajectory-ranked mentor cards, vector fit percentage indicators, and "Why this match?" alignment drawers.
`app/student/requests/`	`STUDENT`	Application dispatch modal with editable AI-assisted introduction pitch and status chips.
`app/student/mentorships/[id]/`	`STUDENT`	12-week syllabus modules, lecture notes download, homework PDF dropzone, and live video room launcher.
`app/alumni/dashboard/`	`ALUMNI`	Active mentee gauges, capacity threshold settings ($N \in [1, 3]$), and incoming application queue.
`app/alumni/mentorships/[id]/`	`ALUMNI`	Course module authoring, syllabus notes upload to Cloudflare R2, and homework grading desk.
`app/alumni/perks/`	`ALUMNI`	Milestone honorarium tracker (Weeks 4, 8, 12) and sponsored developer tool grant voucher claims.
`app/admin/dashboard/`	`ADMIN`	Institutional KPI cards, capacity utilization gauges, Recharts conversion funnels, and stipend approvals.

5. State Management & Data Synchronization

State is partitioned to avoid anti-patterns, separating transient UI interactions, form buffers, and persistent database records:

- **Authentication Session State:** Managed via **NextAuth (Auth.js)** using encrypted JWTs stored in `HttpOnly` cookies, exposing user identity and role scopes across client hooks (`useSession`).
- **Server Cache & Revalidation:** Server Components fetch relational profile and match data directly; client mutations invoke Server Actions with `revalidatePath` to refresh views.
- **Form State & Validation: React Hook Form** paired with **Zod** schemas coordinates client-side validation, error banners, and loading indicators before triggering network actions.
- **Real-Time Media & WebRTC State:** Encapsulated in a custom hook (`useWebRTC`) managing `RTCPeerConnection` instances, local/remote `MediaStream` objects, microphone/camera mutes, and WebSocket signaling status.

6. Design System Tokens & Atomic Components

Component Token	Visual Property / Hex Specification	Usage Scope
Primary Brand	Deep Slate (`#0F172A`) • Electric Indigo (`#2563EB`)	Top navigation headers, primary CTA buttons, and active tabs.
Accent & Badges	Emerald (`#10B981`) • Amber (`#F59E0B`) • Coral (`#EF4444`)	Status chips: `ACTIVE` (Green), `PENDING` (Amber), `CAPACITY FULL` (Red).
AI Assistant Badge	Violet Gradient (`#8B5CF6 → #6366F1`)	Trajectory fit score tags, editable draft sparkles, and auto-summarize buttons.
Typography	Inter / -apple-system (Scale: 12px to 28px)	Standardized typographic scale across headers, body text, and metadata tags.
Capacity Meter	Segmented Pill Gauge (`1 of 2 Slots Available`)	Visual indicator preventing mentor burnout and enforcing capacity limits.

Security Design & Threat Mitigation — AlumniLink

1. Security Architecture & Objectives

The AlumniLink security architecture enforces a zero-trust model across user authentication, inter-service API traffic, real-time media negotiation, object storage access, and database vector querying. Given that the application bridges undergraduate students with professional industry alumni, confidentiality, role segregation, and tamper-proof grading integrity are fundamental requirements.

The platform design addresses five primary security objectives:

- **Identity & Session Confidentiality:** Complete mitigation of session hijacking, token theft, and replay attacks across distributed client-server environments.
- **Authorization & Tenancy Isolation:** Enforcing strict multi-tenant row-level ownership where users can never inspect or mutate unauthorized cycles, profiles, or submissions.
- **P2P Media Privacy:** End-to-end cryptographic encapsulation of real-time audio and video communications without intermediary server decryption.
- **Deterministic Business Boundary Protection:** Preventing race-condition exploits, capacity limit bypasses, and unauthorized grade tampering via atomic database transactions.
- **Secure Local AI Guardrails:** Isolating embedding calculation and model execution within on-premise/local compute to prevent student data exfiltration to third-party providers.

2. STRIDE Threat Analysis & Mitigation Matrix

STRIDE Vector	Identified Threat Scenario	Impacted Layer	Enforced Mitigation Control
Spoofing	An attacker forges JWT session tokens or impersonates another student on WebSocket signaling channels.	NextAuth / FastAPI WebSocket Handshake	Cryptographically signed HMAC-SHA256 tokens stored in `HttpOnly` cookies; WebSockets validate session signatures before joining rooms.
Tampering	A student intercepts and modifies assignment evaluation scores or changes mentor capacity limits.	FastAPI / Prisma Mutation Layer[cite: 1]	Zod/Pydantic runtime boundary enforcement coupled with strict server-side role validation on all database writes.
Repudiation	A mentor claims they never accepted a mentee or an admin disputes approving a milestone stipend disbursement.	Relational Audit Trail (PostgreSQL)	Immutable timestamped transaction records for all cycle status changes, video durations, and payout approvals.
Information Disclosure	Direct access to private Cloudflare R2 assignment PDFs, vector similarity scores, or student contact numbers.	Cloudflare R2 & API Serialization	R2 public access completely disabled; presigned URLs generated with 15-minute TTL; response DTOs filter out PII.
Denial of Service	Adversaries spam embedding generation endpoints or exhaust server memory via concurrent video rooms.	FastAPI In-Memory ML Service	Sliding-window rate limiters (20 req/min/IP), payload size restrictions (≤ 2 KB for embedding text), and P2P video offloading.
Elevation of Privilege	A student invokes administrative payout endpoints or marks their own cycle as completed.	Server-Side Authorization Middleware	Strict RBAC guards checking `session.user.role === 'ADMIN'` before running controller logic; immediate 403 response on failure.

3. Authentication & Token Lifecycle Architecture

3.1 JWT Structure & Dual-Backend Validation

Because the platform utilizes Next.js 14 for the presentation tier and FastAPI for ML and WebSockets, session authenticity must be verified symmetrically across both runtimes without introducing database latency on every request:

```
[User Browser]
      |
      ├─ 1. Submits Credentials (HTTPS POST) ──► [NextAuth Route Handler]
      |                                                  |
      |                                                  ├─ Argon2id Hash Verification
      |                                                  └─ Issues JWE Token Signed via AUTH_SECRET
      |
      ├─ 2. Client Stores Token in HttpOnly, SameSite=Strict, Secure Cookie
      |
      ├─── REST Requests to Next.js ──► Middleware decrypts & validates session token
      |
      └─── WebSocket Handshake to FastAPI ──► FastAPI extracts cookie, verifies HMAC signature,
                                              and binds socket connection to authenticated userId.
```

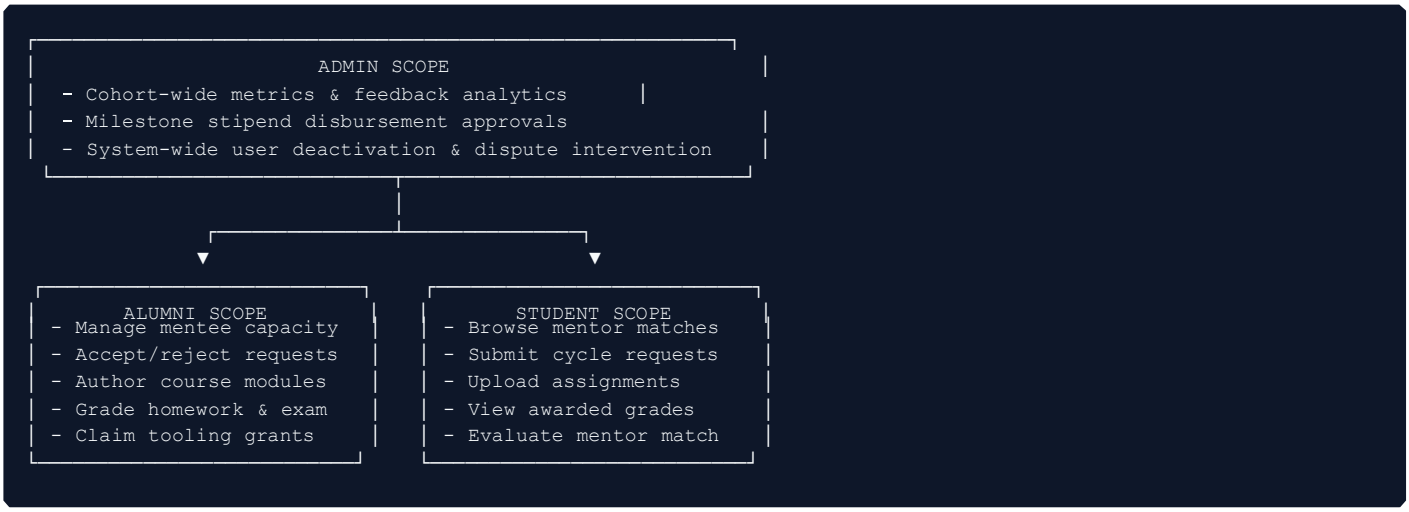
3.2 Password Hashing & Credential Defense

- **Hashing Algorithm:** Passwords undergo salted key derivation using **Argon2id** (Memory: 64 MB, Iterations: 3, Parallelism: 4) prior to database insertion.
- **Brute-Force Throttling:** Failed login attempts trigger exponential backoff delays; IP addresses recording > 5 failed attempts within 10 minutes are locked out for 30 minutes.
- **Credential Hygiene:** Plaintext passwords never touch application logs, telemetry payloads, or database rows.

4. Fine-Grained Authorization & Resource Ownership

AlumniLink operates under a multi-level authorization pipeline combining role-based gates with tenant ownership verification:

4.1 Role-Based Access Control (RBAC) Hierarchy



4.2 Multi-Tenant Data Isolation Logic

> **Ownership Verification Rule:** Every controller mutating or reading cycle data must verify that the requesting user's ID matches either `cycle.studentId` or `cycle.mentorId`. Failure to satisfy ownership halts execution immediately.

```
// Server Action Data Ownership Guard Example
export async function getCycleDeliverables(cycleId: string, session: Session) {
  const cycle = await prisma.mentorshipCycle.findUnique({
    where: { id: cycleId },
    include: { student: true, mentor: true }
  });

  if (!cycle) throw new Error("RESOURCE_NOT_FOUND"); // Returns 404

  // Enforce resource ownership: Only assigned student, mentor, or admin can access
  const isParticipant = cycle.student.userId === session.user.id ||
                        cycle.mentor.userId === session.user.id;
  const isAdmin = session.user.role === "ADMIN";

  if (!isParticipant && !isAdmin) {
    throw new Error("FORBIDDEN_RESOURCE_ACCESS"); // Returns 403
  }

  return cycle;
}
```

5. Media Encryption & Object Storage Security

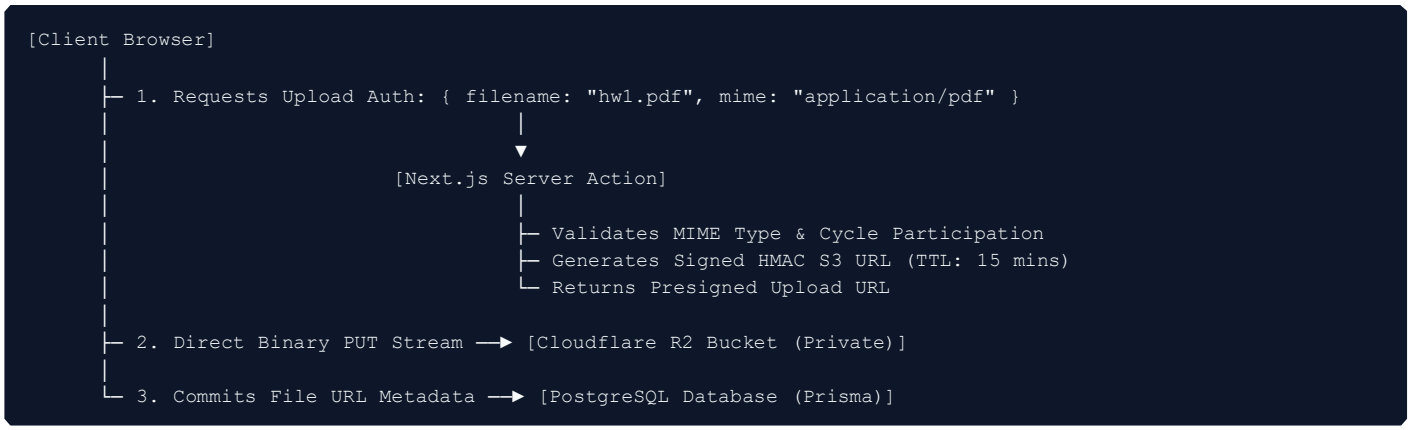
5.1 WebRTC P2P Cryptographic Protocols

To eliminate external surveillance and avoid storing video recordings on cloud infrastructure, all real-time communication operates over encrypted peer-to-peer tunnels:

- **Key Exchange:** Ephemeral key exchange occurs during peer handshake using **Datagram Transport Layer Security (DTLS 1.2 / 1.3)**. Third-party interceptors cannot decipher the cryptographic exchange.
- **Payload Encryption:** Audio and video RTP packets are encrypted at the transport layer using **Secure Real-time Transport Protocol (SRTP)** with AES-CM-128 or AES-GCM-256 ciphers.
- **Signaling Firewall Protection:** The FastAPI WebSocket signaling router enforces message-type validation. A socket registered under `student_A` cannot emit an SDP offer or ICE candidate impersonating `mentor_B`.

5.2 Cloudflare R2 Presigned Access Architecture

All binary artifacts (assignment PDF submissions, code repositories, syllabus notes) are isolated within private Cloudflare R2 object storage with zero public read capabilities:



- **Strict MIME-Type Allowlist:** Allowed extensions are restricted strictly to `application/pdf`, `application/zip`, and `text/plain`. Executable binaries (`.exe`, `.sh`, `.bat`) are rejected at the presigning boundary.
- **File Size Constraints:** Signed upload policies enforce an immutable MB. `content-length-range` capping individual submissions at 25

6. Application-Level Injection & Rate-Limiting Defenses

Vulnerability Vector	Potential Exploitation Path	Implemented Mitigation Standard
SQL & Vector Injection	Manipulated text strings injected into raw `pgvector` cosine distance queries (`<=>`)[cite: 1].	All queries utilize Prisma typed parameterized bindings (`$queryRaw`...``) preventing string interpolation.
Cross-Site Scripting (XSS)	Malicious script tags embedded within student goal statements or assignment review markdown.	Client output sanitization via DOMPurify; React automatically escapes JSX string rendering.
Cross-Site Request Forgery	Unauthorized form submissions dispatched from third-party sites using ambient browser cookies.	All session cookies enforce `SameSite=Strict`; Next.js Server Actions validate internal request headers.
Embedding Endpoint Abuse	Spamming text payloads to exhaust host server CPU capacity during vector calculations.	Sliding-window IP rate limiting (20 requests per minute); text inputs truncated to 512 characters.

7. Responsible AI Boundaries & Data Residency

> **Data Residency Assurance:** Embeddings are generated in-memory on local server CPUs using the open-source `all-MiniLM-L6-v2` model. Student academic records, career goals, and trajectory data are never transmitted to commercial AI providers for model

retraining.

- **Deterministic Primacy:** Business logic and database state constraints take absolute precedence over AI outputs. Recommendation scores generated by vector similarity cannot automatically instantiate mentorship cycles or override capacity limits ($N \in [1, 3]$)[cite: 1].
- **Human Editorial Control:** All LLM-generated introductory drafts and first-session agendas are presented inside editable input fields. Users must explicitly review and confirm all text before sending.
- **Zero Autonomous Mutations:** AI services possess read-only context and have no execution permissions or tool interfaces to update user roles, approve payouts, or modify database tables.

8. Secrets Management & Sanitization Policy

- **Environment Segregation:** Distinct environment configurations isolate `development`, `staging`, and `production` deployments. Production database strings and encryption keys are prohibited within local developer environments.
- **Git Commit Safeguards:** Pre-commit hooks execute `gitleaks` scans across staged code to block commits containing private keys, AWS/R2 secrets, or database URLs.
- **Header Hardening:** Production HTTP responses enforce security headers:

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-eval'; style-src 'self' 'unsafe-inline'
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(self), microphone=(self), geolocation=()
```

- **Log Redaction:** System logs filter out sensitive fields. Session tokens, passwords, student contact details, and bank/UPI IDs are replaced with `[REDACTED]` prior to log persistence.

9. Security Verification & Penetration Testing Suite

Target Gate	Simulated Adversarial Attack	Pass Standard & Verification Metric
Broken Object Level Auth (BOLA)	Student attempts to fetch another student's assignment feedback by guessing UUIDs in the URL.	Server verifies session ownership against cycle ID; returns `403 Forbidden`. Zero leak.
Capacity Lock Race Condition	10 simultaneous parallel HTTP POST requests sent to claim a mentor's single remaining available slot[cite: 1].	Prisma interactive transaction locks mentor row; exactly 1 request succeeds, 9 receive `409 Conflict` [cite: 1].
Privilege Escalation Probe	Manipulating client-side JWT payload from `role: STUDENT` to `role: ADMIN`.	Cryptographic signature verification fails; session immediately terminated with `401 Unauthorized`.
Malicious File Upload	Attempting to upload an executable `.sh` or `.php` script disguised with a `.pdf` extension.	FastAPI / R2 presigned service verifies magic bytes and rejected MIME type; returns `400 Bad Request`.
Signaling Room Hijacking	Connecting to WebSocket `/ws/signaling/{roomId}` for a cycle where the user is neither student nor mentor.	WebSocket server validates user ID against room membership registry; drops connection with code `4403`.
Secret Exposure Audit	Automated regex scan across Next.js public client-side JavaScript bundles and source maps.	Zero database credentials, private keys, or internal API tokens present in client builds.

Testing Strategy & Verification Gates — AlumniLink

1. Quality Assurance Goal

The core objective of the AlumniLink testing strategy is to verify that all functional workflows—ranging from vector-based trajectory discovery and concurrency-safe mentor booking to in-house WebRTC video streaming and LMS assignment grading—operate with mathematical correctness, zero privilege leakage, complete resilience under failure, and responsive usability across target campus devices.

2. Test Pyramid & Execution Frameworks

Test Level	Test Frameworks & Tools	Target Architectural Layer	Execution Cadence
Unit Tests	Jest, ts-jest, Pytest (Python 3.11)	Pure business logic, math utilities, schemas, and DTOs	Pre-commit hook & PR build
Integration Tests	Supertest, Pytest-asyncio, Testcontainers (Postgres)	Prisma transactions, FastAPI endpoints, WebSockets, R2 presign	Continuous Integration (CI)
Concurrency Stress	k6, Autocannon	Mentor row locking & capacity limit enforcement.	Nightly automated test run
WebRTC & E2E QA	Playwright, Fake Media Streams	Browser-to-browser P2P calls, complete user journeys	Pre-release staging gate

3. Unit Testing Specifications

Unit tests run in complete isolation with zero external network or database dependencies:

- **Capacity & Threshold Arithmetic:** Validates `canAcceptMentorship(activeMentees, maxMentees)` ensuring boolean rejection when active mentees equal or exceed limits, and guarantees capacity caps never exceed 3[cite: 1].
- **Prorated Honorarium Calculations:** Tests fee calculation across completed milestone weeks (Week 4: 25%, Week 8: 35%, Week 12: 40%), validating exact decimal precision with zero floating-point rounding errors.
- **Vector Cosine Normalization:** Verifies that local `all-MiniLM-L6-v2` output embeddings conform strictly to 384 dimensions and unit Euclidean length ($\|v\|_2 = 1.0 \pm 10^{-5}$).
- **Runtime Schema Validations:** Tests Zod and Pydantic schemas against edge cases: empty strings, malformed email formats, semester values out of range (< 1 or > 8), and oversized bio text (> 1000 characters).
- **LMS Evaluation Bounds:** Ensures final exam scores, activeness scores, and assignment marks reject inputs < 0 or > 100.
- **Role-Based Permission Matrix:** Verifies that authorization helpers reject invalid role strings and enforce access hierarchies (`STUDENT`, `ALUMNI`, `ADMIN`).
- **Cycle State Transitions:** Validates state transitions, ensuring cycles cannot transition directly from `PENDING` to `COMPLETED` without entering `ACTIVE`.

4. Integration Testing & Transaction Safety

Integration testing verifies that modular application components interact correctly with the database, file storage, and network layers:

4.1 Database Transaction & Capacity Locking

- **Interactive Transaction Atomicity:** Executes `prisma.$transaction` under simulated network delays, confirming that if profile creation fails, the mentor counter is rolled back cleanly without orphaned state.
- **Lock Serialization:** Tests `SELECT ... FOR UPDATE` on `alumni_profiles`, confirming that concurrent requests queue sequentially rather than executing simultaneous dirty reads.
- **pgvector Similarity Queries:** Inserts mock vector embeddings and runs cosine queries (`<=>`) with SQL `WHERE` clauses, confirming that mentors marked `isAvailable = false` or full capacity are filtered out.

4.2 WebRTC Signaling Hub & WebSocket Lifecycle

- **Peer Handshake Negotiation:** Spawns two asynchronous WebSocket clients; verifies that packets `join-room`, `offer`, `answer`, and `ice-candidate` route accurately between peers without cross-talk.
- **Disconnect & Duration Commit:** Simulates a sudden peer socket drop; confirms that the FastAPI signaling manager intercepts the disconnect event and writes accurate timestamps to `SessionLog`.

4.3 Cloudflare R2 Binary Storage Pipeline

- **Presigned Upload Authorization:** Validates that requests to `/api/lms/assignments/submit` produce signed HMAC URLs containing restricted MIME types (`application/pdf`, `application/zip`).
- **Payload Boundary Verification:** Verifies that the S3 client rejects presigned URL generation for requested file sizes exceeding 25 MB.

5. Concurrency & Stress Testing (k6 Load Script)

> **Zero-Capacity-Violation Mandate:** Simulated race condition testing where 10 concurrent requests target an alumnus with exactly 1 available slot must result in exactly 1 success and 9 rejections[cite: 1].

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  scenarios: {
    capacity_race_condition: {
      executor: 'per-vu-iterations',
      vus: 10,              // 10 concurrent student requests
      iterations: 1,
      maxDuration: '10s',
    },
  },
};

export default function () {
  const payload = JSON.stringify({
    mentorId: 'alumni_clx_test_slot_1',
    goal: 'Load testing concurrent capacity lock resilience',
  });

  const params = {
    headers: {
      'Content-Type': 'application/json',
      'Cookie': `next-auth.session-token=mock_student_token_${__VU}`,
    },
  };

  const res = http.post('http://localhost:3000/api/mentorship-requests', payload, params);

  // Validate that requests are either cleanly accepted (201) or rejected with Conflict (409)
  check(res, {
    'Status is 201 or 409': (r) => r.status === 201 || r.status === 409,
  });
}
```

6. End-to-End (E2E) & Manual QA Verification

User Journey	Step-by-Step Verification Procedure	Expected Behavioral Standard
Onboarding → Match	Student signs up, fills academic goals, triggers local embedding generation, and views discovery feed.	Top 3 trajectory-matched mentors render in < 2s with match reasons and capacity gauges.
Request → Cycle Active	Student generates optional AI intro draft, sends request; mentor accepts application.	Mentor active counter increments by 1; dedicated 12-week LMS classroom workspace unlocks.
WebRTC Video Class	Student and mentor enter video room simultaneously with camera/mic enabled.	P2P media renders in < 800ms; audio, video, and screen share controls operate cleanly.
LMS Homework & Grade	Mentor authors Week 4 module; student uploads assignment PDF; mentor awards 92/100.	PDF downloads securely via presigned URL; awarded grade reflects on student dashboard.
Admin Stipend Audit	Administrator logs in, reviews completed 12-week cycles, and approves mentor milestone stipend.	Payout status transitions from `PENDING` to `APPROVED`; audit transaction record committed.
Mobile Viewport QA	Execute core student workflows on mobile emulation (375x812 viewport).	Single-column layout renders correctly; drawer navigation and touch targets operate smoothly.

7. Critical Security & Resiliency Edge Cases

- **Overbooking Immunity:** Capacity limits cannot be exceeded under any circumstances, including concurrent requests, rapid double-clicks, or race conditions[cite: 1].
- **Tenancy Isolation:** Authenticated students cannot view or mutate submissions, notes, or profiles belonging to other pairs (HTTP 403 response enforced).
- **AI Service Outage Resiliency:** If external LLMs or vector workers time out, the UI presents manual textareas, allowing manual outreach and agenda creation without crashing the application.
- **Malformed ID & SQL Injection Defense:** Submitting non-UUID strings, SQL syntax, or oversized arrays to query parameters returns HTTP 400 without leaking stack traces or database errors.
- **Firewall Traversal (NAT/STUN/TURN):** WebRTC connections automatically fall back to Metered.ca TURN relays when campus Wi-Fi blocks direct UDP peer packets.

8. Quality Assurance Exit Criteria & Sign-Off Gate

> **Production Release Gate:** The application is approved for institutional deployment only when all exit criteria pass with zero waivers:

Exit Criteria Standard	Verification Condition & Metric	Sign-Off Gate
Core User Journeys	100% of defined student, alumni, and admin end-to-end flows pass automated Playwright runs.	**MANDATORY**
Capacity Integrity	Zero capacity overbooking violations observed during k6 concurrency stress tests ($N \le 3$)[cite: 1].	**MANDATORY**
Critical Bug Threshold	Zero open P0 (blocker) or P1 (critical security/data integrity) defects in issue backlog.	**MANDATORY**
Code Coverage Baseline	≥ 85% branch coverage on service layer, transactions, and validation schemas.	**MANDATORY**
WebRTC Media Success	≥ 98% successful P2P connection rate across simulated campus Wi-Fi networks.	**MANDATORY**
AI Graceful Fallback	Manual message entry, session notes, and agenda creation function during simulated AI outages.	**MANDATORY**
Cloud Budget Compliance	Production deployment operates completely within verified $0 free cloud resource quotas.	**MANDATORY**

CI/CD Pipeline — AlumniLink

1. Pipeline Objectives & Goals

The primary goal of the AlumniLink Continuous Integration and Continuous Deployment (CI/CD) pipeline is to automatically verify code quality, maintain full static type safety across TypeScript and Python runtimes, validate relational schema migrations, and safely deploy the decoupled frontend and backend services to zero-cost cloud hosting without service interruption.

2. Branching Strategy & Environment Isolation

The development workflow adheres to a structured Git feature-branch model designed to ensure code stability and maintain a clean audit history:

Branch Pattern	Purpose & Release Cadence	Protection Rules & Access Controls
`main`	Production-ready deployment branch. Contains only verified, peer-reviewed, and test-passing code.	Protected branch. Direct pushes prohibited; requires ≥ 1 code review approval and all CI status checks passing.
`feature/<name>`	Isolated development of specific functional capabilities (e.g., `feature/webrtc-signaling`, `feature/lms-grading`).	Branched from `main`. Frequent rebase/merge against `main` to eliminate drift. Merged exclusively via Pull Requests.
`fix/<name>`	Targeted resolution of bugs, validation edge cases, or regression defects.	Short-lived branch. Requires unit test reproduction of defect prior to merging into `main`.

3. Continuous Integration (CI) Workflow

Every commit pushed to a Pull Request or directly targeting the `main` branch triggers an automated GitHub Actions pipeline executing across isolated runner containers:

```
[Push / Pull Request Trigger]
        │
        ├──► [Job 1: Frontend Next.js 14 Quality Gate]
        │       ├── 1. Checkout repository & Setup Node.js 20 LTS
        │       ├── 2. Install dependencies via cached npm ci
        │       ├── 3. Execute ESLint check (zero warnings permitted)
        │       ├── 4. Execute TypeScript strict type checking (tsc --noEmit)
        │       ├── 5. Validate Prisma schema (prisma validate & dry-run migration)
        │       ├── 6. Run frontend unit test suites (Jest)
        │       └── 7. Compile production Next.js build (npm run build)
        │
        └──► [Job 2: Backend FastAPI & ML Quality Gate]
                ├── 1. Setup Python 3.11 environment
                ├── 2. Install dependencies (pip install -r requirements.txt)
                ├── 3. Execute code formatting & linting (Ruff / Black)
                ├── 4. Run Pytest suites (unit tests for capacity, fees, & schemas)
                └── 5. Verify local sentence-transformers model load on CPU
```

4. Continuous Deployment (CD) & Zero-Downtime Releases

Upon successful merge of an approved Pull Request into `main`, automated deployment webhooks deploy the respective infrastructure tiers:

- **Prisma Database Migration Check:** Prior to application container swaps, database migrations run via `prisma migrate deploy` against Neon PostgreSQL. If a migration encounters conflict or fails, the workflow immediately halts to prevent schema desynchronization.
- **Frontend Web App (Vercel):** Automated edge deployment builds static assets and server components. Deployment uses verified production environment variables. Vercel maintains the previous active deployment during compilation; on build failure, the existing healthy production bundle is retained without interruption.
- **Backend API & Signaling Server (Render):** Render rebuilds the FastAPI container from the updated Dockerfile, spins up the new container, verifies HTTP 200 on `/api/health`, and switches DNS traffic seamlessly before tearing down the old container.

5. GitHub Actions Configuration (`.github/workflows/ci.yml`)

The canonical workflow configuration orchestrating automated verification across both stacks:

```yaml
name: AlumniLink Continuous Integration Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  frontend-ci:
    name: Frontend Verification (Next.js 14)
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js LTS
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run ESLint
        run: npm run lint

      - name: Run TypeScript Compiler Verification
        run: npm run typecheck

      - name: Validate Prisma Schema
        run: npx prisma validate

      - name: Run Unit & Integration Tests
        run: npm test -- --coverage

      - name: Compile Production Build
        run: npm run build
        env:
          DATABASE_URL: ${{ secrets.CI_DATABASE_URL }}
          AUTH_SECRET: ${{ secrets.CI_AUTH_SECRET }}

  backend-ci:
    name: Backend Verification (FastAPI & ML)
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r backend/requirements.txt

      - name: Run Ruff Linting & Formatting Checks
        run: ruff check backend/

      - name: Run Backend Pytest Suite
        run: pytest backend/tests/ -v --maxfail=1
```

6. Environment Variable Governance & Secret Isolation

Environment configurations are partitioned into distinct operational environments with strict secret isolation:

Variable Key	Runtime Environment / Scope	Security Handling & Source
`DATABASE_URL`	Next.js, FastAPI & Prisma Engine	Encrypted SSL PostgreSQL connection string managed via platform secrets.
`AUTH_SECRET`	NextAuth & FastAPI Backend Gateway	32-byte cryptographic hex key used for symmetric JWT verification.
`CLOUDFLARE_R2_ACCESS_KEY`	Next.js Server Actions & S3 Client	Scoped IAM credentials granting PUT/GET presigned URL creation to private bucket.
`CLOUDFLARE_R2_SECRET_KEY`	Next.js Server Actions & S3 Client	Private signing secret; strictly omitted from client-side bundles.
`METERED_TURN_SECRET`	FastAPI WebRTC Signaling Router	Server secret generating dynamic time-limited credentials for WebRTC TURN relays.
`SMTP_PASSWORD`	Nodemailer Notification Engine	Transactional email SMTP credential for automated notification queues.

7. Production Release Checklist & Deployment Gate

> **Mandatory Release Sign-Off:** Before promoting code to active production use, the deployment team verifies that all release checklist items are fully checked:

Release Verification Item	Verification Condition & Acceptance Gate	Gate Status
Automated CI Pass	All GitHub Actions jobs (lint, typecheck, tests, build) complete green on `main`.	VERIFIED
Production Build Verification	Next.js production build completes without warnings or unoptimized bundle bloat.	VERIFIED
Database Migrations Reviewed	Relational changes reviewed for non-destructive schema alter operations on Neon DB.	VERIFIED
Environment Variables Active	All required secrets and keys configured in production hosting control panels.	VERIFIED
Health Check Smoke-Test	Live endpoints `/api/health` return HTTP 200 across frontend and backend services.	VERIFIED
Capacity Lock Verification[cite: 1]	Live check confirms concurrency safety and transactional limits operate on live DB[cite: 1].	VERIFIED
WebRTC Peer Connectivity	STUN/TURN negotiation verified operational across live campus Wi-Fi network test.	VERIFIED

Observability & Telemetry — AlumniLink

1. Observability Goals & Philosophy

The core objective of the AlumniLink observability architecture is to rapidly detect, isolate, and diagnose operational anomalies during production pilot deployments. By providing real-time visibility into local vector embedding inference, database transaction health, WebRTC peer connection states, and LMS assignment flows, the system ensures high reliability while operating under a strict zero-cost infrastructure budget.

2. Structured Server-Side Logging

Both Next.js 14 and FastAPI write machine-readable, structured JSON logs to standard output (`stdout`). Log levels (`INFO` , `WARN` , `ERROR`) are strictly applied to categorize system events:

Event Category	Severity	Triggering Condition	Captured Context (Non-PII)
Authentication	`WARN` / `ERROR`	Failed credential checks, expired sessions, invalid token signatures, or unauthorized RBAC routes.	`event: "AUTH_FAILURE"` , `attemptedRole` , `clientIp` , `reason` .
API & Validation	`WARN`	Zod or Pydantic validation failures on incoming form payloads.	`event: "VALIDATION_ERROR"` , `endpoint` , `failedFields[]` , `statusCode: 400` .
Capacity Conflicts	`WARN`	Race-condition rejections when requests exceed mentor availability ($N \le 3$)[cite: 1].	`event: "CAPACITY_CONFLICT"` , `mentorId` , `activeCount` , `maxLimit` [cite: 1].
Local ML / AI Failures	`ERROR`	CPU memory exhaustion or timeout during `all-MiniLM-L6-v2` embedding generation.	`event: "EMBEDDING_FAILURE"` , `model: "all-MiniLM-L6-v2"` , `durationMs` , `errorMsg` .
Database & Tx	`ERROR`	Prisma transaction aborts, deadlocks, or connection pool timeouts on Neon PostgreSQL.	`event: "DB_TX_ABORT"` , `operation` , `errorCode` , `retryCount` .
Unhandled Exceptions	`FATAL`	Uncaught runtime crashes in server components or asynchronous background tasks.	`event: "UNCAUGHT_EXCEPTION"` , `stackTraceId` , `route` , `timestamp` .

JSON Log Format Example


```
{
  "timestamp": "2026-04-12T10:14:32.184Z",
  "level": "WARN",
  "service": "alumnilink-backend-fastapi",
  "event": "CAPACITY_CONFLICT",
  "mentorId": "alumni_clx98231",
  "activeMentees": 2,
  "maxMentees": 2,
  "message": "Mentorship request rejected: mentor capacity ceiling reached",
  "traceId": "trace_9f8a2b3c4d5e"
}
```

3. Key System & Product Metrics

The system aggregates operational health signals and cohort performance metrics to inform placement administrators and developers:

3.1 Platform Health & Engagement Metrics

- **User Engagement Signals:** Counts of registered students, onboarded alumni, active cycles, and overall mentor pool utilization rate (`activeMentees / maxMentees`).
- **Application Conversion:** Mentorship request dispatch volume, acceptance velocity (time from submission to mentor decision), and acceptance vs. rejection ratios.
- **LMS Milestone Completion:** Percentage of active cycles submitting weekly assignments, module syllabus coverage, and completion rate of 12-week cycles.
- **In-House Video Attendance:** Completed WebRTC call volume, average session duration (minutes), and missed session counters logged via `SessionLog` [cite: 1].
- **AI & Local ML SLA:** Execution latency of local CPU embeddings (< 50ms target) and failure rate of auxiliary generative drafting calls.

4. WebRTC Call Telemetry & Signaling State Logs

Because video streaming is peer-to-peer, client-side diagnostic telemetry is captured to monitor connection health across restricted campus networks without recording call content:

WebRTC Metric / Event	Observed Telemetry Property	Diagnostic Purpose
ICE Connection State	`checking` → `connected` → `failed`	Detects campus firewall blocks; verifies transition from direct STUN to Metered.ca TURN.
Candidate Type	`host`, `srflx` (STUN), `relay` (TURN)	Monitors relay bandwidth usage to ensure operations remain within free monthly quotas.
Call Heartbeat / Pings	WebSocket ping/pong (every 30s)	Determines active session duration; records automated attendance in `SessionLog` [cite: 1].
Inactivity Triggers	No peer handshake 10m post-schedule	Triggers transactional reminder emails to both parties via Nodemailer[cite: 1].

5. Synthetic Health Checks & Keep-Alive Pings

> **Free-Tier Cold-Start Mitigation:** To prevent Render free web services from sleeping after 15 minutes of inactivity, an external cron monitor (Cron-Job.org / UptimeRobot) dispatches synthetic HTTP GET pings every 10 minutes.

5.1 Deep Health Check Endpoint Specification

Endpoint: `GET /api/health`

```
{
  "status": "healthy",
  "timestamp": "2026-04-12T10:15:00.000Z",
  "checks": {
    "database": {
      "status": "up",
      "latencyMs": 14
    },
    "pgvector": {
      "status": "up",
      "installed": true
    },
    "mlEngine": {
      "status": "ready",
      "model": "all-MiniLM-L6-v2",
      "device": "cpu"
    },
    "signalingWebSocket": {
      "status": "ready",
      "activeRooms": 3
    }
  }
}
```

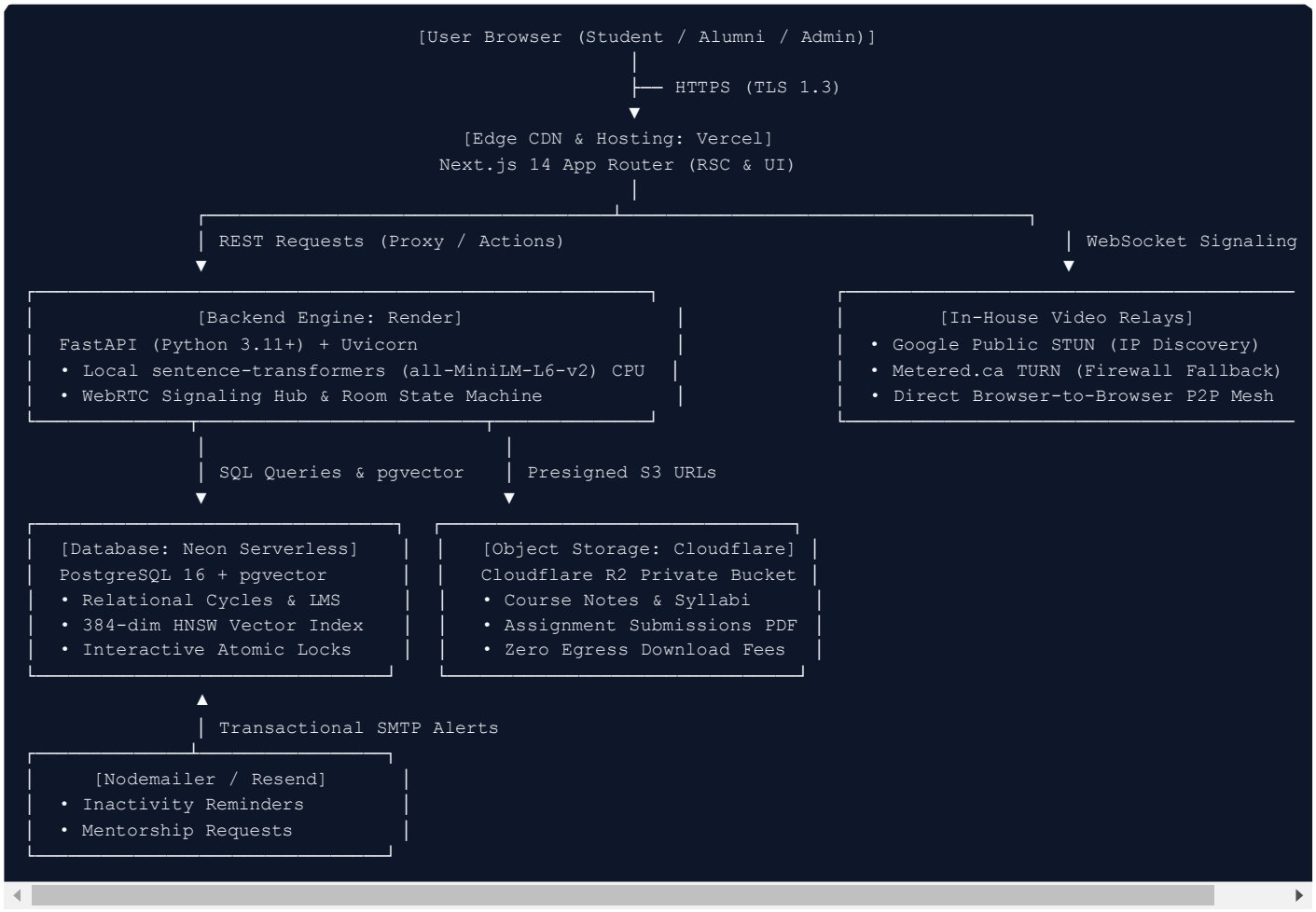

6. Data Privacy, Redaction & Log Hygiene

- **Zero Credential Logging:** Passwords, password hashes, JWT session secrets, and Cloudflare R2 private keys are strictly prohibited from log outputs.
- **PII Redaction:** Student resumes, personal contact numbers, and confidential assignment text are masked prior to emission.
- **Private Note Scrubbing:** Personal reflections or meeting notes recorded in `SessionLog` remain isolated in encrypted database storage and are never written to observability stdout streams.
- **Sanitized Tracing:** Error stack traces containing database schema layouts or internal environment variable paths are caught and sanitized before logging.

Deployment Architecture — AlumniLink

1. Deployment Architectural Topology

The AlumniLink production infrastructure uses a multi-tier, decoupled architecture designed to operate with high availability within a verified 100% free-tier cloud budget. Frontend presentation, serverless API operations, in-memory ML vectorization, real-time WebSocket signaling, relational storage, and private binary asset storage are isolated into specialized infrastructure tiers:



2. Core Infrastructure Components

System Component	Hosting Provider & Plan	Runtime Scope & Responsibilities	Allocated Free-Tier Quota
Frontend & Edge	Vercel (Hobby Free Tier)	Server-rendered React components, client dashboards, route middleware, and static asset caching.	100 GB Bandwidth, automated SSL certificates.
API & Signaling Server	Render (Free Web Service)	FastAPI backend, native WebSocket signaling hub, and in-memory CPU sentence-transformers embedding generation.	512 MB RAM, 750 compute hours/month (Zero cost).
Relational & Vector DB	Neon Serverless PostgreSQL 16	Relational user data, mentorship cycles, LMS modules, and 384-dimensional vector similarity indexing via `pgvector`.	0.5 GB storage, shared vCPU compute.
Binary Object Storage	Cloudflare R2 (S3 API)	Secure storage for lecture notes, course materials, and student assignment deliverable uploads.	10 GB standard storage, 10M Class B reads, $0 egress bandwidth.
WebRTC Traversal	Google STUN + Metered.ca TURN	NAT discovery and encrypted relay fallback for campus Wi-Fi restricted firewalls.	50 GB/month TURN relay bandwidth.
Transactional Mailer	Nodemailer via Resend / Brevo	Automated notification dispatches for request acceptances, agendas, and inactivity nudge warnings.	100–300 emails/day free quota.

3. Environment Variables Configuration

Production secrets and connection credentials are injected strictly through platform configuration dashboards and are excluded from Git version control:

Variable Name	Target Component	Description & Operational Purpose
`DATABASE_URL`	Next.js / FastAPI / Prisma	Encrypted pooled connection string to Neon PostgreSQL 16 instance.
`DIRECT_URL`	Prisma CLI	Direct unpooled connection string required for executing relational database migrations.
`AUTH_SECRET`	NextAuth & FastAPI Gateway	Cryptographic symmetric secret used to sign and verify session JWTs across both runtimes.
`NEXTAUTH_URL`	NextAuth Client	Canonical production URL (e.g., `https://alumnilink.vercel.app`) for authentication callbacks.
`CLOUDFLARE_R2_ACCOUNT_ID`	Next.js Server Actions	Cloudflare root account identifier for S3-compatible API access.
`CLOUDFLARE_R2_ACCESS_KEY`	Next.js Server Actions	Scoped IAM access token authorizing presigned URL creation.
`CLOUDFLARE_R2_SECRET_KEY`	Next.js Server Actions	Cryptographic private key for signing PUT/GET presigned requests.
`CLOUDFLARE_R2_BUCKET`	Next.js Server Actions	Target private bucket name (`alumnilink-production-assets`).
`METERED_TURN_API_KEY`	FastAPI Signaling Hub	API token to retrieve dynamic, time-limited TURN relay credentials.
`SMTP_HOST` / `SMTP_PORT`	Nodemailer Transport	Secure mail server host (e.g., `smtp.resend.com`) and port (`465` or `587`).
`SMTP_USER` / `SMTP_PASS`	Nodemailer Transport	Authenticated mailer credentials for dispatching transactional alerts.

4. Step-by-Step Deployment Process

Deployments execute via an automated GitOps delivery workflow:

1. **Code Commit & Push:** Verified code changes are merged into the `main` branch via reviewed Pull Requests on GitHub.
2. **Automated CI Verification:** GitHub Actions triggers `ci.yml` to run linting, static type verification (`tsc --noEmit`), schema validation, and Pytest/Jest test suites.
3. **Database Migration Review:** Schema migrations are applied to Neon PostgreSQL using `prisma migrate deploy`. Direct manual alterations in production are prohibited.
4. **Container & Edge Compilation:**
   - **Vercel:** Ingests the updated Next.js 14 codebase, builds static routes, optimizes bundles, and deploys to global edge nodes.
   - **Render:** Triggers a container rebuild from the repository Dockerfile, installs Python dependencies, loads the model into `all-MiniLM-L6-v2` memory, and launches Uvicorn.
5. **Post-Deployment Smoke Verification:** Automated keep-alive pings verify HTTP 200 on that `/api/health`. The deployment team confirms role routing, WebRTC signaling, and vector matching execute without errors.

> **Zero-Downtime Rollback Assurance:** If either Vercel or Render experiences a build failure or failed health check during container initialization, existing live production routing is retained, preventing downtime.

5. Horizontal Scaling & Evolution Strategy

While the MVP deployment is optimized for a single-institution pilot (25 students, 10 alumni mentors) within free cloud quotas, the architecture accommodates phased horizontal scaling:

- **Pilot Institutional Scope (Phase 1 — Current MVP):** Runs on single shared Neon compute nodes and unified Render web services. Direct P2P WebRTC offloads all media bandwidth directly to client browsers.
- **Distributed Background Workers (Phase 2):** Offload heavy transactional mail queues and bulk PDF compression into Redis-backed asynchronous worker queues (Celery / BullMQ) on separate container workers.

- **Edge Caching & Connection Pooling (Phase 3):** Introduce Prisma Accelerate and Redis caching layers for frequently queried mentor profile directories and match rankings.
- **Dedicated Media SFU Tier (Phase 4):** If expanding from 1-on-1 calls to group webinars (> 2 participants), integrate self-hosted LiveKit or Mediasoup Selective Forwarding Units (SFUs) on dedicated compute instances.

Cost Analysis & Financial Allocation — AlumniLink

1. Financial Philosophy & Primary Objectives

The core financial objective of AlumniLink is to deploy a full-scale, institutional-grade application under a **strict $0 technical infrastructure mandate** while systematically planning for scaled growth. By eliminating third-party SaaS subscriptions for video calling (Zoom, Google Meet) and replacing paid embedding APIs with CPU-optimized local inference, 100% of the project's financial budget is protected and redirected toward human incentives: **alumni milestone honorariums and sponsored developer tool subscriptions**.

2. Production Infrastructure Cost Breakdown: $0 Cloud Mandate

Every tier of the technical architecture is hosted within verified free allocations, maintaining complete production availability for the pilot cohort without incurring cloud service charges:

System Tier	Selected Provider & Plan	Allocated Free-Tier Capacity	Billed Monthly Cost
Frontend & Edge	Vercel (Hobby Free Tier)	100 GB Bandwidth, Global Edge CDN, Automated SSL	**$0.00 / mo**
Backend API & WebSockets	Render (Free Web Service Plan)	512 MB RAM, 750 Compute Hours/month (Persistent WebSockets)	**$0.00 / mo**
Database & Vector Store	Neon Serverless PostgreSQL 16	0.5 GB Storage, Shared Compute, Native `pgvector` extension	**$0.00 / mo**
Local Trajectory Embeddings	FastAPI CPU In-Memory Model	`all-MiniLM-L6-v2` running locally (Zero per-token API fees)	**$0.00 / mo**
Video Infrastructure	Native WebRTC Mesh + Google STUN	Direct peer-to-peer browser streams (Zero media server costs)	**$0.00 / mo**
TURN Firewall Traversal	Metered.ca (Free Developer Plan)	50 GB Monthly Relay Bandwidth (Restricted NAT fallback)	**$0.00 / mo**
LMS Binary Storage	Cloudflare R2 (S3-Compatible API)	10 GB Storage, 10M Read operations, **$0 Egress Download Fees**	**$0.00 / mo**
Transactional Alerts	Resend / Brevo (Free Plan)	100–300 Emails/day for approvals, agendas & inactivity alerts	**$0.00 / mo**
Custom Domain & SSL	Institutional Subdomain / Vercel DNS	Free institutional wildcard DNS or `.vercel.app` SSL routing	**$0.00 / mo**
Total Monthly Infrastructure Expense			**$0.00 / month**

3. Alumni Incentive Pool Allocation Model

Because cloud infrastructure spend is zero, institutional grants, alumni association endowments, and department placement funds are deployed directly to solve the human retention challenge:

> **The Human Incentive Formula:** Retaining high-caliber alumni mentors requires rewarding their commitment without introducing commercial overhead. Available capital is allocated into three distinct mentor support pools:

Incentive Category	Budget Allocation	Deliverable / Verification Trigger	Average Per-Mentor Value
Milestone-Based Stipend	**50%** of capital pool	Prorated disbursements at Week 4 (25%), Week 8 (35%), and Week 12 (40%) verified via session attendance and graded assignments.	₹5,000 – ₹10,000 ($60 – $120) per completed 12-week mentee cycle.
Sponsored Developer Tools	**30%** of capital pool	Granted upon course onboarding and publishing syllabus modules. 3-to-6-month license vouchers for GitHub Copilot, ChatGPT Plus, or cloud sandboxes.	~$30 – $60 bulk institutional voucher license.
Executive Swag & Networking	**20%** of capital pool	Custom engraved "Founding Mentor" physical desk plaques and funded city alumni networking mixer dinners.	₹1,500 – ₹2,500 ($18 – $30) per mentor cohort package.

4. Operational Cost Controls & Resource Safeguards

To ensure that the platform remains within free-tier quotas and maintains predictable low-cost operations during pilot scaling, the system enforces several architectural cost controls:

- **Embedding Caching in PostgreSQL:** Trajectory vectors are generated only once when a student or alumnus mutates their goal statement or skills. Subsequent searches read the stored 384-dimensional vector directly from inference. `pgvector`, eliminating redundant CPU
- **Restricted Input Context Windows:** Text payloads sent to local ML and auxiliary LLM drafting tools are strictly truncated to a maximum of 512 tokens. This minimizes CPU cycle utilization and prevents timeout latency.
- **Explicit User Invocations:** Generative AI features (introduction drafts and first-session agendas) execute only on explicit button clicks. Background automated regeneration is strictly prohibited.
- **Direct Cloudflare R2 Uploads:** Student assignment PDFs and mentor course notes upload directly from the user's browser to Cloudflare R2 via presigned URLs. Binary data bypasses application servers, preserving backend bandwidth and keeping PostgreSQL database sizes minimal.
- **Server-Side Pagination:** All directory queries, incoming application queues, and analytics feeds enforce default pagination (`limit = 10`, `offset = n`) to limit memory overhead and compute duration.
- **Keep-Alive Ping Throttling:** External synthetic monitoring pings the backend server `/api/health` endpoint every 10 minutes, maintaining readiness without exhausting Render's monthly 750-hour compute limit.

5. Financial Risk Assessment & Mitigation

Identified Financial Risk	Potential Budget Impact	Architectural Mitigation Strategy
Neon DB Storage Exhaustion	PostgreSQL database breaches the free 0.5 GB storage tier, causing write failures or forced tier upgrades.	Binary assets (PDFs, code files) are strictly routed to Cloudflare R2; PostgreSQL stores only structured relational metadata and 384-dim float vectors.
TURN Relay Bandwidth Overrun	Restricted campus Wi-Fi networks route all video traffic through Metered.ca TURN relays, exceeding the 50 GB free allocation.	WebRTC peer negotiation prioritizes direct host and STUN reflection candidates; TURN relays are engaged strictly as a secondary fallback for symmetric NATs.
Render Compute Overages	Running multiple background workers or unoptimized ML models exceeds the free 750 compute hours.	Consolidated single-service container deployment running Uvicorn and FastAPI in-memory; cron keep-alive pings throttled to 10-minute intervals.
External LLM API Spikes	Uncontrolled student requests to Gemini drafting endpoints incur recurring monthly API charges.	Sliding-window IP rate limiting (20 requests/hour per user); client forms provide manual textareas as a primary zero-cost alternative.

6. Long-Term Scaling Economics & Unit Costs

While the initial deployment targets a focused pilot cohort (25 students, 10 alumni mentors) at exactly $0.00 infrastructure cost, the platform's cost curve remains linear and predictable as adoption scales across departments:

Scale Horizon	Cohort Size	Infrastructure Tier Adjustments	Estimated Monthly Hosting Cost
Pilot MVP (Current)	25 Students 10 Mentors	100% Free Tiers (Vercel Hobby, Render Web Service, Neon Free, Cloudflare R2 Free, Metered Free).	**$0.00 / month**
Department Launch	250 Students 50 Mentors	Neon Launch Plan ($19/mo for compute & storage scale); Render Individual Plan ($7/mo for dedicated RAM).	**~$26.00 / month** (≈ $0.08 per active student)
Campus-Wide Rollout	2,500 Students 400 Mentors	Neon Scale Plan ($69/mo); Render Standard Instance ($25/mo); Cloudflare R2 Paid Egress ($5/mo); Dedicated TURN Relay ($20/mo).	**~$119.00 / month** (≈ $0.04 per active student)

7. Pilot Financial Position Statement

Core Economic Thesis: The objective of AlumniLink is predictable, zero-cost operation during institutional pilot validation. By engineering a native WebRTC video mesh and local CPU-based sentence-transformer vector matching, the platform eliminates software license dependencies, proving that high-retention, structured mentorship can be delivered sustainably at an infrastructure cost of exactly $0[cite: 1].

Project Roadmap & Execution Plan — AlumniLink

1. Roadmap Philosophy & Delivery Strategy

The AlumniLink project roadmap is structured into a rigorous 12-week development cycle tailored for a two-person engineering team. The timeline transitions methodically from foundation infrastructure and deterministic capacity management to in-house video integration, modular LMS features, local ML embeddings, and campus pilot delivery. Each sprint produces verified, deployable deliverables designed to eliminate technical debt and enforce zero-downtime releases.

2. 12-Week Sprint Breakdown

Phase 1: Foundations & Core Data Engine (Weeks 1 – 3)

- **Week 1 — Foundation & Architecture Bootstrap:** Complete system requirements analysis; initialize decoupled Next.js 14 App Router repository and FastAPI backend structure; configure managed Neon PostgreSQL 16 instance with the `pgvector` extension; establish Prisma ORM schemas; integrate NextAuth with JWT session handling and role cookies; build base UI component primitives (buttons, inputs, cards, dialogs).
- **Week 2 — Profiles, Roles & Boundary Validation:** Build student and alumni profile onboarding forms; implement Zod and Pydantic validation schemas; set up role-based access control (RBAC) middleware isolating `STUDENT`, `ALUMNI`, and `ADMIN` scopes; implement mentor availability toggles and capacity ceiling inputs ($N \le 3$).
- **Week 3 — Mentorship Requests & Atomic Capacity Locking:** Develop the mentor directory browsing interface; build the request application flow; implement interactive database transactions (`SELECT ... FOR UPDATE`) on request acceptance to guarantee zero capacity overbooking under concurrency surges; add Nodemailer SMTP dispatch for request alerts.

Phase 2: Real-Time Calling, LMS & Semantic Matching (Weeks 4 – 7)

- **Week 4 — Modular LMS Workspace & Asset Storage:** Build the mentor course authoring suite; configure Cloudflare R2 S3 bucket integration with presigned upload URLs; implement course module accordions, syllabus markdown preview, and student homework submission pipelines.
- **Week 5 — Native WebRTC 1-on-1 Video Integration:** Implement FastAPI WebSocket signaling hub (`/ws/signaling/{roomId}`); build peer-to-peer browser video view with local PIP and remote streams; integrate Google STUN and Metered.ca TURN fallback relays; add call controls (mute, camera, screen share); implement automated attendance duration logging in `SessionLog`.
- **Week 6 — Local Semantic Trajectory Matching:** Load the Hugging Face `all-MiniLM-L6-v2` model locally into FastAPI memory; compute 384-dimensional dense vectors on CPU in < 30ms; configure HNSW vector cosine indexes in PostgreSQL; execute single-query candidate retrieval combining vector similarity with deterministic capacity constraints.
- **Week 7 — AI Assistants & Fallback Workflows:** Integrate lightweight LLM endpoints for editable outreach intro drafts, 3-point pre-call agendas, and session notes summarization; implement manual text fallback modes ensuring full application operability if AI services fail or time out.

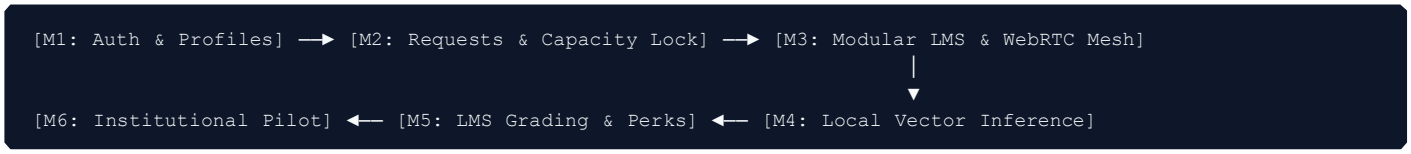
Phase 3: Quality, Incentives, Governance & Pilot Launch (Weeks 8 – 12)

- **Week 8 — LMS Grading & Capstone Evaluation:** Build mentor final evaluation sheets with score inputs (1–100) for Activeness, Homework Assignments, and Final Exam; implement automated grade aggregation and final feedback commit; release mentor capacity slot atomically upon cycle completion.
- **Week 9 — Alumni Incentives, Perks & Payout Ledger:** Build administrative honorarium approval screens; implement prorated weekly fee calculations (Weeks 4, 8, 12); create tool grant voucher redemption interfaces for GitHub Copilot and ChatGPT Plus licenses.
- **Week 10 — Administrative Analytics & Security Audits:** Build institutional analytics dashboards using Recharts (tracking acceptance rates, completion ratios, capacity utilization); execute STRIDE threat penetration testing; verify BOLA data isolation and zero secret exposure in client bundles.
- **Week 11 — End-to-End QA, Concurrency Stress & Mobile Hardening:** Run automated Playwright end-to-end user journeys; execute k6 concurrency scripts simulating parallel requests on single available slots; optimize touch targets, drawer layouts, and responsive components for mobile viewports.
- **Week 12 — Pilot Launch, Demo Video & Final Delivery:** Deploy the verified platform across Vercel, Render, Neon, and Cloudflare R2; onboard initial campus pilot cohort (25 students, 10 mentors); gather real-user feedback; produce master architecture diagrams, complete documentation suite, and recorded demonstration video.

3. Master Project Milestone Matrix

Milestone	Target Milestone Name	Core Architectural Deliverables & Verification Standard	Target Week
M1	**Core Foundation & Auth**	Dual-stack setup (Next.js + FastAPI), PostgreSQL 16 schema, JWT authentication, and validated student/alumni onboarding forms.	Week 2
M2	**Capacity & Requests**	Mentor directory, application submission, transactional row-level lock acceptance, and strict $N \le 3$ concurrency enforcement.	Week 3
M3	**Classroom & Video Mesh**	Modular 12-week LMS syllabus authoring, Cloudflare R2 assignment uploads, and browser-to-browser WebRTC 1-on-1 video rooms.	Week 5
M4	**Local Vector Matching**	FastAPI CPU inference via `all-MiniLM-L6-v2`, 384-dim HNSW vector search, and hybrid deterministic candidate filtering.	Week 6
M5	**Grading & Incentives**	LMS 3-part evaluation grading desk, cycle completion capacity release, prorated stipend calculation, and sponsored perk claims.	Week 9
M6	**QA & Pilot Delivery**	k6 concurrency pass, zero critical bugs, public HTTPS multi-tier deployment, and live institutional pilot cohort execution.	Week 12

4. Milestone Dependency Sequence Flow



5. Engineering Risk Management & Contingency Strategies

Identified Schedule Risk	Potential Milestone Impact	Implemented Engineering Contingency Plan
WebRTC ICE Traversal Blockers	Campus firewalls block direct UDP traffic, delaying Milestone M3.	Pre-configure Metered.ca TURN relay server; client automatically attempts relay connection upon candidate failure.
Render RAM Exhaustion (ML)	Sentence-transformers model breaches free 512 MB memory allocation on Render.	Use quantized `all-MiniLM-L6-v2` ONNX runtime weights (drops memory footprint to under 120 MB on CPU).
Capacity Lock Deadlocks	High concurrent application volume causes transaction aborts in PostgreSQL.	Implement deterministic query ordering with exponential backoff retries in Prisma transaction wrappers.
Cloudflare R2 Presign Latency	Slow signed URL generation delays student assignment uploads.	Cache AWS-S3 client instances; generate upload policies with 15-minute validity buffers.

> **Production Pilot Readiness Standard:** Completion of Week 12 marks the formal delivery of the verified MVP to the campus pilot cohort. Zero-cost infrastructure quotas are actively monitored to sustain predictable, ongoing operations.

Team Responsibilities — AlumniLink

1. Vertical Feature Distribution Model

Rather than splitting work horizontally by frontend (UI) versus backend (server), each team member takes **100% end-to-end full-stack ownership** of specific product features. For every assigned feature, the owner is responsible for the user interface, client components, API route handlers, database queries, and business validation rules. This prevents handoff bottlenecks and guarantees end-to-end delivery of each module.

2. Complete Feature Allocation Matrix

Feature Module	Full-Stack Owner	End-to-End Scope (UI + API + Database + Logic)
Feature 1: AI Trajectory Matching & Discovery	Rashi Agarwal	• **UI:** Mentor discovery feed, filter pills, search input, match cards, "Why this match?" drawer. • **Backend & ML:** FastAPI local embedding endpoint (`all-MiniLM-L6-v2`), sub-30ms CPU inference. • **DB:** PostgreSQL `pgvector` column, HNSW cosine distance indexing (`<=>`), deterministic capacity pre-filters.
Feature 2: Native WebRTC 1-on-1 Video Classroom	Pranshi Soni	• **UI:** P2P video stage (remote + local PIP), dock controls (mute, camera, screen-share), pre-call checklist. • **Signaling:** FastAPI WebSocket hub (`/ws/signaling/{roomId}`) for SDP offers, answers, and ICE routing. • **STUN/TURN & DB:** STUN + Metered.ca TURN fallback, call duration timer, and auto-attendance commit to `SessionLog` .
Feature 3: Capacity Locking & Request Flow	Rashi Agarwal	• **UI:** Request modal, editable AI intro draft, alumni review drawer, Accept/Decline action buttons. • **Backend & Tx:** Concurrency-safe atomic transaction (`SELECT ... FOR UPDATE`) on request acceptance. • **DB & Rules:** Mentor capacity checks ($N \le 3$), active mentee counter increment, Nodemailer email triggers.
Feature 4: 12-Week Modular LMS & Notes	Pranshi Soni	• **UI:** 12-week accordion roadmap, markdown syllabus viewer, assignment task dropzone. • **Backend:** Cloudflare R2 presigned upload/download URL generator, MIME validation (PDF/ZIP ≤ 25 MB). • **DB & Schemas:** `course_modules` , `assignments` , and `submissions` tables, relational foreign key links.
Feature 5: Final LMS Grading & Capstone Evaluation	Rashi Agarwal	• **UI:** Mentor final grade desk, 3 score inputs (0-100), student grade report card view. • **Backend:** Score aggregation logic, input bounds validation (Zod + Pydantic), grade submission handler. • **DB & Lifecycle:** Commits to `final_evaluations` , marks cycle `COMPLETED` , and atomically releases mentor capacity slot.
Feature 6: Alumni Retention, Perks & Payouts	Pranshi Soni	• **UI:** Mentor perks dashboard, tool voucher claim cards (GitHub Copilot, ChatGPT Plus), stipend ledger. • **Backend:** Prorated stipend calculation (Weeks 4, 8, 12), admin approval endpoints. • **DB:** `mentor_perk_grants` , `mentorship_payouts` tables, UTR payment audit records.
Feature 7: Authentication, RBAC & Onboarding	Rashi Agarwal	• **UI:** Login, Register, Role Selector (Student vs. Alumni), multi-step profile onboarding wizard. • **Backend:** NextAuth / Auth.js session handling, Argon2id password hashing, server-side RBAC guards. • **DB:** `users` , `student_profiles` , `alumni_profiles` schemas and migrations.
Feature 8: Admin Analytics & Institutional Dashboard	Pranshi Soni	• **UI:** Administrative control center with Recharts metric cards, conversion funnel charts. • **Backend:** SQL aggregation queries for capacity utilization, acceptance rates, and cycle completion stats. • **DB:** Optimized view queries across active requests, cycles, and student cohort numbers.
Shared Platform Infrastructure	Joint Ownership	• **Deployment & Hosting:** Vercel (Next.js), Render (FastAPI), Neon (PostgreSQL 16), Cloudflare R2. • **CI/CD & Testing:** GitHub Actions pipeline, k6 concurrency stress tests, Playwright E2E pilot tests.

3. Detailed Breakdown by Team Member

3.1 Rashi Agarwal — Primary Feature Suite

Rashi is responsible for building the core matching engine, capacity locking, authentication, and grading evaluation from the user interface down to the database:

- **Feature 1 — AI Trajectory Matching & Discovery:**
  - *Frontend:* Builds the complete student discovery feed, search bar, mentor card grid, trajectory similarity badges ("94% Match"), and "Why this match?" drawers.
  - *Backend & DB:* Integrates the Hugging Face `all-MiniLM-L6-v2` model locally in FastAPI on CPU, calculates 384-dim vectors in < 30ms, and executes PostgreSQL `pgvector` cosine distance queries (`<=>`) combined with availability checks.
- **Feature 3 — Concurrency-Safe Mentorship Request Flow:**
  - *Frontend:* Builds the request application modal, editable AI intro draft textarea, and the alumni pending requests intake drawer.
  - *Backend & DB:* Writes the atomic database transaction (`SELECT ... FOR UPDATE`) executed on request acceptance, enforcing that mentors never exceed their max capacity ($N \le 3$). Configures Nodemailer to trigger transactional confirmation emails.
- **Feature 5 — Final Test Grading & Capstone Evaluation:**
  - *Frontend:* Builds the mentor evaluation grading desk with score sliders (0–100) and the student report card view.
  - *Backend & DB:* Validates scores via Zod/Pydantic, aggregates total weighted performance, stores records in `final_evaluations` , marks the cycle `COMPLETED` , and decrements `activeMentees` to release the mentor's slot.
- **Feature 7 — Authentication, RBAC & Onboarding:**
  - *Frontend:* Builds the sign-up, sign-in, and onboarding forms with validation errors and loading states.
  - *Backend & DB:* Implements NextAuth with JWT session cookies, password hashing with Argon2id, and server-side RBAC middleware for `STUDENT` , `ALUMNI` , and `ADMIN` scopes. Defines core `User` and `Profile` Prisma models.

3.2 Pranshi Soni — Primary Feature Suite

Pranshi is responsible for the in-house WebRTC video room, modular LMS classroom, alumni retention perks, and institutional administrative analytics from the user interface down to the database:

- **Feature 2 — Native WebRTC 1-on-1 Video Classroom:**
  - *Frontend:* Builds the in-app video room layout, local/remote stream rendering, floating control bar (mute, camera, screen-share), and pre-call agenda checklist panel.
  - *Backend & Network:* Constructs the FastAPI WebSocket signaling hub (`/ws/signaling/{roomId}`) for SDP offers, answers, and ICE candidate exchanges. Configures Google STUN and Metered.ca TURN relays for NAT firewall traversal.
  - *Database:* Automatically logs session attendance and call duration in the `SessionLog` table.
- **Feature 4 — 12-Week Modular LMS & Syllabus Notes:**
  - *Frontend:* Builds the course module accordion view, markdown reading container, and drag-and-drop assignment dropzone.
  - *Backend & Storage:* Sets up the Cloudflare R2 S3 bucket client, generates 15-minute HMAC presigned upload/download URLs, and enforces MIME-type restrictions (PDF/ZIP ≤ 25 MB).
  - *Database:* Models and manages the `course_modules` , `assignments` , and `submissions` tables.
- **Feature 6 — Alumni Retention, Perks & Payout Ledger:**
  - *Frontend:* Builds the mentor rewards dashboard, sponsored tool voucher cards (GitHub Copilot, ChatGPT Plus), and payment audit table.
  - *Backend & DB:* Writes the stipend calculation engine prorating amounts across Weeks 4, 8, and 12, models the table, and builds `mentorship_payouts` admin disbursement endpoints.
- **Feature 8 — Admin Analytics & Institutional Dashboard:**
  - *Frontend:* Builds the administrator control center featuring Recharts visualizations for cohort progress and capacity health.
  - *Backend & DB:* Writes aggregate SQL queries computing acceptance rates, completion ratios, active student counts, and mentor capacity utilization metrics.

4. Feature Integration & Testing Handshake

Cross-Feature Integration Points:

- **Matching → Requests (Rashi):** Rashi handles both candidate matching and the request pipeline, ensuring a seamless flow from mentor discovery to transactional cycle creation.
- **Classroom → Video (Pranshi):** Pranshi connects the modular LMS workspace with the WebRTC video classroom, allowing mentors and students to launch live sessions directly from their active week module.
- **Cycle Lifecycle → Evaluation Handshake (Joint):** When Pranshi's LMS modules reach Week 12, Rashi's grading engine activates to record final scores, which then triggers Pranshi's payout ledger for stipend approval.
- **Joint Release Verification:** Both members run the k6 concurrency stress tests, conduct end-to-end Playwright user journeys, and deploy the application to Vercel, Render, Neon, and Cloudflare R2 for the campus pilot.

GitHub Repository Structure & Codebase Governance — AlumniLink

1. Repository Layout Philosophy

The AlumniLink repository organizes the decoupled Next.js 14 presentation tier and the FastAPI (Python) backend into a unified, clean monorepo structure. By establishing clear architectural boundaries, frontend UI components, business services, database definitions, local ML inference scripts, WebRTC signaling hubs, and print-ready documentation remain modular, independently testable, and maintainable.

2. Complete Directory Tree

```
alumnilink/
├── .github/
│   └── workflows/
│       ├── ci.yml                    # Automated Linting, Typecheck, Jest & Pytest runs
│       └── deploy.yml                # Staging & production deployment triggers
│
├── app/                              # Next.js 14 App Router (RSC & Client Pages)
│   ├── (auth)/
│   │   ├── login/page.tsx            # Credential login & role redirects
│   │   ├── register/page.tsx         # Account registration with email validation
│   │   └── onboarding/page.tsx       # Multi-step goal & capacity intake wizards
│   ├── student/
│   │   ├── dashboard/page.tsx        # Active tracks, milestones, upcoming video calls
│   │   ├── mentors/page.tsx          # AI-ranked mentor discovery feed & filter pills
│   │   ├── requests/page.tsx         # Outgoing application status tracker
│   │   └── mentorships/[id]/         # Dedicated 12-week LMS classroom workspace
│   ├── alumni/
│   │   ├── dashboard/page.tsx        # Mentee intake desk, slot capacity gauges
│   │   ├── requests/page.tsx         # Review queue with transactional Accept/Decline
│   │   ├── mentorships/[id]/         # Module authoring, assignment review & grading
│   │   └── perks/page.tsx            # Developer tool grants & stipend ledger
│   ├── admin/
│   │   ├── dashboard/page.tsx        # Institutional KPIs & Recharts analytics
│   │   ├── users/page.tsx            # Directory & role-management table
│   │   └── payouts/page.tsx          # Milestone stipend verification & approvals
│   └── api/                          # Route Handlers & Server Actions
│       ├── auth/[...nextauth]/       # Auth.js JWT session callbacks
│       ├── matches/route.ts          # Semantic vector query proxy
│       ├── mentorship-requests/      # Application submission & capacity locks
│       └── lms/                      # Module, assignment & final evaluation APIs
│
├── backend/                          # FastAPI Python Backend (ML & WebSockets)
│   ├── app/
│   │   ├── api/
│   │   │   ├── v1/
│   │   │   │   ├── embeddings.py     # CPU all-MiniLM-L6-v2 vector endpoints
│   │   │   │   └── health.py         # Deep health checks for Neon & memory
│   │   │   └── websocket/
│   │   │       └── signaling.py      # WebRTC SDP offer/answer & ICE relay hub
│   │   ├── core/
│   │   │   ├── config.py             # Pydantic v2 BaseSettings & environment variables
│   │   │   └── security.py           # JWT signature validation from NextAuth cookie
│   │   ├── ml/
│   │   │   └── model_loader.py       # Hugging Face sentence-transformers model instance
│   │   └── main.py                   # Uvicorn entry point & CORS configuration
│   ├── requirements.txt              # Python 3.11 dependencies (torch, transformers, etc.)
│   └── Dockerfile                    # Optimized container build for Render deployment
│
├── components/                       # Reusable React Components (TypeScript)
│   ├── ui/                           # Button, Modal, Badge, Card, Progress primitives
│   ├── profiles/                     # Onboarding forms, skills chips, capacity sliders
│   ├── matching/                     # Discovery cards, vector score pills, alignment drawer
│   ├── classroom/                    # 12-Week accordion, markdown viewer, dropzone box
│   ├── video/                        # WebRTC peer media canvas, control dock, agenda panel
│   └── admin/                        # Cohort metrics, Recharts charts, payout tables
│
├── services/                         # Core Domain Business Logic Layer
│   ├── ProfileService.ts             # Profile mutation & embedding trigger coordination
│   ├── MatchingService.ts            # Hybrid cosine similarity + capacity filter logic
│   ├── MentorshipRequestService.ts   # Concurrency-safe request state transitions
│   ├── LMSService.ts                 # Module authoring, assignment submissions & grading
│   ├── VideoService.ts               # Session duration tracking & WebRTC room lifecycles
│   ├── PayoutService.ts              # Prorated weekly fee calculation (Weeks 4, 8, 12)
│   └── NotificationService.ts        # Nodemailer transactional email dispatches
│
├── repositories/                     # Data Access Layer (Prisma Queries)
│   ├── UserRepository.ts             # Identity lookups & RBAC role queries
│   ├── ProfileRepository.ts          # Student & alumni profile record mutations
│   ├── MatchRepository.ts            # PostgreSQL pgvector raw HNSW distance queries
│   ├── MentorshipRepository.ts       # Atomic interactive transaction locks & cycles
```

```
│   └── LMSRepository.ts                # Modules, assignments & evaluation submissions
│
├── lib/                                # Cross-Cutting Infrastructure Adapters
│   ├── auth.ts                         # NextAuth configuration & cookie options
│   ├── prisma.ts                       # Global PrismaClient singleton instance
│   ├── r2.ts                           # Cloudflare R2 S3 presigned URL generator
│   └── email.ts                        # Nodemailer SMTP transport setup
│
├── validators/                         # Runtime Input Schema Definitions
│   ├── profile.ts                      # Zod schemas for student & alumni onboarding
│   ├── mentorship.ts                   # Request payloads, cycle status enums
│   ├── lms.ts                          # Module syllabus bounds & 0-100 score constraints
│   └── session.ts                      # Session notes & duration validation schemas
│
├── prisma/                             # Database Schemas & Migration History
│   ├── schema.prisma                   # Relational entities, vector(384), and constraints
│   └── migrations/                     # Versioned SQL migration files
│
├── tests/                              # Quality Assurance Test Suites
│   ├── unit/                           # Schema validation, capacity math, fee calculations
│   ├── integration/                    # Prisma transactions, R2 upload auth, WebSockets
│   └── e2e/                            # Playwright end-to-end browser journeys & k6 load scripts
│
├── docs/                               # Project Documentation Suite (HTML Modules)
│   ├── Project_Overview.html           # Domain background, goals & team context
│   ├── BRD.html                        # Business requirements & KPI definitions
│   ├── UX_Requirements.html            # User flows, wireframes & interaction contracts
│   ├── TRD.html                        # Technical architecture & non-functional bounds
│   ├── HLD.html                        # System topology & service components
│   ├── Database_Design.html            # Schemas, ER diagrams & pgvector indexes
│   ├── API_Specification.html          # REST contracts & WebSocket signaling protocol
│   ├── LLD.html                        # Class designs, sequence flows & state machines
│   ├── Security_Design.html            # STRIDE modeling, zero-trust & token policies
│   ├── Testing_Strategy.html           # Verification gates, exit criteria & k6 scripts
│   ├── CICD_Pipeline.html              # GitHub Actions workflows & release checklists
│   ├── Observability.html              # Structured logging, metrics & health checks
│   ├── Deployment_Architecture.html    # Multi-tier hosting & $0 cloud allocation
│   ├── Cost_Analysis.html              # $0 cloud mandate & alumni retention pools
│   └── Project_Roadmap.html            # 12-Week schedule & sprint milestones
│
├── public/                             # Static Frontend Assets
│   ├── brand/                          # SVG logos, institutional crests, icons
│   └── placeholders/                   # Default user avatars & module thumbnails
│
├── .env.example                        # Documented environment template (No secrets)
├── .gitignore                          # Strict exclusion for .env, node_modules, build caches
├── package.json                        # Next.js 14 scripts, dependencies & Prisma hooks
├── tsconfig.json                       # Strict TypeScript compiler options
└── README.md                           # Quickstart guide, architecture overview & badges
```

3. Codebase Architectural Boundaries & Guidelines

> **Governance Rule:** Architectural layers must strictly enforce the single-responsibility principle to prevent code coupling and maintain modularity across the 12-week development cycle.

Directory Path	Architectural Responsibility	Strict Codebase Governance Guidelines
`app/`	Presentation & Route Handlers	Contains React Server Components, client pages, and thin Server Actions. **No raw database queries** are permitted directly inside page components.
`backend/`	FastAPI Services & WebSockets	Dedicated Python environment. Runs local `all-MiniLM-L6-v2` CPU embeddings and WebRTC signaling hubs. Encapsulated in its own Dockerfile.
`services/`	Business Logic & Workflows	Encapsulates core business rules: capacity checks ($N \le 3$), prorated fees, and cycle state machines. Completely decoupled from HTTP frameworks.
`repositories/`	Data Access Layer	The only layer authorized to call `prisma` queries and raw PostgreSQL `pgvector` SQL commands. Facilitates unit test mocking.
`lib/`	Reusable Infrastructure	Stateless third-party adapters (Cloudflare R2 presigned clients, NextAuth configuration, and Nodemailer SMTP transports).
`validators/`	Schema Boundary Validation	Defines runtime Zod and Pydantic validation schemas. Every client mutation is validated here prior to invoking service handlers.
`prisma/`	Relational Schema & Migrations	Contains `schema.prisma` and versioned SQL migrations. Direct database schema modifications outside of Prisma migrations are strictly forbidden.
`docs/`	Formal Engineering Documentation	Houses the complete 15-module documentation suite formatted as standalone, print-ready HTML files.

4. Environment Variable Template (`.env.example`)

The `.env.example` file is committed to version control to document required variables, while actual production secrets are injected through hosting dashboards and never committed:

```
#  ========================================
# ALUMNILINK PRODUCTION ENVIRONMENT TEMPLATE
# ========================================

# Database & Vectors (Neon PostgreSQL 16)
DATABASE_URL="postgresql://user:password@ep-sample-pooler.neon.tech/alumnilink?sslmode=require"
DIRECT_URL="postgresql://user:password@ep-sample.neon.tech/alumnilink?sslmode=require"

# NextAuth / Auth.js Configuration
AUTH_SECRET="0a1b2c3d4e5f6789abcdef0123456789abcdef0123456789abcdef0123456789"
NEXTAUTH_URL="https://alumnilink.vercel.app"

# FastAPI Backend & WebSockets
NEXT_PUBLIC_API_URL="https://alumnilink-backend.onrender.com"
NEXT_PUBLIC_WS_URL="wss://alumnilink-backend.onrender.com"

# Cloudflare R2 Object Storage (Presigned S3 API)
CLOUDFLARE_R2_ACCOUNT_ID="your_cloudflare_account_id"
CLOUDFLARE_R2_ACCESS_KEY="your_r2_access_key"
CLOUDFLARE_R2_SECRET_KEY="your_r2_secret_key"
CLOUDFLARE_R2_BUCKET="alumnilink-production-assets"

# WebRTC STUN / TURN Credentials
METERED_TURN_DOMAIN="alumnilink.metered.ca"
METERED_TURN_SECRET="your_metered_turn_secret_key"

# Transactional Mailer (Nodemailer SMTP)
SMTP_HOST="smtp.resend.com"
SMTP_PORT=465
SMTP_USER="resend"
SMTP_PASSWORD="re_sample_smtp_token_12345"
EMAIL_FROM="notifications@alumnilink.edu"
```

5. Git Workflow & Commit Hygiene

- **Feature Branches:** Work is organized into short-lived branches: `feature/<name>` for new modules and `fix/<name>` for bug resolutions.
- **Secret Scanning:** Pre-commit hooks run `gitleaks` to detect and block credentials, API tokens, or `.env` files before they enter git history.
- **Deterministic Builds:** Package lockfiles (`package-lock.json` and `requirements.txt`) are committed to ensure identical dependency trees across local development and CI/CD containers.

AlumniLink — README & Quickstart

1. Executive Summary

AlumniLink is an outcome-oriented, full-stack mentorship and Learning Management System (LMS) platform designed for higher education institutions. The platform connects undergraduate students with industry alumni based on semantic career trajectories and technical goals rather than shallow departmental tags. By combining local CPU-based vector embeddings, deterministic concurrency-safe capacity locks ($N \le 3$), browser-native 1-on-1 WebRTC video calling, and a 12-week modular classroom, AlumniLink protects mentors from burnout while providing verified academic outcomes under a strict $0 cloud infrastructure spend.

2. Core Architectural Features

- **Local Trajectory Matching:** Computes 384-dimensional dense vectors on CPU via `all-MiniLM-L6-v2` inside FastAPI, performing sub-30ms HNSW cosine similarity search via `pgvector`.
- **Deterministic Capacity Protection:** Concurrency-safe interactive database transactions (mentors `SELECT ... FOR UPDATE`) guarantee alumni can never be overbooked under simultaneous request spikes ($N \in [1, 3]$).
- **Native In-House WebRTC Video:** 1-on-1 peer-to-peer browser video rooms negotiated via FastAPI WebSocket signaling with Google STUN and Metered.ca TURN fallback (zero third-party subscriptions like Zoom or Meet).
- **12-Week Modular LMS Workspace:** Mentors author weekly syllabi, publish lecture notes, and distribute coding assignments, while students upload PDF and code deliverables directly to private Cloudflare R2 buckets.
- **Composite Final Evaluation & Grading:** Mentors evaluate students across Activeness, Homework Assignments, and Final Exam performance (0–100 scale), atomically completing cycles and releasing capacity slots.
- **Alumni Retention & Honorarium Ledger:** Prorated stipend tracking (Weeks 4, 8, 12) with administrator payout approval alongside sponsored developer tool grants (GitHub Copilot and ChatGPT Plus).
- **Automated Attendance & Inactivity Alerts:** Real-time WebSocket connection monitoring logs session durations in `SessionLog` and dispatches automated reminder notifications via Nodemailer SMTP.

3. Production Technology Stack

Layer	Technology	Architectural Scope & Hosting Tier
Frontend UI	Next.js 14 (App Router), TypeScript, Tailwind CSS	Server-rendered dashboards, client video canvas; hosted on Vercel (Hobby Free).
Backend API & Signaling	FastAPI (Python 3.11+), Uvicorn	Asynchronous REST endpoints, WebSocket signaling router; hosted on Render.
Database & Vectors	PostgreSQL 16 + `pgvector`	Relational cycle records, HNSW vector indexing; hosted on Neon Serverless.
Local ML Engine	Hugging Face Sentence-Transformers	`all-MiniLM-L6-v2` running locally on CPU in FastAPI (Zero API cost).
Real-Time Media	Native WebRTC, STUN/TURN Relays	P2P media streams; Google Public STUN + Metered.ca TURN relay fallback.
Object Storage	Cloudflare R2 (S3-Compatible API)	Presigned HMAC upload/download links with **$0 egress bandwidth fees**.
Auth & Validation	NextAuth (JWT), Zod, Pydantic v2	Encrypted HTTP-only session cookies and multi-layer input boundary validation.
Analytics & Mailer	Recharts, Nodemailer (SMTP)	Institutional KPI dashboards and automated transactional email alerts.

4. System User Roles & Permissions (RBAC)

- **Students:** Complete goal onboarding, explore AI-ranked mentor matches, generate editable AI outreach drafts, attend WebRTC video sessions, submit LMS assignments, and track weekly progress.
- **Alumni Mentors:** Manage mentee capacity limits ($N \le 3$), review incoming applications, host 1-on-1 video classes, publish course modules, grade deliverables, and claim stipend/tool grants.
- **Administrators:** Monitor cohort KPIs, track mentor pool utilization rates, inspect attendance duration logs, and approve milestone stipend disbursements.

5. Local Engineering Setup & Installation

Ensure you have **Node.js 20 LTS**, **Python 3.11+**, and a running **PostgreSQL 16** instance with the `pgvector` extension enabled.

5.1 Repository Setup

```
# 1. Clone the monorepo
git clone https://github.com/team-g239/alumnilink.git
cd alumnilink

# 2. Install Next.js frontend dependencies
npm install

# 3. Setup and activate Python virtual environment for FastAPI backend
cd backend
python3 -m venv venv
source venv/bin/activate        # On Windows: venv\Scripts\activate
pip install -r requirements.txt
cd ..
```

5.2 Database Initialization

```
# Generate Prisma Client and apply database migrations
npx prisma generate
npx prisma migrate dev --name init
```

5.3 Running the Dual-Stack Local Environment

```
# Terminal 1: Launch FastAPI Backend & Signaling Server (Port 8000)
cd backend
source venv/bin/activate
uvicorn app.main:app --reload --port 8000

# Terminal 2: Launch Next.js 14 Frontend UI (Port 3000)
npm run dev
```

6. Environment Variables Configuration (`.env`)

Create a `.env` file in the project root based on the template below. Real production secrets must never be committed to source control:

```
# Database Connection (Neon PostgreSQL 16 with pgvector)
DATABASE_URL="postgresql://user:password@ep-sample-pooler.neon.tech/alumnilink?sslmode=require"
DIRECT_URL="postgresql://user:password@ep-sample.neon.tech/alumnilink?sslmode=require"

# NextAuth / Auth.js Configuration
AUTH_SECRET="your_32_byte_symmetric_hex_jwt_secret"
NEXTAUTH_URL="http://localhost:3000"

# Decoupled FastAPI Backend Endpoint
NEXT_PUBLIC_API_URL="http://127.0.0.1:8000"
NEXT_PUBLIC_WS_URL="ws://127.0.0.1:8000"

# Cloudflare R2 S3 Presigned Uploads
CLOUDFLARE_R2_ACCOUNT_ID="your_cloudflare_account_id"
CLOUDFLARE_R2_ACCESS_KEY="your_r2_access_key"
CLOUDFLARE_R2_SECRET_KEY="your_r2_secret_key"
CLOUDFLARE_R2_BUCKET="alumnilink-production-assets"

# WebRTC STUN / TURN Network Relays
METERED_TURN_DOMAIN="alumnilink.metered.ca"
METERED_TURN_SECRET="your_metered_turn_secret_key"

# Transactional Notifications (Nodemailer SMTP)
SMTP_HOST="smtp.resend.com"
SMTP_PORT=465
SMTP_USER="resend"
SMTP_PASSWORD="your_smtp_password"
EMAIL_FROM="notifications@alumnilink.edu"
```

7. Quality Assurance & Verification Checks

Before submitting pull requests or merging into `main`, ensure all automated verification scripts pass cleanly:

```
# 1. Run ESLint syntax & style rules
npm run lint

# 2. Run strict TypeScript compiler typechecking
npm run typecheck

# 3. Run frontend Jest & backend Pytest test suites
npm test
cd backend && pytest && cd ..

# 4. Compile production Next.js 14 bundle
npm run build
```

> **Institutional Capstone Submission Gate:** The AlumniLink repository adheres to a zero-cost production mandate, full type safety, and concurrency-safe transactional workflows. All 15 engineering documentation modules are maintained in the `docs/` directory for evaluation and audit.

Architecture Decision Records (ADR) — AlumniLink

1. ADR Governance & Purpose

Architecture Decision Records document the key architectural choices made during the development of AlumniLink. Each record outlines the technical context, the alternatives considered, the final decision, and the operational trade-offs evaluated by the two-person engineering team across the 12-week schedule.

2. Architectural Decision Register

ADR-001: Decoupled Next.js 14 Frontend and FastAPI (Python) Backend

ACCEPTE

Context: AlumniLink requires a responsive, server-rendered dashboard with role-based routing (Next.js 14) alongside high-throughput asynchronous workloads, including in-memory local vector embedding generation and real-time WebSocket signaling for WebRTC video calling.

Decision: Deploy a decoupled architecture featuring Next.js 14 (TypeScript) for presentation and edge routing, and FastAPI (Python 3.11) for machine learning execution and WebSocket signaling hub management.

Consequences & Trade-Offs: Requires maintaining cross-language type contracts (TypeScript interfaces aligned with Pydantic v2 schemas) and managing two deployment targets (Vercel and Render) within free-tier quotas. However, it isolates CPU-intensive vector calculations from user-facing frontend routes and eliminates external ML API bills.

ADR-002: Managed PostgreSQL 16 with Prisma ORM & Row-Level Locking

ACCEPTE

Context: Mentorship matching and cycle management involve relational entities (users, profiles, LMS modules, assignments, evaluations) requiring ACID transaction guarantees to prevent mentor overbooking.

Decision: Use PostgreSQL 16 managed on Neon Serverless accessed via Prisma ORM for type-safe schema definitions and interactive transactions.

Consequences & Trade-Offs: Requires strict connection pooling (Neon PgBouncer) to prevent connection pool exhaustion during concurrent bursts. Provides row-level locking (`SELECT ... FOR UPDATE`) required to serialize concurrent acceptance requests.

ADR-003: In-Database Vector Storage via PostgreSQL pgvector & HNSW Indexing

ACCEPTE

Context: Storing trajectory embeddings in a dedicated external vector database (e.g., Pinecone, Weaviate) introduces distributed latency, multi-system synchronization overhead, and potential service charges.

Decision: Store 384-dimensional dense vectors directly within PostgreSQL using the native `pgvector` extension indexed via Hierarchical Navigable Small World (HNSW) cosine operators (`<=>`).

Consequences & Trade-Offs: Consumes storage within Neon's 0.5 GB free-tier allocation. Enables single-query hybrid search where cosine similarity ranking and hard relational filters (`isAvailable = true AND activeMentees < maxMentees`) execute in a single SQL operation under 50ms.

ADR-004: Local CPU Embeddings via Hugging Face all-MiniLM-L6-v2

ACCEPTE

Context: Relying on cloud embedding APIs (OpenAI text-embedding-ada-002 or Gemini Embeddings) creates network dependencies, per-token billing, and rate-limit risks during student onboarding spikes.

Decision: Run the open-source sentence-transformers model `all-MiniLM-L6-v2` locally in memory inside FastAPI on CPU.

Consequences & Trade-Offs: Adds approximately 120 MB memory overhead to the Python backend container. Generates 384-dimensional embeddings locally in under 30ms with zero token expenses and protects student career statements from leaving the hosting infrastructure.

ADR-005: Deterministic Business Rules Override AI Probabilistic Outputs

ACCEPTE

Context: Generative AI models and semantic vector distances can hallucinate, produce false positives, or recommend alumni whose current bandwidth is exhausted.

Decision: AI vector matching functions solely as an informational candidate-ranking engine; deterministic relational logic strictly dictates eligibility and application permissions.

Consequences & Trade-Offs: Students cannot be matched with unavailable alumni regardless of trajectory similarity score. Prevents mentor fatigue and guarantees administrative compliance.

ADR-006: Two-Checkpoint Concurrency-Safe Capacity Locking

ACCEPTE

Context: A popular alumnus with 1 open seat could receive simultaneous acceptance clicks from multiple pending student applications, resulting in a race condition.

Decision: Filter out full-capacity mentors during the initial recommendation query, and re-check capacity atomically using `SELECT ... FOR UPDATE` inside a `SERIALIZABLE` database transaction upon request acceptance.

Consequences & Trade-Offs: A second student attempting to claim the same seat receives an immediate HTTP 409 Conflict (`CAPACITY EXCEEDED`) instead of creating an unauthorized active cycle. Strictly preserves the mentor capacity boundary ($N \in [1, 3]$) under concurrency.

ADR-007: Native In-House WebRTC 1-on-1 Video Mesh over Third-Party SDKs

ACCEPTE

Context: Commercial video APIs (Twilio Video, Agora, Daily.co) charge per-participant-minute rates, while external redirection (Zoom, Google Meet links) breaks session tracking and risks dropouts.

Decision: Build native peer-to-peer browser video rooms utilizing standard WebRTC primitives, WebSocket signaling via FastAPI, and Metered.ca TURN fallback.

Consequences & Trade-Offs: Limits native multi-party calling to 1-on-1 sessions (mesh topologies degrade at $N > 2$ due to client upload saturation). Completely eliminates third-party video licensing bills, enforces attendance logging via WebSocket heartbeats, and keeps communication in-app.

ADR-008: Cloudflare R2 Object Storage for LMS Courseware & Submissions

ACCEPTE

Context: Mentor lecture notes and student assignment code/PDF uploads consume significant storage. Storing files as PostgreSQL bytea blobs degrades database performance, while AWS S3 charges egress bandwidth fees.

Decision: Store all course materials and homework files in private Cloudflare R2 buckets accessed via short-lived (15-minute) HMAC presigned URLs.

Consequences & Trade-Offs: Requires generating presigned URLs server-side for both upload and download operations. Delivers 10 GB of free storage with zero egress download bandwidth fees, protecting database compute.

ADR-009: Graceful Fallback Workflows for Auxiliary Generative AI

ACCEPTE

Context: External LLM APIs (Gemini 1.5 Flash) used for outreach intro drafts, session agendas, and notes summarization may face provider outages, quota exhaustion, or network latency.

Decision: Auxiliary AI capabilities are designed as non-blocking enhancements; all related UI forms provide manual editable textareas by default.

Consequences & Trade-Offs: If an AI drafting request times out or returns HTTP 503, the application remains fully usable without breaking student outreach or session note logging.

ADR-010: Single-Institution Pilot Validation Scope

ACCEPTE

Context: Attempting to support multi-institution tenant partitioning, dynamic white-label branding, and custom SSO integrations during initial development risks scope creep for a two-person engineering team.

Decision: Scope the 12-week MVP to a single partner university pilot cohort (25 students, 10 mentors) with a shared database schema.

Consequences & Trade-Offs: Multi-tenant isolation is deferred to subsequent architectural phases. Allows the team to validate trajectory matching relevance, WebRTC connection rates, and LMS module completion rates.